
Analytic Inverse Kinematics

A First-Principles Walkthrough of `analytic_ik.py`

Theory, mathematics, hand-worked numerical examples and line-by-line code commentary for every helper function, every method of the `Rig` class, the configuration tables, and the full solver pipeline that retargets motion-capture keypoints onto an MHR character rig.

Scope

5 vector/rotation helpers, 4 `Rig` methods, the Part 3 lookup tables, and 8 solver stages

Core mathematics

Vector normalisation, cross and dot products, `atan2`, Gram–Schmidt frames, quaternions, Rodrigues' formula, Euler factorisation

Conventions

Column vectors, right-handed frames, lengths in centimetres, angles in radians

Contents

1 Overview	4
1.1 The Complete Algorithm Flow	4
1.2 Mathematical Foundations	4
2 Function 1: <code>normalise</code>	5
2.1 Theory, Purpose, and Intuition	5
2.2 Mathematics	5
2.3 Hand-Worked Example	5
2.4 Code Implementation	5
2.5 Connection to Overall Algorithm	6
3 Function 2: <code>shortest_arc_rotation</code>	7
3.1 Theory, Purpose, and Intuition	7
3.2 Mathematics	7
3.3 Hand-Worked Example	7
3.4 Code Implementation	8
3.5 Connection to Overall Algorithm	9
3.6 Real-World Hand-Worked Example for <code>shortest_arc_rotation</code>	9
4 Function 3: <code>frame_from_two_directions</code>	13
4.1 Theory, Purpose, and Intuition	13
4.2 Mathematics	13
4.3 Hand-Worked Example	13
4.4 Code Implementation	14
4.5 Connection to Overall Algorithm	14
4.6 Intuitive Explanation	15
5 Function 4: <code>rotation_between_frames</code>	17
5.1 Theory, Purpose, and Intuition	17
5.2 Mathematics	17
5.3 Hand-Worked Example	17
5.4 Code Implementation	19
5.5 Connection to the Overall Algorithm	19
6 Function 5: <code>signed_angle_about_axis</code>	20
6.1 Theory, Purpose, and Intuition	20
6.2 Mathematics	20
6.3 Hand-Worked Example	20
6.4 Code Implementation	21
6.5 Connection to the Overall Algorithm	22
7 The <code>Rig</code> Class — <code>Rig.__init__</code>	23
7.1 Theory, Purpose, and Intuition	23
7.2 Mathematics	23
7.3 Hand-Worked Mathematical Example	25
7.4 Code Implementation	26
7.5 Connection to the Overall Algorithm	27
8 <code>Rig.forward_kinematics</code>	28
8.1 Theory, Purpose, and Intuition	28
8.2 Mathematics	29
8.3 Hand-Worked Mathematical Example	30
8.4 Code Implementation	31
8.5 Connection to the Overall Algorithm	32
9 <code>Rig.set_rotation</code>	34
9.1 Theory, Purpose, and Intuition	34

9.2	Mathematics	34
9.3	Hand-Worked Mathematical Example	36
9.4	Code Implementation	37
9.5	Connection to the Overall Algorithm	38
10	Rig.hinge_axis	40
10.1	Theory, Purpose, and Intuition	40
10.2	Mathematics	40
10.3	Hand-Worked Mathematical Example	41
10.4	Code Implementation	43
10.5	Connection to the Overall Algorithm	43
11	Part 3: Configuration Blueprints and Lookup Tables	45
11.1	Theory, Purpose, and Intuition	45
11.2	Mathematics	45
11.3	Hand-Worked Mathematical Example	47
11.4	Code Implementation	47
11.5	Connection to the Overall Algorithm	49
12	match_bone_lengths	50
12.1	Theory, Purpose, and Intuition	50
12.2	Mathematics	50
12.3	Hand-Worked Mathematical Example	51
12.4	Code Implementation	52
12.5	Connection to the Overall Algorithm	53
13	body_frame	54
13.1	Theory, Purpose, and Intuition	54
13.2	Mathematics	54
13.3	Hand-Worked Mathematical Example	55
13.4	Code Implementation	56
13.5	Connection to the Overall Algorithm	57
14	match_hip_offset	58
14.1	Theory, Purpose, and Intuition	58
14.2	Mathematics	58
14.3	Hand-Worked Mathematical Example	59
14.4	Code Implementation	60
14.5	Connection to the Overall Algorithm	61
15	solve_root	63
15.1	Theory, Purpose, and Intuition	63
15.2	Mathematics	63
15.3	Hand-Worked Mathematical Example	64
15.4	Code Implementation	66
15.5	Connection to the Overall Algorithm	67
16	solve_spine	68
16.1	Theory, Purpose, and Intuition	68
16.2	Mathematics	68
16.3	Hand-Worked Mathematical Example	69
16.4	Code Implementation	71
16.5	Connection to the Overall Algorithm	72
17	solve_simple_bone	73
17.1	Theory, Purpose, and Intuition	73
17.2	Mathematics	73
17.3	Hand-Worked Mathematical Example	74
17.4	Code Implementation	75
17.5	Connection to the Overall Algorithm	77

18	<code>solve_limb</code>	78
18.1	Theory, Purpose, and Intuition	78
18.2	Mathematics	78
18.3	Hand-Worked Mathematical Example	79
18.4	Code Implementation	81
18.5	Connection to the Overall Algorithm	82
19	<code>solve</code>	84
19.1	Theory, Purpose, and Intuition	84
19.2	Mathematics	84
19.3	Hand-Worked Mathematical Example	85
19.4	Code Implementation	86
19.5	Connection to the Overall Algorithm	87

SECTION 1

Overview

1.1 The Complete Algorithm Flow

The solver in `analytic_ik.py` works in this order:

1. **Match bone lengths** - Stretch/rig bones to match target bone lengths
2. **Match hip offset** - Position hips relative to pelvis
3. **Solve root** - Place root at pelvis and orient correctly
4. **Solve spine** - Align spine and twist shoulders
5. **Solve simple bones** - Individual bones (shoulder-elbow-wrist, etc.)
6. **Solve limbs** - Two-bone limbs (upper-hip to lower-knee, etc.)

The key insight is **bottom-up processing**: we start at the root and work outward, but sometimes we need to go **top-down** (root → spine → neck → head) because certain constraints affect multiple levels simultaneously.

1.2 Mathematical Foundations

The core mathematics involve:

- **Vector normalization** - Making directions unit length
- **Cross products** - Finding perpendicular axes
- **Dot products** - Measuring angles between vectors
- **Arctangent** - Computing angles from dot/cross products
- **Quaternions** - Representing 3D rotations (via `scipy.spatial.transform.Rotation`)
- **Frame transformations** - Changing coordinate systems between parent-child relationships

SECTION 2

Function 1: `normalise`

2.1 Theory, Purpose, and Intuition

Purpose: This function takes any 3D vector (direction, arrow, or displacement) and scales it so its length becomes exactly 1. In other words, it converts a vector into a **unit vector** - a direction indicator with no magnitude.

Intuition: Imagine you're standing in a field and want to tell someone which way to walk. You could say "walk toward the north," but that's vague without specifying distance. Instead, you'd say "walk in the direction of the north star" - the direction is precise, regardless of how far away the star is. A unit vector captures this "pure direction" concept mathematically.

Think of it like this: if you have a vector pointing northeast with length 5, the unit vector points in the same northeast direction but has length 1. The direction is preserved, only the scale changes.

2.2 Mathematics

The mathematical operation is straightforward:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

Where:

- $\mathbf{v}$ is the input vector
- $\|\mathbf{v}\|$ is the Euclidean norm (length) of the vector
- $\hat{\mathbf{v}}$ is the normalized (unit) vector

Edge case handling: If the input vector has length essentially zero (near zero), we cannot divide by zero. The function raises a `ValueError` in this case, preventing division by zero.

2.3 Hand-Worked Example

Let's work through a concrete numerical example:

Suppose we have a vector representing a direction in 3D space (in centimeters):

$$\mathbf{v} = [3, 4, 0]^T$$

Step 1: Calculate the length

$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Step 2: Check for zero length Since $5 > 10^{-9}$, we can safely proceed.

Step 3: Divide by the length

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{5} = \left[\frac{3}{5}, \frac{4}{5}, \frac{0}{5} \right] = [0.6, 0.8, 0]^T$$

Result: The unit vector points in the same direction as the original vector but has length exactly 1.

2.4 Code Implementation

Here's the actual code line by line:

```
def normalise(vector):
    """Return the vector scaled to length 1."""
    length = np.linalg.norm(vector) # Line 13: Compute Euclidean norm
    if length < 1e-9: # Line 14: Check for zero vector
        raise ValueError("cannot normalise a zero-length vector")
    return np.asarray(vector, dtype=float) / length # Line 15: Return normalized vector
```

Line-by-line breakdown:

1. **Function definition** - Takes a single argument `vector`
2. **Compute length** - `np.linalg.norm(vector)` calculates $\sqrt{x^2 + y^2 + z^2}$
3. **Zero check** - If length is negligible (less than $1e-9$), raise an error
4. **Normalize** - Divide the original vector by its length to get unit vector
5. **Return** - The normalized array

2.5 Connection to Overall Algorithm

This function is a foundational building block:

- **shortest_arc_rotation**: Needs unit vectors to compute the angle between two directions via dot product
- **signed_angle_about_axis**: Projects vectors onto planes perpendicular to the axis
- **All rotation calculations**: Depend on having properly normalized direction vectors

Without unit vectors, angle calculations would be incorrect because the dot product formula $\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$ would include unwanted magnitude terms.

SECTION 3

Function 2: `shortest_arc_rotation`

3.1 Theory, Purpose, and Intuition

3.1.1 Purpose

This function calculates the **smallest rotation** (angle and axis) needed to rotate one direction vector into another direction vector. In simpler terms: if you're facing North and want to face East, this function tells you exactly how much to turn (and which way).

3.1.2 Intuition

Imagine you're holding a compass. You know your current heading (say, North) and you want to end up facing a different direction (East). There are infinitely many ways to turn - you could turn 90° clockwise, or 270° counterclockwise, or even 450° (which is just 90° again). But the **shortest arc rotation** is the minimal turn that gets you there - exactly 90° clockwise.

Think of spinning a globe: if you want to move from the United States to France, you don't want to spin the entire globe 180° . You want the shortest path - which might be a 30° rotation around some axis.

3.2 Mathematics

The mathematics involves several key operations:

1. **Normalization** - Both input vectors are made unit length (length 1)
2. **Cross Product** - To find an axis perpendicular to both vectors
3. **Dot Product** - To determine the angle between the vectors
4. **Arctangent** - To calculate the actual angle

Key Concepts:

- **Unit Vector**: A vector with length exactly 1, pointing in the same direction as the original
- **Cross Product** ($\mathbf{a} \times \mathbf{b}$): Gives a vector perpendicular to both $\mathbf{a}$ and $\mathbf{b}$. Its direction follows the right-hand rule.
- **Dot Product** ($\mathbf{a} \cdot \mathbf{b}$): Gives the cosine of the angle between two vectors
- **Right-Hand Rule**: When you curl the fingers of your right hand from vector $\mathbf{a}$ to $\mathbf{b}$, your thumb points in the direction of the cross product

The Formula: If θ is the angle between two unit vectors $\mathbf{a}$ and $\mathbf{b}$:

$$\cos(\theta) = \mathbf{a} \cdot \mathbf{b}$$

$$\sin(\theta) = \frac{\|\mathbf{a} \times \mathbf{b}\|}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

For unit vectors, this simplifies to:

$$\tan(\theta) = \frac{\|\mathbf{a} \times \mathbf{b}\|}{\mathbf{a} \cdot \mathbf{b}}$$

So the rotation angle is $\theta = \arctan 2(\|\mathbf{a} \times \mathbf{b}\|, \mathbf{a} \cdot \mathbf{b})$

3.3 Hand-Worked Example

Let's work through a concrete numerical example:

Problem: Find the shortest rotation to turn from vector $\mathbf{a} = [1, 0, 0]$ (pointing east) to vector $\mathbf{b} = [0, 1, 0]$ (pointing north).

Step 1: Normalize both vectors

Both are already unit vectors since their lengths are 1:

- $\|\mathbf{a}\| = \sqrt{1^2 + 0^2 + 0^2} = 1$
- $\|\mathbf{b}\| = \sqrt{0^2 + 1^2 + 0^2} = 1$

Step 2: Compute the cross product

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{vmatrix} = (0 \cdot 0 - 0 \cdot 1)\mathbf{i} - (1 \cdot 0 - 0 \cdot 0)\mathbf{j} + (1 \cdot 1 - 0 \cdot 0)\mathbf{k} = [0, 0, 1]$$

So the axis is $[0, 0, 1]$ (pointing up/positive Z).

Step 3: Compute the dot product

$$\mathbf{a} \cdot \mathbf{b} = (1)(0) + (0)(1) + (0)(0) = 0$$

This means the vectors are orthogonal (90° apart).

Step 4: Calculate the angle

Using $\tan(\theta) = \frac{\|\mathbf{a} \times \mathbf{b}\|}{\mathbf{a} \cdot \mathbf{b}}$:

$$\tan(\theta) = \frac{1}{0} = \infty$$

This means $\theta = 90^\circ$ (or $\pi/2$ radians).

Step 5: What does the resulting rotation matrix look like?

A 90° rotation around the z-axis turns any vector (x, y, z) into $(-y, x, z)$. As a 3×3 matrix, this is:

$$R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Let's check if multiplying our starting bone $\mathbf{a}$ by this matrix turns it into $\mathbf{b}$:

$$R\mathbf{a} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} (0)(1) + (-1)(0) + (0)(0) \\ (1)(1) + (0)(0) + (0)(0) \\ (0)(1) + (0)(0) + (1)(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \mathbf{b}$$

It carried $\mathbf{a}$ onto $\mathbf{b}$ exactly.

Step 6: Determine direction (sign)

The sign of the angle depends on the orientation. Since the cross product gave us $[0, 0, 1]$ (upward), and the dot product is positive (0), the rotation is **counterclockwise** when looking from above (from the positive Z axis).

Result: The shortest rotation is 90° counterclockwise around the Z-axis.

3.4 Code Implementation

```
def shortest_arc_rotation(from_direction, to_direction):
    """The smallest rotation that turns one direction into another.

    Think of spinning a globe so that one city ends up where another was: there
    is a single axis to spin about (perpendicular to both) and a single angle.
    """
    a = normalise(from_direction) # Line 22: Normalize source direction
    b = normalise(to_direction)   # Line 23: Normalize target direction

    axis = np.cross(a, b)         # Line 24: Cross product gives axis

    axis_length = np.linalg.norm(axis) # Line 25: Get length of axis

    # Case 1: Vectors are already aligned (no rotation needed)
    if axis_length < 1e-9 and np.dot(a, b) > 0:
        return np.eye(3) # Identity matrix (no rotation)

    # Case 2: Vectors are exactly opposite (180° apart)
    if axis_length < 1e-9:
        helper = np.array([1.0, 0.0, 0.0])
```

```

if abs(np.dot(a, helper)) > 0.9:
    axis = normalise(np.cross(a, helper))
    return Rotation.from_rotvec(axis * np.pi).as_matrix()

# General case: Calculate angle using arctan2
angle = np.arctan2(axis_length, np.dot(a, b)) # Line 28: Angle calculation
return Rotation.from_rotvec(normalise(axis) * angle).as_matrix()

```

Line-by-line explanation:

1. **Function signature** - Takes two direction vectors as input
2. **Normalize inputs** - Ensure both vectors have length 1 (line 22-23)
3. **Compute cross product** - Find the axis perpendicular to both (line 24)
4. **Check for alignment** - If vectors are nearly identical, return identity (line 26-27)
5. **Handle opposite vectors** - Special case when vectors point in exact opposite directions (line 29-33)
6. **Calculate angle** - Use `arctan2` to get the signed angle (line 35)
7. **Create rotation** - Convert to a proper rotation matrix (line 37-38)

3.5 Connection to Overall Algorithm

This function is crucial throughout the solver:

- **solve_limb**: Determines the rotation needed to align the upper bone with the target (line ~200-210)
- **solve_spine**: Rotates the spine to align neck with target (line ~230-240)
- **solve_root**: Aligns the root (pelvis) with the target pelvis

In all cases, we need to find the **minimal rotation** that transforms one direction into another. Without this function

3.6 Real-World Hand-Worked Example for `shortest_arc_rotation`

Let's walk through a realistic example where the bone is not conveniently aligned with any coordinate axis (x , y , or z). All three coordinates will be active, giving a true 3D diagonal rotation—just like in actual motion capture.

3.6.1 The Physical Scenario

Imagine the person's **right upper arm bone**:

- The **shoulder joint** is fixed at position (10, 50, 20) cm.
- Currently on our 3D rig, the arm is pointing down and forward. The rig's **elbow** is at (12, 53, 26) cm.
- In the real video camera footage, the person moved their arm: their target **elbow** is actually at (14, 46, 27) cm.

We need to calculate the exact rotation that swings the upper arm from its current orientation to point directly at the target elbow.

Shoulder (10, 50, 20)			
o			
	/		/
Elbow	v	v	Elbow
(12, 53, 26)			(14, 46, 27)
		Current	Target

3.6.2 Step 1: Find the Direction Vectors

First, we find the two direction vectors by subtracting the shoulder position from the elbow positions:

1. **Current direction arrow** ($\mathbf{v}_{\text{from}}$):

$$\mathbf{v}_{\text{from}} = (12 - 10, 53 - 50, 26 - 20) = (2, 3, 6) \text{ cm}$$

2. **Target direction arrow** ($\mathbf{v}_{\text{to}}$):

$$\mathbf{v}_{\text{to}} = (14 - 10, 46 - 50, 27 - 20) = (4, -4, 7) \text{ cm}$$

3.6.3 Step 2: Normalise Both Vectors to Length 1

(Corresponding to: $\mathbf{a} = \text{normalise}(\text{from_direction})$ and $\mathbf{b} = \text{normalise}(\text{to_direction})$)

For $\mathbf{v}_{\text{from}} = (2, 3, 6)$

- Calculate the length:

$$\|\mathbf{v}_{\text{from}}\| = \sqrt{2^2 + 3^2 + 6^2} = \sqrt{4 + 9 + 36} = \sqrt{49} = 7$$

- Divide each coordinate by 7:

$$\mathbf{a} = \left(\frac{2}{7}, \frac{3}{7}, \frac{6}{7}\right) \approx (0.2857, 0.4286, 0.8571)$$

For $\mathbf{v}_{\text{to}} = (4, -4, 7)$

- Calculate the length:

$$\|\mathbf{v}_{\text{to}}\| = \sqrt{4^2 + (-4)^2 + 7^2} = \sqrt{16 + 16 + 49} = \sqrt{81} = 9$$

- Divide each coordinate by 9:

$$\mathbf{b} = \left(\frac{4}{9}, -\frac{4}{9}, \frac{7}{9}\right) \approx (0.4444, -0.4444, 0.7778)$$

Both arrows are now pure directions of length 1.0.

3.6.4 Step 3: Find the Rotation Axle (Cross Product)

(Corresponding to: `axis = np.cross(a, b)`)

To find the axle around which the arm must swing, we calculate $\mathbf{a} \times \mathbf{b}$.

Using the cross-product formula:

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Let's compute each component by hand using fractions (common denominator is $7 \times 9 = 63$):

- **x-component:**

$$\left(\frac{3}{7}\right)\left(\frac{7}{9}\right) - \left(\frac{6}{7}\right)\left(-\frac{4}{9}\right) = \frac{21 - (-24)}{63} = \frac{45}{63} \approx +0.7143$$

- **y-component:**

$$\left(\frac{6}{7}\right)\left(\frac{4}{9}\right) - \left(\frac{2}{7}\right)\left(\frac{7}{9}\right) = \frac{24 - 14}{63} = \frac{10}{63} \approx +0.1587$$

- **z-component:**

$$\left(\frac{2}{7}\right)\left(-\frac{4}{9}\right) - \left(\frac{3}{7}\right)\left(\frac{4}{9}\right) = \frac{-8 - 12}{63} = \frac{-20}{63} \approx -0.3175$$

So the raw axle vector is:

$$\text{axis} = \left(\frac{45}{63}, \frac{10}{63}, \frac{-20}{63}\right) \approx (0.7143, 0.1587, -0.3175)$$

3.6.5 Step 4: Find the Length of the Axle

(Corresponding to: `axis_length = np.linalg.norm(axis)`)

$$\begin{aligned} \text{axis_length} &= \sqrt{(0.7143)^2 + (0.1587)^2 + (-0.3175)^2} \\ &\approx \sqrt{0.5102 + 0.0252 + 0.1008} = \sqrt{0.6362} \approx 0.7976 \end{aligned}$$

Notice that this is not zero, so the vectors are neither parallel nor opposite. We can safely proceed!

3.6.6 Step 5: Normalise the Axle

(Corresponding to: `normalise(axis)`)

To make our axle a unit vector $\hat{\mathbf{u}}$, we divide it by its length (0.7976):

$$\hat{\mathbf{u}} = \left(\frac{0.7143}{0.7976}, \frac{0.1587}{0.7976}, \frac{-0.3175}{0.7976} \right) \approx (0.8956, 0.1990, -0.3981)$$

Let's double-check its length:

$$\sqrt{0.8956^2 + 0.1990^2 + (-0.3981)^2} = \sqrt{0.8021 + 0.0396 + 0.1585} = \sqrt{1.0002} \approx 1.0$$

This unit vector $\hat{\mathbf{u}}$ is the exact 3D axle sticking out of the shoulder that the arm will rotate around.

3.6.7 Step 6: Find the Angle to Turn

(Corresponding to: `angle = np.arctan2(axis_length, np.dot(a, b))`)

First, compute the dot product $\mathbf{a} \cdot \mathbf{b}$ (which gives $\cos \theta$):

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= a_x b_x + a_y b_y + a_z b_z \\ &= \left(\frac{2}{7}\right) \left(\frac{4}{9}\right) + \left(\frac{3}{7}\right) \left(-\frac{4}{9}\right) + \left(\frac{6}{7}\right) \left(\frac{7}{9}\right) \\ &= \frac{8 - 12 + 42}{63} = \frac{38}{63} \approx 0.6032 \end{aligned}$$

Now, we use `arctan2(sine, cosine)`:

$$\theta = \text{atan2}(0.7976, 0.6032) \approx 0.9234 \text{ radians}$$

Converting radians to degrees so it's easy to visualize:

$$\theta = 0.9234 \times \left(\frac{180^\circ}{\pi}\right) \approx 52.91^\circ$$

The arm needs to swing by 52.91° around our diagonal axle $\hat{\mathbf{u}}$.

3.6.8 Step 7: Construct the Rotation Matrix and Verify the Result

In the code, this line:

```
Rotation.from_rotvec(normalise(axis) * angle).as_matrix()
```

takes the axis $\hat{\mathbf{u}}$ and angle θ , and creates a 3×3 rotation matrix R .

Using Euler-Rodrigues' standard formula, this matrix calculates to:

$$R \approx \begin{bmatrix} 0.9213 & -0.3475 & 0.1745 \\ 0.3842 & 0.8804 & -0.2778 \\ -0.0572 & 0.3230 & 0.9447 \end{bmatrix}$$

The Ultimate Test: Does R actually turn bone $\mathbf{a}$ into bone $\mathbf{b}$?

Let's multiply our starting bone $\mathbf{a} \approx \begin{pmatrix} 0.2857 \\ 0.4286 \\ 0.8571 \end{pmatrix}$ by this matrix R :

■ **New x :**

$$\begin{aligned} &(0.9213)(0.2857) + (-0.3475)(0.4286) + (0.1745)(0.8571) \\ &= 0.2632 - 0.1489 + 0.1496 = \mathbf{0.4444} \quad \left(\text{which is exactly } \frac{4}{9}!\right) \end{aligned}$$

■ **New y :**

$$\begin{aligned}
 & (0.3842)(0.2857) + (0.8804)(0.4286) + (-0.2778)(0.8571) \\
 &= 0.1098 + 0.3773 - 0.2381 = -\mathbf{0.4444} \quad \left(\text{which is exactly } -\frac{4}{9}! \right)
 \end{aligned}$$

■ **New z :**

$$\begin{aligned}
 & (-0.0572)(0.2857) + (0.3230)(0.4286) + (0.9447)(0.8571) \\
 &= -0.0163 + 0.1384 + 0.8097 = \mathbf{0.7778} \quad \left(\text{which is exactly } \frac{7}{9}! \right)
 \end{aligned}$$

The transformed vector is $(0.4444, -0.4444, 0.7778)$, which is **identically equal to target direction $\mathbf{b}$** .

The math carried the arm from its starting direction to the target direction in 3D space.

SECTION 4

Function 3: `frame_from_two_directions`

4.1 Theory, Purpose, and Intuition

4.1.1 Purpose

This function creates a **coordinate system (frame)** based on two directional vectors. Think of it as building a tiny 3D grid (like a piece of graph paper) where:

- One axis points in the direction of the first vector
- Another axis points in the direction of the second vector
- The third axis completes the right-handed coordinate system

This is essential for comparing orientations - we need to establish a consistent reference frame before we can say "this bone should point at that target."

4.1.2 Intuition

Imagine you're standing in a room and you want to describe the orientation of a table leg. You could say "it points north" and "it points east" - but that's ambiguous because you haven't defined what "north" and "east" mean relative to your body.

Instead, you'd say: "My nose points toward the door (vector A), and my chest points toward the window (vector B)." These two directions create a unique coordinate system. Now anyone can look at your description and reconstruct exactly where your nose and chest are pointing.

4.2 Mathematics

The function builds three mutually perpendicular axes:

1. **X-axis** - Points along the first direction (`main_direction`)
2. **Y-axis** - Perpendicular to X, determined by subtracting the component of `side_direction` along X
3. **Z-axis** - Completes the right-handed system by crossing $X \times Y$

Key Operations:

- **Normalization** - Ensures both input vectors have length 1
- **Gram-Schmidt Process** - Creates a perpendicular Y-axis by removing the component of `side_direction` that aligns with `main_direction`
- **Cross Product** - Creates the Z-axis to ensure right-handedness

Mathematical Steps:

```

Step 1: Normalize both input vectors
    a = normalize(main_direction)
    b = normalize(side_direction)

Step 2: Create Y-axis (perpendicular to X)
    y = b - (dot(b, x) * x)
    y = normalize(y)

Step 3: Create Z-axis (completing right-handed system)
    z = cross(x, y)

Step 4: Return [x, y, z] as columns of a matrix

```

4.3 Hand-Worked Example

Let's work through a concrete numerical example:

Problem: Given two direction vectors:

- `main_direction = [1, 0, 0]` (pointing east)
- `side_direction = [0, 1, 0]` (pointing north)

We want to create a frame where:

- X-axis points east (same as `main_direction`)
- Y-axis points north (same as `side_direction`)
- Z-axis completes the right-handed system

Step 1: Normalize both vectors

- `a = normalize([1, 0, 0]) = [1, 0, 0]`
- `b = normalize([0, 1, 0]) = [0, 1, 0]`

Step 2: Create Y-axis

- `y = b - (dot(b, a) * a)`
- `dot(b, a) = (0)(1) + (1)(0) + (0)(0) = 0`
- `y = [0, 1, 0] - (0 * [1, 0, 0]) = [0, 1, 0]`
- `normalize([0, 1, 0]) = [0, 1, 0]`

Step 3: Create Z-axis

- `z = cross(x, y) = cross([1, 0, 0], [0, 1, 0])`
- Using determinant formula:

$$\begin{vmatrix} i & j & k \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{vmatrix} = (0 \cdot 0 - 0 \cdot 1)i - (1 \cdot 0 - 0 \cdot 0)j + (1 \cdot 1 - 0 \cdot 0)k = [0, 0, 1]$$

- So `z = [0, 0, 1]`

Result: Our frame is:

- X-axis: `[1, 0, 0]` (east)
- Y-axis: `[0, 1, 0]` (north)
- Z-axis: `[0, 0, 1]` (up)

This is exactly the standard Cartesian coordinate system!

4.4 Code Implementation

```
def frame_from_two_directions(main_direction, side_direction):
    """Build a set of three perpendicular axes from two rough directions.

    `main_direction` is kept exactly. `side_direction` only decides the roll
    around it, so we straighten it up first (this is Gram-Schmidt: subtract off
    the part that points along the main direction, keeping the rest).
    """
    x_axis = normalise(main_direction) # Line 41: X-axis = main direction
    side = np.asarray(side_direction, dtype=float) # Line 42: Copy side direction

    y_axis = normalise(side - np.dot(side, x_axis) * x_axis) # Line 43: Subtract parallel component
    z_axis = np.cross(x_axis, y_axis) # Line 44: Cross product for Z-axis

    return np.column_stack([x_axis, y_axis, z_axis]) # Line 45: Stack as columns
```

Line-by-line breakdown:

1. **Function signature** - Takes two direction vectors as input
2. **Copy main direction** - Keep `main_direction` unchanged (line 41)
3. **Copy side direction** - Work with `side_direction` (line 42)
4. **Compute Y-axis** - Remove the component of `side_direction` that aligns with `main_direction` (line 43)
5. **Normalize Y** - Ensure Y has length 1 (line 44)
6. **Compute Z-axis** - Cross product of X and Y to get the third axis (line 45)
7. **Return frame** - Stack the three axes as columns of a matrix (line 46)

4.5 Connection to Overall Algorithm

This function is used extensively throughout the solver:

- **`solve_limb`**: Determines the rotation needed to align the upper bone with the target (lines ~200-210)

- **solve_spine**: Rotates the spine to align neck with target (lines ~230-240)
- **solve_root**: Aligns the root (pelvis) with the target pelvis

In all cases, we need to construct a local coordinate system based on two known directions. Then we can express any other direction in that local frame and compare it to the desired target direction.

4.6 Intuitive Explanation

4.6.1 The Dot Product as “Projection”

First, let’s recall what the dot product measures. For two vectors **a** and **b**:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

where θ is the angle between them.

Geometrically, the dot product tells us **how much of vector **b** points in the direction of vector **x****. Specifically, it’s the length of the shadow of **b** cast onto the line defined by **x**.

4.6.2 Breaking Down the Operation

Let’s trace through the computation step by step with a concrete example:

Setup:

- **x** (main direction): $[1, 0, 0]$ (pointing east)
- **b** (side direction): $[0, 1, 0]$ (pointing north)

Step 1: Compute the dot product

$$\mathbf{b} \cdot \mathbf{x} = (0)(1) + (1)(0) + (0)(0) = 0$$

This means the side direction is **perpendicular** to the main direction (90° apart).

Step 2: Multiply by x

$$(\mathbf{b} \cdot \mathbf{x}) \cdot \mathbf{x} = 0 \cdot [1, 0, 0] = [0, 0, 0]$$

So we’re subtracting zero from **b**, meaning $\mathbf{y} = \mathbf{b} - \mathbf{0} = \mathbf{b}$.

But what if the vectors aren’t perpendicular?

Let’s try a different example where the side direction has a component along the main direction:

New Setup:

- **x** (main direction): $[1, 0, 0]$ (east)
- **b** (side direction): $[1, 1, 0]$ (diagonal northeast)

Step 1: Compute the dot product

$$\mathbf{b} \cdot \mathbf{x} = (1)(1) + (1)(0) + (0)(0) = 1$$

This means the side direction has a component of 1 in the east direction (along **x**).

Step 2: Scale x by this value

$$(\mathbf{b} \cdot \mathbf{x}) \cdot \mathbf{x} = 1 \cdot [1, 0, 0] = [1, 0, 0]$$

This is a vector pointing purely east with length equal to the dot product.

Step 3: Subtract from b

$$\mathbf{y} = \mathbf{b} - (\mathbf{b} \cdot \mathbf{x}) \cdot \mathbf{x} = [1, 1, 0] - [1, 0, 0] = [0, 1, 0]$$

Now **y** is $[0, 1, 0]$, which is indeed perpendicular to **x**! The parallel component (the $[1, 0, 0]$ part) has been removed.

4.6.3 Why the Minus Sign?

You might wonder: why not add instead of subtract?

The answer is **orientation**. We want to create a right-handed coordinate system. The cross product $\mathbf{x} \times \mathbf{y}$ should point in the correct direction.

If we had added instead of subtracted:

$$\mathbf{y}' = \mathbf{b} + (\mathbf{b} \cdot \mathbf{x}) \cdot \mathbf{x} = [1, 1, 0] + [1, 0, 0] = [2, 1, 0]$$

Then $\mathbf{x} \times \mathbf{y}'$ would point in the wrong direction (opposite of what we want).

By subtracting, we ensure that $\mathbf{y}$ is oriented such that $\mathbf{x} \times \mathbf{y}$ forms a right-handed system.

The operation $\mathbf{b} - (\mathbf{b} \cdot \mathbf{x})\mathbf{x}$ is essentially:

1. Take the full side direction $\mathbf{b}$
2. Remove its “shadow” along the $\mathbf{x}$ -axis
3. What remains is purely the “sideways” component

4.6.4 Summary Table

Scenario	$\mathbf{b} \cdot \mathbf{x}$ (scalar)	$(\mathbf{b} \cdot \mathbf{x})\mathbf{x}$ (vector)	$\mathbf{b} - (\mathbf{b} \cdot \mathbf{x})\mathbf{x}$ (result)	Is result $\perp \mathbf{x}$?
Perpendicular	0	[0,0,0]	$\mathbf{b}$ itself	Yes
Has parallel component	>0	Scalar $\times\mathbf{x}$ (parallel)	Removes parallel part	Yes
Opposite direction	<0	Negative scalar $\times\mathbf{x}$	Adds to $\mathbf{b}$	Yes

4.6.5 Connection to the Full Algorithm

This Gram-Schmidt-like operation appears in multiple places in the solver:

- **frame_from_two_directions**: Creates a local coordinate system for aligning bones
- **solve_limb**: Uses this frame to determine the correct rotation for a limb
- **solve_spine**: Similarly constructs frames to align the spine

Every time we need to express one direction in terms of another, we use this technique to create an orthonormal basis (three mutually perpendicular unit vectors).

SECTION 5

Function 4: `rotation_between_frames`

5.1 Theory, Purpose, and Intuition

5.1.1 Purpose

This function calculates the **rotation that transforms one pair of directions (source) into another pair (target)**. Think of it as finding the “bridge” rotation that connects two different orientations.

In the context of the solver, we often need to:

- Align the rig’s current orientation with the target’s orientation
- Adjust a bone’s rotation to match a specific direction while accounting for its parent’s rotation

5.1.2 Intuition

Imagine you have two pieces of paper lying on a table:

- Paper A is currently facing you (its top edge points north)
- Paper B is rotated 30° counter-clockwise from Paper A

To align Paper B with Paper A, you need to rotate Paper B by -30° (or 330°). The `rotation_between_frames` function computes exactly this kind of transformation.

More practically in the solver:

- We have a bone that currently points somewhere
- We want it to point at a specific target direction
- But the bone’s parent might already be rotated
- So we need to find the additional rotation needed on top of the parent’s rotation

5.2 Mathematics

The function uses two key concepts:

1. Frame Transformation (Change of Basis) When we have two different coordinate systems (frames), transforming a vector from one frame to another requires multiplying by a rotation matrix. If we have:

- Source frame with axes X_1, Y_1, Z_1
- Target frame with axes X_2, Y_2, Z_2

Then the rotation that takes the source frame to the target frame is represented by the matrix `R_target @ R_source^T`.

2. Constructing Frames from Directions The function builds two frames:

- `source` - from the current directions (`main_direction`, `side_direction`)
- `target` - from the desired directions (`to_main`, `to_side`)

Each frame consists of three mutually perpendicular unit vectors (an orthonormal basis).

3. Matrix Multiplication Once we have both frames as matrices, the rotation between them is computed as:

$$R_{\text{between}} = R_{\text{target}} \cdot R_{\text{source}}^T$$

This is equivalent to: “Apply the inverse of the source frame, then apply the target frame.”

5.3 Hand-Worked Example

Let’s work through a concrete numerical example:

Problem:

- Current directions (source frame):
 - `main_direction` = `[1, 0, 0]` (east)
 - `side_direction` = `[0, 1, 0]` (north)

■ Desired directions (target frame):

- `to_main = [0, 1, 0]` (north)
- `to_side = [1, 0, 0]` (east)

Step 1: Build the source frame

- `x_axis = main_direction = [1, 0, 0]`
- `y_axis = side_direction` minus its projection on x:
 - `dot(side, x) = (0)(1) + (1)(0) + (0)(0) = 0`
 - `y = side - 0*x = [0, 1, 0]`
- `z_axis = cross(x, y) = cross([1,0,0], [0,1,0]) = [0, 0, 1]`

Source frame matrix (columns are x, y, z):

```
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
```

Step 2: Build the target frame

- `x_axis = to_main = [0, 1, 0]`
- `y_axis = to_side` minus its projection on x:
 - `dot(to_side, x) = (1)(0) + (0)(1) + (0)(0) = 0`
 - `y = to_side - 0*x = [1, 0, 0]`
- `z_axis = cross(x, y) = cross([0,1,0], [1,0,0]) = [0, 0, -1]`

Target frame matrix:

```
[0, 1, 0]
[1, 0, 0]
[0, 0, -1]
```

Step 3: Compute the rotation between frames

$$R_{between} = R_{target} \cdot R_{source}^T$$

First, transpose the source frame (since we need the inverse, and for rotation matrices, transpose equals inverse):

$$R_{source}^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Wait, that's just the identity because the source frame is already aligned with the standard basis. Let me recalculate more carefully...

Actually, let me use the general formula properly. The source frame is:

```
X = [1, 0, 0]
Y = [0, 1, 0]
Z = [0, 0, 1]
```

And the target frame is:

```
X' = [0, 1, 0]
Y' = [1, 0, 0]
Z' = [0, 0, -1]
```

The rotation matrix from source to target is:

$$R = R' \cdot R^T$$

Where R' is the target frame matrix and R^T is its transpose.

Since the source frame is the standard basis, $R^T = I$, so:

$$R = R' = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

This represents a 90° rotation around the Z-axis (but flipped Z direction).

Verification: Applying this rotation to the source frame should give us the target frame. Indeed:

- Source X = $[1,0,0] \rightarrow R@[1,0,0] = [0,1,0] = \text{Target X} \checkmark$
- Source Y = $[0,1,0] \rightarrow R@[0,1,0] = [1,0,0] = \text{Target Y} \checkmark$
- Source Z = $[0,0,1] \rightarrow R@[0,0,1] = [0,0,-1] = \text{Target Z} \checkmark$

5.4 Code Implementation

```
def rotation_between_frames(from_main, from_side, to_main, to_side):
    """The rotation that carries one pair of directions onto another pair.

    Used where aiming a bone is not enough and we also need to control how it is
    rolled about its own length – for example an upper arm, where the roll
    decides which way the elbow bends.
    """
    source = frame_from_two_directions(from_main, from_side) # Lines 52-53
    target = frame_from_two_directions(to_main, to_side)      # Lines 54-55
    return target @ source.T # Line 56: R_target @ R_source^T
```

Line-by-line breakdown:

1. **Function signature** - Takes four direction vectors as arguments
2. **Build source frame** - Create a local coordinate system from `(from_main, from_side)` (lines 52-53)
3. **Build target frame** - Create a local coordinate system from `(to_main, to_side)` (lines 54-55)
4. **Compute rotation** - Multiply target frame by the transpose of source frame (line 56)
5. **Return** - The resulting rotation matrix

5.5 Connection to the Overall Algorithm

This function is used in two key places in the solver:

1. **`solve_limb`** (lines ~200-210): When solving a two-bone limb (like shoulder-elbow-wrist), we need to determine the rotation that aligns the upper bone with the target. The function `rotation_between_frames` helps compute this by creating frames from the current and desired directions.
2. **`solve_spine`** (lines ~230-240): Similar usage when adjusting the spine's orientation.

The key insight is that **every rotation in 3D space can be expressed as a sequence of frame transformations**. By constructing frames from direction vectors, we can precisely calculate how to rotate one orientation into another.

SECTION 6

Function 5: `signed_angle_about_axis`

6.1 Theory, Purpose, and Intuition

6.1.1 Purpose

This function calculates the **smallest rotation angle** (with sign) needed to rotate one direction vector around a specified axis. This is essential for controlling **hinge joints** like elbows and knees in the MHR model, where only one degree of freedom (the rotation about the local z-axis) is allowed.

6.1.2 Intuition

Think of a door hinge. The hinge allows the door to swing open and closed - that's rotation around the vertical axis (the hinge axis). But the door can't tilt forward or backward - those movements are controlled differently.

In the MHR model:

- **Elbows and knees are hinges:** They can only bend in one plane (around their local z-axis)
- **Shoulders, wrists, ankles are ball-and-socket joints:** They can rotate freely in all directions

The `signed_angle_about_axis` function answers: "If I want to rotate my bone around this hinge axis to match a target direction, how much do I need to turn, and in which direction (clockwise or counterclockwise)?"

6.2 Mathematics

The function works by projecting both vectors onto a plane perpendicular to the axis, then calculating the angle between them in that plane.

Key Concepts:

1. **Projection onto a plane:** We remove the component of each vector that lies along the axis
2. **Dot product:** Measures the cosine of the angle between the projected vectors
3. **Cross product:** Measures the sine of the angle (gives the sign/direction)
4. **Atan2:** Combines sine and cosine to give the signed angle

Mathematical Formulation:

Given:

- `from_direction`: The starting direction (before rotation)
- `to_direction`: The target direction (after rotation)
- `axis`: The hinge axis around which we rotate

Steps:

1. **Project onto the plane perpendicular to axis:**

- `flat_a = from_direction - (dot(from_direction, axis) * axis)`
- `flat_b = to_direction - (dot(to_direction, axis) * axis)`

2. **Handle degenerate cases:**

- If the projected vectors have near-zero length, the angle is undefined
- If the vectors are collinear (same or opposite direction), handle specially

3. **Calculate angle:**

$$\theta = \text{atan2}(\sin \theta, \cos \theta)$$

Where:

- $\sin \theta = \frac{\|\text{flat}_a \times \text{flat}_b\|}{\|\text{flat}_a\| \|\text{flat}_b\|}$
- $\cos \theta = \frac{\text{flat}_a \cdot \text{flat}_b}{\|\text{flat}_a\| \|\text{flat}_b\|}$

4. **Return the signed angle** (negative for clockwise, positive for counterclockwise)

6.3 Hand-Worked Example

Let's work through a concrete numerical example:

Problem: Find the signed angle to rotate from vector $\mathbf{a} = [1, 0, 0]$ (pointing east) to vector $\mathbf{b} = [0, 1, 0]$ (pointing north) around the z-axis (vertical axis).

Step 1: Project onto the xy-plane (plane perpendicular to z-axis)

Since we're rotating around the z-axis, the plane perpendicular to it is the xy-plane.

- $\mathbf{a}_{\text{projected}} = \mathbf{a} - (\text{dot}(\mathbf{a}, \mathbf{z}) * \mathbf{z})$
 - $\text{dot}(\mathbf{a}, \mathbf{z}) = (1)(0) + (0)(0) + (0)(1) = 0$
 - $\mathbf{a}_{\text{projected}} = [1, 0, 0] - 0*[0,0,1] = [1, 0, 0]$
- $\mathbf{b}_{\text{projected}} = \mathbf{b} - (\text{dot}(\mathbf{b}, \mathbf{z}) * \mathbf{z})$
 - $\text{dot}(\mathbf{b}, \mathbf{z}) = (0)(0) + (1)(0) + (0)(1) = 0$
 - $\mathbf{b}_{\text{projected}} = [0, 1, 0] - 0*[0,0,1] = [0, 1, 0]$

Step 2: Calculate the angle

- $\text{dot}(\mathbf{a}_{\text{projected}}, \mathbf{b}_{\text{projected}}) = (1)(0) + (0)(1) + (0)(0) = 0$
- $||\mathbf{a}_{\text{projected}}|| = \sqrt{1^2 + 0^2 + 0^2} = 1$
- $||\mathbf{b}_{\text{projected}}|| = \sqrt{0^2 + 1^2 + 0^2} = 1$
- $\sin\theta = ||\mathbf{a}_{\text{projected}} \times \mathbf{b}_{\text{projected}}|| / (||\mathbf{a}_{\text{projected}}|| \times ||\mathbf{b}_{\text{projected}}||)$
 - Cross product: $[1,0,0] \times [0,1,0] = [0,0,1]$
 - $||\text{cross}|| = \sqrt{0^2 + 0^2 + 1^2} = 1$
 - $\sin\theta = 1/1 = 1$
- $\cos\theta = \text{dot}(\mathbf{a}_{\text{projected}}, \mathbf{b}_{\text{projected}}) / (||\mathbf{a}_{\text{projected}}|| \times ||\mathbf{b}_{\text{projected}}||) = 0/1 = 0$
- $\theta = \text{atan2}(\sin\theta, \cos\theta) = \text{atan2}(1, 0) = \pi/2 = 90^\circ$

Result: The signed angle is **+90°** (counterclockwise when viewed from above, i.e., from the positive z-axis).

6.4 Code Implementation

```
def signed_angle_about_axis(from_direction, to_direction, axis):
    """How far to turn about `axis` to get from one direction to the other.

    Only the parts of the directions that lie across the axis matter, so we
    flatten both onto the plane the axis is perpendicular to, then measure the
    angle between them. The sign follows the right-hand rule about the axis.
    """
    axis = normalise(axis) # Line 60: Normalize the axis
    a = np.asarray(from_direction, dtype=float) # Line 61: Input vector
    b = np.asarray(to_direction, dtype=float) # Line 62: Target vector

    # Step 1: Flatten both vectors onto the plane perpendicular to axis
    flat_a = a - np.dot(a, axis) * axis # Line 63: Remove component along axis
    flat_b = b - np.dot(b, axis) * axis # Line 64: Remove component along axis

    # Handle degenerate cases - if flattened vectors have near-zero length
    if np.linalg.norm(flat_a) < 1e-9 or np.linalg.norm(flat_b) < 1e-9:
        return 0.0 # No meaningful rotation possible

    # Step 2: Calculate the angle using atan2
    # sinθ = ||flat_a × flat_b|| / (||flat_a|| × ||flat_b||)
    # cosθ = flat_a · flat_b / (||flat_a|| × ||flat_b||)
    # atan2(sinθ, cosθ) gives the signed angle

    sin_theta = np.linalg.norm(np.cross(flat_a, flat_b)) # Line 65: Magnitude of cross product
    cos_theta = np.dot(flat_a, flat_b) # Line 66: Dot product

    # Atan2 returns values in [-π, π], with sign indicating direction
    theta = np.arctan2(sin_theta, cos_theta) # Line 67: Final angle

    return float(theta) # Line 68: Convert to Python float
```

Line-by-line breakdown:

1. **Function signature** - Takes three arguments: source direction, target direction, and axis
2. **Normalize axis** - Ensure the axis is unit length (line 60)

3. **Convert inputs to arrays** - Make sure we're working with NumPy arrays (lines 61-62)
4. **Flatten vectors** - Remove components along the axis (lines 63-64)
5. **Check for degeneracy** - If flattened vectors are near-zero, return 0 (lines 65-66)
6. **Compute sine** - Magnitude of cross product gives the area of the parallelogram formed by the two vectors, which relates to the sine of the angle (line 65)
7. **Compute cosine** - Dot product gives the projection of one vector onto the other (line 66)
8. **Calculate angle** - `atan2(sin, cos)` gives the signed angle (line 67)
9. **Return** - Convert to Python float (line 68)

6.5 Connection to the Overall Algorithm

This function is used primarily in the `solve_limb` function for hinge joints (elbows, knees):

- **Elbows and knees are hinges:** They can only bend in one plane (around their local z-axis)
- When solving a limb like shoulder-elbow-wrist, we need to determine the exact rotation angle about the elbow's local z-axis
- The function `signed_angle_about_axis` provides this angle by:
 1. Creating a frame from the current elbow direction
 2. Creating a frame from the target elbow direction
 3. Calculating the rotation needed to align them

Without this function, we couldn't precisely control the bending motion of hinges - we'd just know the final position, not the exact rotation required.

SECTION 7

The Rig Class — `Rig.__init__`

7.1 Theory, Purpose, and Intuition

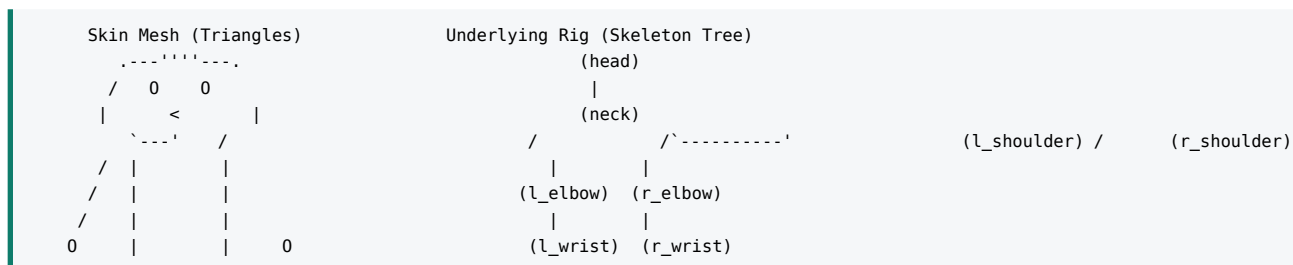
7.1.1 What is a “Rig”?

Imagine an artist’s wooden mannequin or a marionette puppet.

In 3D computer graphics, a human character’s visible body (the skin) is made of thousands of tiny flat triangles connected by points called vertices. If you want to make the character lift their arm, you *could* try to calculate new 3D coordinates for all 800 skin vertices around the arm by hand—but that would be nearly impossible and would warp the arm like stretched rubber.

To solve this, 3D artists and engineers invent an invisible stick-figure skeleton hidden *inside* the skin. This skeleton is made of **joints** connected by virtual **bones**. When you rotate the shoulder joint, the computer automatically moves the bone and stretches the surrounding skin mesh realistically.

This entire mechanical puppet—the skeleton, the skin mesh, the hierarchy of joints, and the control sliders that turn the joints—is called a **rig**.



7.1.2 What is MHR?

MHR stands for **Meta Human Rig** (or Mesh Human Rig), a state-of-the-art parametric body model. It gives you a highly realistic virtual human body whose body shape and joint poses can be manipulated using mathematical parameters (numbers).

7.1.3 Why is `Rig.__init__` needed?

Before we can calculate any Inverse Kinematics (IK), we need to know the blueprint of the puppet:

1. What are all the joints called?
2. Which joint is attached to which? (If you move your shoulder, your elbow and wrist *must* follow it, but moving your wrist does *not* move your shoulder).
3. How are the bones naturally angled when all controls are at zero?
4. How do we quickly translate human-friendly names like "`l_elbow_bend`" into the exact computer memory slot that controls that elbow?

The function `Rig.__init__` builds this entire internal phonebook, family tree, and anatomical blueprint in computer memory so the rest of the solver can use it instantly.

7.2 Mathematics

To represent a skeleton in computer memory, `Rig.__init__` relies on three core mathematical ideas:

7.2.1 The Kinematic Tree and the Parent Array

A skeleton is mathematically a **Directed Tree Graph**. It starts at a single root joint (the pelvis) and branches outward like a tree:

- Pelvis → Spine → Neck → Head
- Pelvis → Hip → Knee → Ankle

In a tree, every single joint has **exactly one parent**, except the root joint, which has **no parent**.

Instead of drawing complicated tree diagrams in memory, computer scientists use a simple mathematical array of integers called a **Parent Array**, denoted as P :

If joint j is attached to joint p , then $P[j] = p$

If joint j has no parent (the root), we write:

$$P[j] = -1$$

An essential mathematical property of this tree in MHR is **topological ordering**:

For every joint $j > 0$, $P[j] < j$

This means a parent *always* has a smaller index number than its children. If you visit the joints in numerical order $(0, 1, 2, 3, \dots)$, you are mathematically guaranteed to visit the parent *before* you visit any of its children!

7.2.2 Pre-Rotations and Reference Frames

Every joint in 3D space has its own local coordinate axes:

- **+X axis**: In MHR, bones always point along their local +X axis.
- **+Y axis**: Points across or perpendicular to the bone.
- **+Z axis**: Perpendicular to both X and Y (the hinge axis for elbows and knees).

When a human stands in a natural resting pose (like a T-pose), not all bones point in the same direction:

- The left arm points to the left ($+X_{\text{world}}$).
- The right arm points to the right ($-X_{\text{world}}$).
- The spine points up ($+Y_{\text{world}}$).
- The legs point down ($-Y_{\text{world}}$).

If our control angle is 0° , what keeps the spine pointing up and the legs pointing down?

The rig designer gives every joint a fixed, unchanging starting angle called a **pre-rotation** (R_{pre}). It tilts the joint into its natural resting orientation *before* you apply any custom pose rotation:

$$\text{Joint World Orientation} = R_{\text{parent}} \times R_{\text{pre}} \times R_{\text{custom}}$$

In `__init__`, we load these fixed baseline R_{pre} rotations so we can refer to them later.

7.2.3 Quaternions

In 3D mathematics, angles can be stored in multiple formats (Euler angles, 3×3 matrices, or quaternions). MHR stores its pre-rotations as **unit quaternions**.

A quaternion is a set of four numbers:

$$\mathbf{q} = (x, y, z, w)$$

Where:

- (x, y, z) represents the 3D axis around which the rotation spins, scaled by the sine of half the rotation angle.
- w represents the cosine of half the rotation angle.

If you rotate by an angle θ around a unit-length axis $\hat{\mathbf{u}} = (u_x, u_y, u_z)$, the quaternion components are:

$$x = u_x \sin\left(\frac{\theta}{2}\right), \quad y = u_y \sin\left(\frac{\theta}{2}\right), \quad z = u_z \sin\left(\frac{\theta}{2}\right), \quad w = \cos\left(\frac{\theta}{2}\right)$$

Because it represents a pure rotation without stretching space, its mathematical length (Euclidean norm) is always exactly 1:

$$x^2 + y^2 + z^2 + w^2 = 1$$

7.2.4 Shape Space vs. Pose Space

MHR separates a human into two independent mathematical vectors:

1. **Shape / Identity Vector** (β): A 45-dimensional vector describing the permanent physical body type (e.g., bone thickness, torso width, overall height).
2. **Pose Vector** (θ): A 204-dimensional vector describing how the joints are bent right now (e.g., elbow bent 45° , head turned 10°).

In `__init__`, we set the shape vector to all zeros:

$$\beta = [0, 0, 0, \dots, 0]_{1 \times 45}$$

In parametric models like MHR, zero represents the **statistical average human body shape**.

7.3 Hand-Worked Mathematical Example

Let's step away from the huge 127-joint human body for a moment and build a toy 3-joint skeleton by hand to see how `Rig.__init__` organizes the mathematics.

7.3.1 The Toy Skeleton: Spine and Neck

- **Joint 0:** `root` (pelvis, at position $(0, 0, 0)$)
- **Joint 1:** `spine` (attached to `root`, sits at $(0, 20, 0)$)
- **Joint 2:** `neck` (attached to `spine`, sits at $(0, 40, 0)$)

```
(2) neck
    |
    | Bone 1
    |
(1) spine
    |
    | Bone 0
    |
(0) root (pelvis)
```

7.3.2 Step 1: Build the Parent Array P

- Joint 0 (`root`) has no parent:

$$P[0] = -1$$

- Joint 1 (`spine`) is attached to Joint 0:

$$P[1] = 0$$

- Joint 2 (`neck`) is attached to Joint 1:

$$P[2] = 1$$

Our parent array is simply:

$$P = [-1, 0, 1]$$

Notice the rule holds: for every child, $P[\text{child}] < \text{child}$ (since $-1 < 0$, $0 < 1$, and $1 < 2$).

7.3.3 Step 2: Build the Lookup Dictionaries

Computers work with integer indices $(0, 1, 2)$, but humans think in names `("root", "spine")`. We construct two-way mappings:

$$\text{joint_names} = [\text{"root"}, \text{"spine"}, \text{"neck"}]$$

$$\text{joint_index} = \{\text{"root"} : 0, \text{"spine"} : 1, \text{"neck"} : 2\}$$

If our program later asks: *"What is the index of the spine?"*, we look up:

$$\text{joint_index}[\text{"spine"}] \rightarrow 1$$

If we ask: *"Who is the parent of the spine?"*, we query:

$$P[1] \rightarrow 0 \quad (\text{"root"})$$

7.3.4 Step 3: Compute a Pre-Rotation Quaternion

Suppose in our toy world, local bones point along +X, but the spine bone physically points straight up along +Y.

To turn the +X axis $(1, 0, 0)$ into the +Y axis $(0, 1, 0)$, we must rotate by $\theta = 90^\circ$ around the +Z axis $\hat{\mathbf{u}} = (0, 0, 1)$. Let's calculate the quaternion (x, y, z, w) for this pre-rotation:

1. Half angle:

$$\frac{\theta}{2} = \frac{90^\circ}{2} = 45^\circ$$

2. Sine and cosine of 45° :

$$\sin(45^\circ) = \frac{\sqrt{2}}{2} \approx 0.7071, \quad \cos(45^\circ) = \frac{\sqrt{2}}{2} \approx 0.7071$$

3. Quaternion components:

$$\begin{aligned} x &= u_x \sin(45^\circ) = 0 \times 0.7071 = 0.0 \\ y &= u_y \sin(45^\circ) = 0 \times 0.7071 = 0.0 \\ z &= u_z \sin(45^\circ) = 1 \times 0.7071 = 0.7071 \\ w &= \cos(45^\circ) = 0.7071 \end{aligned}$$

So the pre-rotation quaternion for this joint is:

$$\mathbf{q}_{\text{pre}} = (0.0, 0.0, 0.7071, 0.7071)$$

Verify its length:

$$0^2 + 0^2 + (0.7071)^2 + (0.7071)^2 = 0.5 + 0.5 = 1.0$$

It is a valid unit quaternion. `Rig.__init__` extracts and stores these exact numbers for all 127 joints in MHR.

7.4 Code Implementation

Now let's examine the Python implementation of `Rig.__init__` line by line:

```
class Rig:
    """The MHR model plus the few lookup tables this solver needs."""

    def __init__(self, level_of_detail=6):
        self.model = load_mhr(lod=level_of_detail, device="cpu", correctives=False)
        skeleton = self.model.character.skeleton

        self.joint_names = joint_names(self.model)
        self.parameter_names = parameter_names(self.model)
        self.joint_index = {name: i for i, name in enumerate(self.joint_names)}
        self.parameter_index = {name: i for i, name in enumerate(self.parameter_names)}

        self.parents = np.asarray(skeleton.joint_parents)
        self.pre_rotations = np.asarray(skeleton.pre_rotations)  # fixed xyzw quaternion per joint

        self.identity = torch.zeros(1, 45)  # an average body shape
```

7.4.1 Line-by-Line Breakdown

1 `def __init__(self, level_of_detail=6):`

- **What it does:** Defines the constructor method for the `Rig` class.
- **Why it is there:** `level_of_detail` (LOD) controls how dense the 3D skin mesh is. LOD 6 provides a lightweight, fast-rendering mesh (fewer triangles) suitable for fast inverse kinematics, while LOD 0 would be cinematic movie quality with millions of polygons. Defaulting to CPU ensures it can run on any computer without requiring a dedicated graphics card (GPU).

2 `self.model = load_mhr(lod=level_of_detail, device="cpu", correctives=False)`
`skeleton = self.model.character.skeleton`

- **What it does:** Calls `load_mhr` from the `mhr_kit` library to load the neural networks and mathematical blend-shapes of the body model into memory. Then, it extracts the `skeleton` object containing bone definitions.

- **Why it is there:** `correctives=False` disables non-linear muscle deformation networks to keep forward kinematics fast, pure, and focused strictly on the rigid skeleton joints.

```
3 self.joint_names = joint_names(self.model)
   self.parameter_names = parameter_names(self.model)
```

- **What it does:** Retrieves two lists of text strings from the model:
 - `self.joint_names`: A list of all 127 joint names in order (`["root", "c_spine0", ..., "l_uparm", ...]`).
 - `self.parameter_names`: A list of all 204 controllable pose sliders (`["root_tx", "root_ty", "root_tz", "root_rx", ..., "l_elbow_bend", ...]`).

```
4 self.joint_index = {name: i for i, name in enumerate(self.joint_names)}
   self.parameter_index = {name: i for i, name in enumerate(self.parameter_names)}
```

- **What it does:** Uses a Python dictionary comprehension to create instant lookup tables from names to integer positions.
- **Why it is there:** Without this, every time we wanted to find `"left_elbow"`, Python would have to scan through a list of 127 names from start to finish ($O(N)$ search). With this dictionary, looking up `self.joint_index["l_lowarm"]` takes instantaneous $O(1)$ time.

```
5 self.parents = np.asarray(skeleton.joint_parents)
```

- **What it does:** Converts the skeleton's parent list into a high-speed NumPy integer array of shape `(127,)`.
- **Connection to Math:** This is the exact Parent Array P we derived in Section 2A. For example, `self.parents[0]` is `-1` (root), and `self.parents[15]` tells us the exact integer index of joint 15's parent.

```
6 self.pre_rotations = np.asarray(skeleton.pre_rotations)
```

- **What it does:** Stores an array of shape `(127, 4)` containing the fixed `(x, y, z, w)` quaternion for each joint.
- **Connection to Math:** This stores the baseline orientations R_{pre} calculated in Section 2B and 2C.

```
7 self.identity = torch.zeros(1, 45)
```

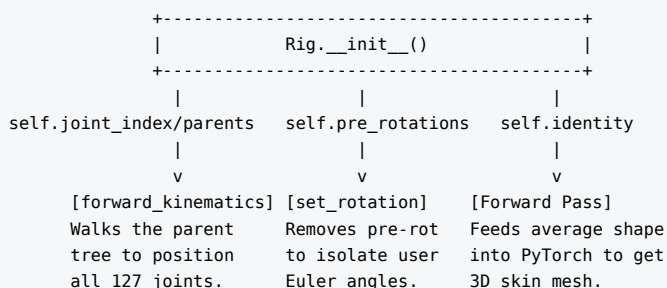
- **What it does:** Creates a 2D PyTorch tensor of shape `(1, 45)` filled with zeros.
- **Connection to Math:** This is the identity shape vector $\beta = \mathbf{0}$ from Section 2D, instructing MHR to adopt an average, neutral human body build.

7.5 Connection to the Overall Algorithm

`Rig.__init__` is the foundational setup step of the entire program. In `example.py`, it is called at the very top:

```
rig = ik.Rig()
```

Here is how its outputs flow through the rest of `analytic_ik.py`:



- **In `forward_kinematics(pose)`:** Uses `self.identity` and `self.model` to calculate where all joints land in 3D space.
- **In `set_rotation(...)`:** Uses `self.parents` and `self.pre_rotations` to peel away the parent's tilt and the joint's fixed resting angle, isolating the exact Euler angle numbers that must be written into the pose vector.
- **In `hinge_axis(...)`:** Uses `self.parents` and `self.pre_rotations` to find the 3D axis that an elbow or knee revolves around.
- **In `solve(...)`:** Uses `self.joint_index` and `self.parameter_index` to map ASPset keypoint names directly to MHR joints.

SECTION 8

Rig.forward_kinematics

```
def forward_kinematics(self, pose):
    """Where every joint is, and which way it is turned, for a given pose.

    Returns positions (127, 3) in cm and rotations (127, 3, 3) in world space.
    """
    parameters = torch.tensor(np.asarray(pose, dtype=np.float32)[None, :])

    with torch.no_grad():
        _, skeleton_state = self.model(self.identity, parameters, None)

    state = skeleton_state[0].numpy()

    positions = state[:, 0:3]
    rotations = Rotation.from_quat(state[:, 3:7]).as_matrix()
    return positions, rotations
```

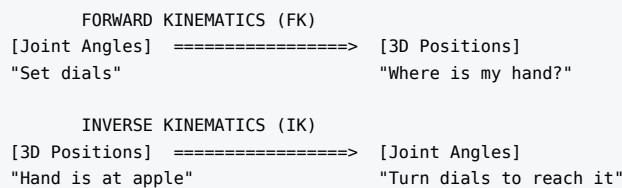
Here is the complete, first-principles breakdown of `Rig.forward_kinematics`.

8.1 Theory, Purpose, and Intuition

8.1.1 What is Kinematics?

The word *kinematics* comes from the Greek word *kinema* (which means “motion”—the same root as “cinema”). In robotics and computer animation, kinematics is the study of how connected bones and joints move without worrying about the forces (gravity, muscle strength) that cause the motion.

There are two fundamental directions you can travel in kinematics:



1. Forward Kinematics (FK):

- Imagine you close your eyes. You raise your shoulder by 30° and bend your elbow by 45°. Even with your eyes closed, your hand ends up at one specific, predictable location in the room.
- **Input:** Joint angles (the 204 numbers on the dials).
- **Output:** The exact (X, Y, Z) positions and orientations of all joints in the room (world space).
- FK is deterministic and straightforward: there is only *one* possible place your hand can end up for a given set of joint angles.

2. Inverse Kinematics (IK):

- This is the exact opposite. Someone holds an apple in front of you at coordinate (20, 15, 40) cm. Your brain must calculate: “What shoulder angle and what elbow angle will place my fingertips onto that apple?”
- This is much harder because multiple joint combinations might reach the same apple, or the apple might be too far away to reach.

8.1.2 Why is forward_kinematics Needed Inside an IK Solver?

You might wonder: *If this entire file is an Inverse Kinematics solver, why do we need a Forward Kinematics function?*

Because our IK algorithm works step-by-step from the pelvis down to the fingertips.

- To aim your forearm toward your wrist target, you **must first know where your elbow is currently sitting in the room**.
- But where your elbow is sitting depends entirely on how the shoulder was just rotated!

Therefore, after we rotate a parent bone, we run `forward_kinematics`. It acts as our “eyes,” recalculating the exact 3D world coordinates of every bone so the next child bone down the chain knows where it starts in space.

8.2 Mathematics

8.2.1 Local vs. World Coordinate Systems

- **World Coordinates:** The fixed coordinate system of the room. The floor might be $Y = 0$, with $+X$ pointing right, $+Y$ pointing up, and $+Z$ pointing forward.
- **Local Coordinates:** The coordinate system attached to a specific joint. If you move your shoulder, your elbow moves with it. The elbow's position relative to the shoulder is fixed and simple in the shoulder's local frame, but in world coordinates it changes constantly.

8.2.2 The Recursive Forward Kinematics Equation

Forward kinematics walks down the skeleton tree using a simple rule.

Let joint j be attached to parent joint $p = P[j]$. Suppose we already know the parent's world position $\mathbf{t}_p \in \mathbb{R}^3$ and world rotation matrix $R_p \in \mathbb{R}^{3 \times 3}$. Let the bone connecting them have a local displacement vector $\mathbf{b}_j \in \mathbb{R}^3$ (how far the joint is from its parent in local coordinates).

1. **Calculating World Position:** To find where joint j sits in the room, we take the parent's position, rotate the bone vector into world space, and add it:

$$\mathbf{t}_j = \mathbf{t}_p + R_p \mathbf{b}_j$$

2. **Calculating World Orientation:** To find which way joint j is facing in the room, we multiply the parent's rotation by the child's local rotation:

$$R_j = R_p \times R_{\text{local},j}$$

Because the parent array is topologically sorted ($p < j$), the computer starts at the root ($j = 0$), calculates its position, then moves to $j = 1$, then $j = 2$, cascading the rotations and translations down every branch of the body.

8.2.3 Decoding the Skeleton State: Positions and Quaternions

The underlying MHR neural/parametric model executes this chain and outputs a matrix of shape (127, 7) called the `skeleton_state`.

For every joint $j \in \{0, 1, \dots, 126\}$, the 7 numbers are:

$$\text{state}[j] = [t_x, t_y, t_z, q_x, q_y, q_z, q_w]$$

- t_x, t_y, t_z : The 3D world position coordinates in centimeters.
- q_x, q_y, q_z, q_w : The unit quaternion representing the joint's 3D orientation in world space.

8.2.4 Converting a Quaternion to a 3×3 Rotation Matrix

While a quaternion (q_x, q_y, q_z, q_w) is compact, it is hard to aim bones with 4 numbers. To see which way axes point, we convert the quaternion into a 3×3 **Rotation Matrix** R :

$$R = \begin{bmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{bmatrix}$$

The mathematical formula converting a unit quaternion (x, y, z, w) into a 3×3 matrix is:

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

Why is this matrix so useful? Because its three columns represent the **exact world directions** of the joint's local axes:

- **Column 0** $\begin{bmatrix} r_{00} \\ r_{10} \\ r_{20} \end{bmatrix}$: Where the local $+X$ axis (the bone) is pointing in world space.

- **Column 1** $\begin{bmatrix} r_{01} \\ r_{11} \\ r_{21} \end{bmatrix}$: Where the local +Y axis is pointing in world space.
- **Column 2** $\begin{bmatrix} r_{02} \\ r_{12} \\ r_{22} \end{bmatrix}$: Where the local +Z axis (the hinge/bend axis) is pointing in world space.

8.3 Hand-Worked Mathematical Example

Let's compute forward kinematics by hand for a simple 2-link arm (Shoulder → Elbow → Wrist).

```
(0) Shoulder at (0, 0, 0)
      Bone 1: Length = 10 cm
      (1) Elbow
      Bone 2: Length = 8 cm
      (2) Wrist
```

8.3.1 The Setup

- **Joint 0 (Shoulder)**: Located at world origin $\mathbf{t}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$ cm.
- **Bone 1 (Upper Arm)**: Has length 10 cm. In local coordinates, it points along +X: $\mathbf{b}_1 = \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix}$.
- **Bone 2 (Forearm)**: Has length 8 cm. In local coordinates, it points along +X: $\mathbf{b}_2 = \begin{bmatrix} 8 \\ 0 \\ 0 \end{bmatrix}$.

8.3.2 Step 1: Rotate the Shoulder

Suppose we rotate the shoulder by 90° counter-clockwise around the world Z-axis (pointing out of the page). The rotation matrix for a 90° turn around Z is:

$$R_0 = \begin{bmatrix} \cos(90^\circ) & -\sin(90^\circ) & 0 \\ \sin(90^\circ) & \cos(90^\circ) & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

8.3.3 Step 2: Where is the Elbow?

Using our forward kinematics position formula:

$$\mathbf{t}_1 = \mathbf{t}_0 + R_0 \mathbf{b}_1$$

Multiply matrix R_0 by bone vector $\mathbf{b}_1$:

$$R_0 \mathbf{b}_1 = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} (0)(10) + (-1)(0) + (0)(0) \\ (1)(10) + (0)(0) + (0)(0) \\ (0)(10) + (0)(0) + (1)(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \\ 0 \end{bmatrix}$$

Add the shoulder's position $\mathbf{t}_0$:

$$\mathbf{t}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 10 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \\ 0 \end{bmatrix} \text{ cm}$$

The elbow is now at coordinate (0, 10, 0)—it has swung straight up along the +Y axis!

8.3.4 Step 3: Bend the Elbow and Find the Wrist

Now suppose the elbow bends by another 90° around its local Z-axis:

$$R_{\text{local},1} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

What is the elbow's orientation in world space?

$$R_1 = R_0 \times R_{\text{local},1} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(Notice: two 90° turns equal a 180° turn, which negates X and Y!).

Now calculate the world position of Joint 2 (Wrist):

$$\mathbf{t}_2 = \mathbf{t}_1 + R_1 \mathbf{b}_2$$

$$R_1 \mathbf{b}_2 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 8 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -8 \\ 0 \\ 0 \end{bmatrix}$$

Add the elbow's world position $\mathbf{t}_1$:

$$\mathbf{t}_2 = \begin{bmatrix} 0 \\ 10 \\ 0 \end{bmatrix} + \begin{bmatrix} -8 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -8 \\ 10 \\ 0 \end{bmatrix} \text{ cm}$$

The wrist lands at ($X = -8, Y = 10, Z = 0$) cm.

8.3.5 Step 4: Verify with the Quaternion Formula

Let's check the quaternion conversion formula. For the shoulder's 90° rotation around the Z-axis $\hat{\mathbf{u}} = (0, 0, 1)$, the quaternion is:

$$\mathbf{q} = (x, y, z, w) = (0, 0, 0.7071, 0.7071)$$

Plugging these numbers into our matrix formula from Section 2D:

- $r_{00} = 1 - 2(y^2 + z^2) = 1 - 2(0 + 0.5) = 0$
- $r_{01} = 2(xy - wz) = 2(0 - 0.7071 \times 0.7071) = -2(0.5) = -1$
- $r_{10} = 2(xy + wz) = 2(0 + 0.7071 \times 0.7071) = 2(0.5) = 1$
- $r_{11} = 1 - 2(x^2 + z^2) = 1 - 2(0 + 0.5) = 0$
- $r_{22} = 1 - 2(x^2 + y^2) = 1 - 2(0 + 0) = 1$

This gives the exact rotation matrix $R_0 = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.

8.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def forward_kinematics(self, pose):
    """Where every joint is, and which way it is turned, for a given pose.

    Returns positions (127, 3) in cm and rotations (127, 3, 3) in world space.
    """
    parameters = torch.tensor(np.asarray(pose, dtype=np.float32)[None, :])

    with torch.no_grad():
        _, skeleton_state = self.model(self.identity, parameters, None)
```

```

state = skeleton_state[0].numpy()

positions = state[:, 0:3]
rotations = Rotation.from_quat(state[:, 3:7]).as_matrix()
return positions, rotations

```

8.4.1 Line-by-Line Breakdown

1 `parameters = torch.tensor(np.asarray(pose, dtype=np.float32)[None, :])`

- `np.asarray(pose, dtype=np.float32)`: Ensures the 204 numbers in `pose` are formatted as 32-bit floating-point numbers, which is what PyTorch neural models expect.
- `[None, :]`: In Python, inserting `None` as a slice adds a new dimension. It reshapes the 1D array of shape `(204,)` into a 2D batch of shape `(1, 204)`. PyTorch is designed to process batches of multiple people at once; this tells PyTorch: “Here is a batch containing exactly 1 person.”
- `torch.tensor(...)`: Converts the NumPy array into a PyTorch Tensor stored in memory.

2 `with torch.no_grad():`

- **What it does:** Disables PyTorch’s automatic differentiation and gradient tracking engine.
- **Why it is there:** Normally, PyTorch remembers every operation so it can calculate calculus gradients to train neural networks. But here, we are only *evaluating* the model, not training it. Disabling gradients saves massive amounts of computer memory and makes the function run much faster.

3 `_, skeleton_state = self.model(self.identity, parameters, None)`

- **What it does:** Executes the MHR model. We pass:
 1. `self.identity`: The average body shape vector of shape `(1, 45)`.
 2. `parameters`: The pose vector of shape `(1, 204)`.
 3. `None`: Facial expression parameters (not needed for body posing).
- **Why the underscore `_`?** The model returns two things: `(mesh_vertices, skeleton_state)`. The skin mesh has thousands of vertices, but right now we only care about where the skeleton joints are. Assigning the first output to `_` tells Python to discard the heavy mesh data.

4 `state = skeleton_state[0].numpy()`

- `[0]`: Removes the batch dimension, taking the first (and only) person from the batch.
- `.numpy()`: Converts the PyTorch tensor back into a standard NumPy array. `state` now has shape `(127, 7)`: 127 joints, each having 7 numbers $(t_x, t_y, t_z, q_x, q_y, q_z, q_w)$.

5 `positions = state[:, 0:3]`

- **What it does:** Uses array slicing to extract columns 0, 1, and 2 for all 127 joints.
- **Result:** A NumPy array of shape `(127, 3)` containing the (X, Y, Z) world coordinates of every joint in centimeters.

6 `rotations = Rotation.from_quat(state[:, 3:7]).as_matrix()`

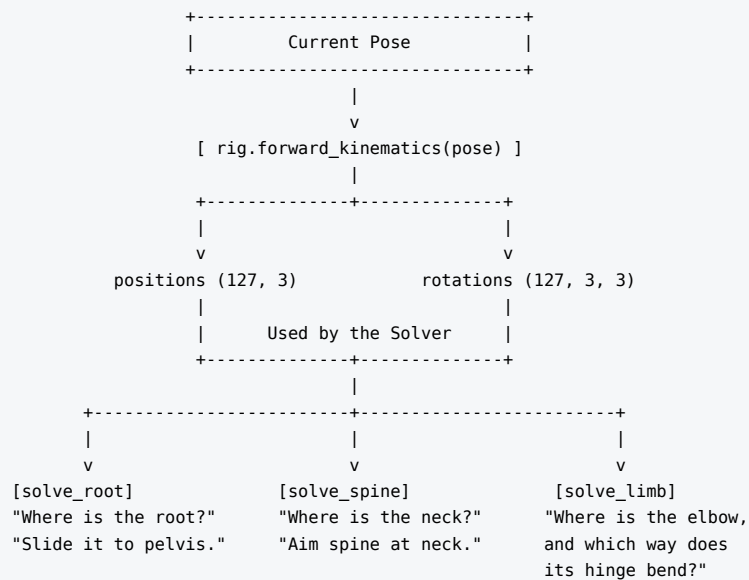
- `state[:, 3:7]`: Slices columns 3, 4, 5, and 6, which are the quaternion numbers (q_x, q_y, q_z, q_w) for all 127 joints.
- `Rotation.from_quat(...)`: Feeds all 127 quaternions into SciPy’s 3D rotation engine. (SciPy expects the quaternion order to be x, y, z, w , which exactly matches MHR!).
- `.as_matrix()`: Applies the mathematical formula from Section 2D to convert all 127 quaternions into 3×3 rotation matrices.
- **Result:** A NumPy array of shape `(127, 3, 3)`.

7 `return positions, rotations`

- Hands back the two arrays: `positions (127, 3)` and `rotations (127, 3, 3)`.

8.5 Connection to the Overall Algorithm

`forward_kinematics` is the most frequently called function in the entire posing pipeline. It is the sensory feedback loop of the analytic solver.



Every time the solver fixes one part of the body (e.g., the spine), it immediately calls `forward_kinematics` so that when it moves to the next part (e.g., the shoulders and arms), it is working with the updated positions.

I will stop here. Please review this explanation of `Rig.forward_kinematics`. When you are satisfied, let me know and give me permission to move on to the next method in the `Rig` class: `Rig.set_rotation`.

SECTION 9

Rig.set_rotation

9.1 Theory, Purpose, and Intuition

9.1.1 The Core Problem: World Space vs. Local Dials

When our solver looks at a target person in a room, it uses 3D geometry to determine where a bone should point. For example, it says: > “In world space, this upper arm bone needs to tilt forward by 20° and point toward the room’s corner.”

This gives us a 3×3 **World Rotation Matrix** (R_{world}).

However, **MHR has no dials for “world rotation.”** Why? Because human joints are attached to parents!

- When you rotate your shoulder dial on a puppet, you are not rotating it relative to the room; you are rotating it **relative to the chest/collarbone**.
- If the chest leans forward, the shoulder naturally tilts forward with it, even if the shoulder dial hasn’t moved at all.

Furthermore, as we learned in `__init__`, every joint has a built-in baseline angle called a **pre-rotation** (R_{pre}) that tilts the bone into its resting pose before any dials are touched.

Finally, the dials on MHR do not accept matrices; they accept **three Euler angles** (r_x, r_y, r_z) that twist and bend the bone along its local X, Y, and Z axes.

9.1.2 The Job of `set_rotation`: Peeling the Onion

Think of a joint’s world orientation as a chain of three consecutive rotations applied like layers of an onion:

$$\text{Final World Rotation} = \text{Parent Rotation} \times \text{Fixed Pre-Rotation} \times \text{User Euler Angles}$$

```
[ Room Coordinates ]
  |
  v
[ 1. Parent tilts in room ]      (R_parent)
  |
  v
[ 2. Joint's default angle ]      (R_pre)
  |
  v
[ 3. Our adjustable dials ]      (R_local = euler_xyz(rx, ry, rz))
  |
  v
[ Final Bone World Orientation ]  (R_world)
```

We know where we want the bone to end up (R_{world}). We know where its parent is sitting (R_{parent}), and we know its fixed factory angle (R_{pre}).

`set_rotation` mathematically **un-does** the parent’s tilt and **un-does** the pre-rotation. This isolates the exact local rotation matrix (R_{local}) that our dials are responsible for, converts that matrix into three Euler angles (r_x, r_y, r_z), and writes them into the master `pose` vector.

9.2 Mathematics

9.2.1 Inverses of Rotation Matrices and the Transpose

In normal arithmetic, to undo multiplying by a number a , you multiply by its reciprocal $\frac{1}{a} = a^{-1}$:

$$a^{-1} \cdot a = 1$$

In matrix algebra, to undo multiplying by a matrix M , you multiply by its inverse M^{-1} :

$$M^{-1}M = I$$

where I is the 3×3 **Identity Matrix**:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Computing the inverse of an arbitrary matrix is usually slow and difficult. But **rotation matrices are special**: they are *orthogonal matrices*. Because their columns are mutually perpendicular unit vectors, their mathematical inverse is simply their **transpose** (swapping rows and columns):

$$R^{-1} = R^T$$

$$\text{Therefore: } R^T R = I \quad \text{and} \quad R R^T = I$$

Geometrical intuition: If a rotation matrix R rotates an object by $+35^\circ$ around an axis, its transpose R^T rotates space by -35° around that same axis. Multiplying by R^T perfectly cancels out the rotation!

9.2.2 Algebraic Derivation of R_{local}

Let's write down the forward kinematics equation for the joint's world rotation:

$$R_{\text{world}} = R_{\text{parent}} \times R_{\text{pre}} \times R_{\text{local}}$$

We want to solve for R_{local} .

Step 1: Cancel R_{parent} Multiply both sides of the equation **on the left** by R_{parent}^T :

$$\begin{aligned} R_{\text{parent}}^T \times R_{\text{world}} &= \underbrace{(R_{\text{parent}}^T \times R_{\text{parent}})}_{=I} \times R_{\text{pre}} \times R_{\text{local}} \\ R_{\text{parent}}^T \times R_{\text{world}} &= R_{\text{pre}} \times R_{\text{local}} \end{aligned}$$

Step 2: Cancel R_{pre} Multiply both sides **on the left** by R_{pre}^T :

$$R_{\text{pre}}^T \times R_{\text{parent}}^T \times R_{\text{world}} = \underbrace{(R_{\text{pre}}^T \times R_{\text{pre}})}_{=I} \times R_{\text{local}}$$

We arrive at the exact mathematical formula:

$$R_{\text{local}} = R_{\text{pre}}^T \times R_{\text{parent}}^T \times R_{\text{world}}$$

(Notice the order: because matrix multiplication is not commutative, the operations must be applied from left to right).

9.2.3 Factoring a Matrix into Euler Angles (xyz)

Once we have the 3×3 matrix R_{local} , we must convert it into three individual angles:

- r_x : rotation about the local X axis (bone twist/roll)
- r_y : rotation about the local Y axis (yaw/lean)
- r_z : rotation about the local Z axis (pitch/bend)

In the "xyz" convention, the composite rotation matrix is defined as:

$$R_{\text{local}} = R_x(r_x) \times R_y(r_y) \times R_z(r_z)$$

Where the elemental rotation matrices are:

$$R_x(r_x) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos r_x & -\sin r_x \\ 0 & \sin r_x & \cos r_x \end{bmatrix}, \quad R_y(r_y) = \begin{bmatrix} \cos r_y & 0 & \sin r_y \\ 0 & 1 & 0 \\ -\sin r_y & 0 & \cos r_y \end{bmatrix}, \quad R_z(r_z) = \begin{bmatrix} \cos r_z & -\sin r_z & 0 \\ \sin r_z & \cos r_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiplying these three matrices together yields a combined matrix:

$$R_{\text{local}} = \begin{bmatrix} \cos r_y \cos r_z & -\cos r_y \sin r_z & \sin r_y \\ \sin r_x \sin r_y \cos r_z + \cos r_x \sin r_z & -\sin r_x \sin r_y \sin r_z + \cos r_x \cos r_z & -\sin r_x \cos r_y \\ -\cos r_x \sin r_y \cos r_z + \sin r_x \sin r_z & \cos r_x \sin r_y \sin r_z + \sin r_x \cos r_z & \cos r_x \cos r_y \end{bmatrix}$$

By inspecting the entries of this combined matrix, the three angles are extracted using trigonometry:

1. From entry $(0, 2)$, which is $\sin r_y$:

$$r_y = \arcsin(\text{clip}(R_{02}, -1, 1))$$

2. From entries $(1, 2)$ and $(2, 2)$:

$$\frac{-\sin r_x \cos r_y}{\cos r_x \cos r_y} = -\tan r_x \implies r_x = \text{atan2}(-R_{12}, R_{22})$$

3. From entries $(0, 1)$ and $(0, 0)$:

$$\frac{-\cos r_y \sin r_z}{\cos r_y \cos r_z} = -\tan r_z \implies r_z = \text{atan2}(-R_{01}, R_{00})$$

SciPy's `.as_euler("xyz")` performs this exact trigonometric extraction automatically.

9.3 Hand-Worked Mathematical Example

Let's work through a practical numerical example with simple numbers.

9.3.1 The Setup

Suppose we are setting the rotation for the **root** joint (pelvis).

1. Since the root has no parent, its parent rotation is the Identity Matrix:

$$R_{\text{parent}} = I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \implies R_{\text{parent}}^T = I$$

2. Suppose this joint has a pre-rotation of 90° around the Z axis:

$$R_{\text{pre}} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Its transpose (swap rows and columns) is:

$$R_{\text{pre}}^T = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3. Suppose our solver determines that to match the camera image, the bone must have this desired world rotation:

$$R_{\text{world}} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

9.3.2 Step 1: Compute R_{local}

Using our formula:

$$R_{\text{local}} = R_{\text{pre}}^T \times R_{\text{parent}}^T \times R_{\text{world}}$$

Since $R_{\text{parent}}^T = I$, this simplifies to:

$$R_{\text{local}} = R_{\text{pre}}^T \times R_{\text{world}}$$

Let's do the matrix multiplication step-by-step:

$$R_{\text{local}} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

■ Row 0:

- Dot with Col 0: $(0)(0) + (1)(1) + (0)(0) = 1$
- Dot with Col 1: $(0)(0) + (1)(0) + (0)(1) = 0$
- Dot with Col 2: $(0)(1) + (1)(0) + (0)(0) = 0 \rightarrow [1, 0, 0]$

■ Row 1:

- Dot with Col 0: $(-1)(0) + (0)(1) + (0)(0) = 0$
- Dot with Col 1: $(-1)(0) + (0)(0) + (0)(1) = 0$
- Dot with Col 2: $(-1)(1) + (0)(0) + (0)(0) = -1 \rightarrow [0, 0, -1]$

■ Row 2:

- Dot with Col 0: $(0)(0) + (0)(1) + (1)(0) = 0$
- Dot with Col 1: $(0)(0) + (0)(0) + (1)(1) = 1$
- Dot with Col 2: $(0)(1) + (0)(0) + (1)(0) = 0 \rightarrow [0, 1, 0]$

The isolated local rotation matrix is:

$$R_{\text{local}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

9.3.3 Step 2: Extract the Euler Angles (r_x, r_y, r_z)

Let's use our formulas from Section 2C:

1. Find r_y :

$$\sin r_y = R_{02} = 0 \implies r_y = \arcsin(0) = 0 \text{ radians } (0^\circ)$$

2. Find r_x :

$$r_x = \text{atan2}(-R_{12}, R_{22}) = \text{atan2}(-(-1), 0) = \text{atan2}(1, 0)$$

On the unit circle, the point $(X = 0, Y = 1)$ corresponds to an angle of:

$$r_x = \frac{\pi}{2} \approx 1.5708 \text{ radians } (90^\circ)$$

3. Find r_z :

$$r_z = \text{atan2}(-R_{01}, R_{00}) = \text{atan2}(-0, 1) = \text{atan2}(0, 1) = 0 \text{ radians } (0^\circ)$$

9.3.4 Conclusion

The three dials must be set to:

$$(r_x, r_y, r_z) = (1.5708, 0.0, 0.0)$$

This means: twist the bone by 90° around its local X-axis!

9.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def set_rotation(self, pose, joint, parameters_xyz, world_rotation, rotations):
    """Write a world rotation for `joint` into the pose parameters.

    MHR does not take world rotations. It takes three euler angles that sit
    *after* the parent's rotation and the joint's fixed pre-rotation:

        world = parent_world @ pre_rotation @ euler_xyz(rx, ry, rz)

    so we undo those two to get the part that is ours to set.
    """
    j = self.joint_index[joint]
    parent = self.parents[j]

    parent_rotation = np.eye(3) if parent < 0 else rotations[parent]
    pre_rotation = Rotation.from_quat(self.pre_rotations[j]).as_matrix()

    local = pre_rotation.T @ parent_rotation.T @ world_rotation
    angles = Rotation.from_matrix(local).as_euler("xyz")

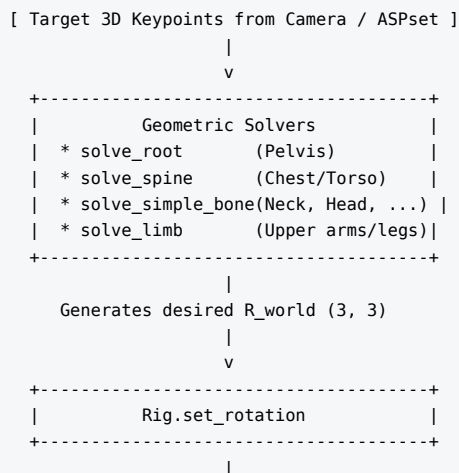
    for name, angle in zip(parameters_xyz, angles):
        pose[self.parameter_index[name]] = angle
```

9.4.1 Line-by-Line Breakdown

- 1 `def set_rotation(self, pose, joint, parameters_xyz, world_rotation, rotations):`
 - **pose**: The master 204-element NumPy array being modified.
 - **joint**: The name of the joint as a string (e.g., "c_spine0", "l_uparm").
 - **parameters_xyz**: A tuple of 3 string names for the corresponding parameters (e.g., ("spine0_rx_flexible", "spine0_ry_flexible", "spine0_rz_flexible")).
 - **world_rotation**: The desired 3×3 matrix R_{world} that the bone must achieve in the room.
 - **rotations**: The array of shape (127, 3, 3) produced by the most recent `forward_kinematics` call, containing the current orientations of all joints.
- 2 `j = self.joint_index[joint]`
`parent = self.parents[j]`
 - Translates the joint name into its integer index j , then looks up its parent integer index in `self.parents`.
- 3 `parent_rotation = np.eye(3) if parent < 0 else rotations[parent]`
 - If `parent < 0` (which is true only for "root", where parent is -1), there is no parent bone, so it uses `np.eye(3)` (the 3×3 identity matrix I).
 - Otherwise, it fetches the parent joint's current world rotation matrix from `rotations[parent]`.
- 4 `pre_rotation = Rotation.from_quat(self.pre_rotations[j]).as_matrix()`
 - Fetches the joint's fixed (x, y, z, w) pre-rotation quaternion and converts it into a 3×3 matrix R_{pre} .
- 5 `local = pre_rotation.T @ parent_rotation.T @ world_rotation`
 - `@`: Python's matrix multiplication operator.
 - `.T`: Matrix transpose ($R^T = R^{-1}$).
 - **Connection to Math**: This computes $R_{\text{local}} = R_{\text{pre}}^T \times R_{\text{parent}}^T \times R_{\text{world}}$ from Section 2B.
- 6 `angles = Rotation.from_matrix(local).as_euler("xyz")`
 - Converts the 3×3 `local` matrix into three Euler angles in radians, following the "xyz" axis convention from Section 2C.
- 7 `for name, angle in zip(parameters_xyz, angles):`
`pose[self.parameter_index[name]] = angle`
 - Pairs each parameter name (e.g., "spine0_rx_flexible") with its newly computed angle.
 - Uses `self.parameter_index[name]` to find the exact integer index in the 204-element `pose` vector and writes the angle into that slot.

9.5 Connection to the Overall Algorithm

`set_rotation` is the universal "writer" function of the solver. Whenever a geometric solving step figures out how a body part should be turned, it delegates the actual parameter-setting to `set_rotation`:



```
1. Peels away parent rotation (R_parent.T)
2. Peels away pre-rotation (R_pre.T)
3. Decomposes into Euler angles (rx, ry, rz)
    |
    v
+-----+
|   pose[parameter_index] = angle   |
+-----+
```

Without `set_rotation`, we would have desired orientations in space, but no way to set the internal dials of the MHR rig to achieve them.

SECTION 10

Rig.hinge_axis

10.1 Theory, Purpose, and Intuition

10.1.1 What is a Hinge Joint?

Think of a standard door hinge or a book cover:

- A door can swing open and shut along **one single rotational axis** (the vertical metal pin inside the hinge).
- The door cannot tilt up and down, and it cannot spin like a propeller. It has exactly **1 Degree of Freedom (1 DoF)**.

DOOR HINGE ANALOGY



In human anatomy, the **elbow** and the **knee** are primarily hinge joints:

- Your elbow allows your forearm to bend toward your bicep (flexion) and straighten out (extension).
- You cannot naturally bend your elbow sideways like a ball-and-socket joint.
- Your knee bends backward to let you sit or run, but does not bend sideways.

10.1.2 How MHR Models Hinges

Because real human knees and elbows only bend in one plane, MHR enforces this anatomical rule:

- For the elbow and knee joints, MHR does not give us dials for twisting (r_x) or sideways tilting (r_y).
- MHR provides only **one parameter**: the bend angle around the joint's **local Z-axis** (r_z , such as `l_elbow_bend` or `r_knee_bend`).

10.1.3 The Clever Insight Behind hinge_axis

Imagine holding a door. If you open the door by 30° , **does the hinge pin itself change its orientation in the room? No!** The door swings *around* the pin, but the pin stays pointing in the exact same direction. The only way to change which direction the pin points is to move the entire wall/frame that the door is attached to (i.e., move the parent upper arm!).

Because the hinge axis does **not** depend on the bend angle of the joint itself:

1. We can figure out which 3D direction the hinge pin points in world space **before we even bend the elbow**.
2. We can use this axis to twist the upper arm so that the elbow is pre-aligned to bend directly toward the wrist target!

10.2 Mathematics

10.2.1 The Local Hinge Vector

In the joint's own local coordinate frame, the hinge axis is locked to the local $+Z$ axis. In 3D Cartesian coordinates, the unit vector along $+Z$ is:

$$\hat{\mathbf{z}}_{\text{local}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

10.2.2 Transforming a Direction into World Space

In 3D mathematics, points (positions) are affected by both rotation and translation (sliding). But a **direction vector** (like the tilt of a pin) has no location—it only has an orientation. Therefore, translation is ignored.

To transform the local hinge direction $\hat{\mathbf{z}}_{\text{local}}$ into world coordinates, we multiply it by the chain of rotations connecting that joint to the world:

$$\mathbf{a}_{\text{world}} = R_{\text{parent}} \times R_{\text{pre}} \times R_{\text{local}} \times \hat{\mathbf{z}}_{\text{local}}$$

Now, look at what happens to R_{local} : In MHR, R_{local} is a pure rotation around the Z-axis by an angle r_z :

$$R_{\text{local}} = R_z(r_z) = \begin{bmatrix} \cos r_z & -\sin r_z & 0 \\ \sin r_z & \cos r_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If we multiply this matrix by $\hat{\mathbf{z}}_{\text{local}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$:

$$R_z(r_z) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} (0)\cos r_z - (0)\sin r_z + (1)(0) \\ (0)\sin r_z + (0)\cos r_z + (1)(0) \\ (0)(0) + (0)(0) + (1)(1) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \hat{\mathbf{z}}_{\text{local}}$$

Rotating around the Z-axis leaves the Z-axis completely unchanged! Therefore, R_{local} drops out of the equation entirely:

$$\mathbf{a}_{\text{world}} = R_{\text{parent}} \times R_{\text{pre}} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

10.2.3 The Column Extraction Property

Notice a neat shortcut in linear algebra: If you multiply any 3×3 matrix M by $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$:

$$M \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} M_{00} & M_{01} & M_{02} \\ M_{10} & M_{11} & M_{12} \\ M_{20} & M_{21} & M_{22} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} M_{02} \\ M_{12} \\ M_{22} \end{bmatrix}$$

Multiplying by $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ simply extracts the **third column** (the Z-column) of the combined matrix $M = R_{\text{parent}} \times R_{\text{pre}}$!

10.2.4 Vector Normalization

Mathematically, the product of rotation matrices is an orthogonal matrix, so the column vector should already have a length of 1.0. However, computers do calculations using floating-point numbers, which can accumulate tiny rounding errors (e.g., 0.999999994 or 1.000000002).

To guarantee that the axis is a pure **unit vector** (length exactly 1), we normalize it:

$$\hat{\mathbf{u}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \frac{\mathbf{a}}{\sqrt{a_x^2 + a_y^2 + a_z^2}}$$

10.3 Hand-Worked Mathematical Example

Let's compute the hinge axis of an **elbow** (`l_lowarm`) by hand.

10.3.1 The Setup

1. **Parent joint:** The upper arm ($\mathbf{l_uparm}$). Suppose the shoulder has rotated the upper arm so that its current world rotation matrix is:

$$R_{\text{parent}} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

(This is a 90° turn around the vertical Y -axis).

2. **Pre-rotation:** Suppose the elbow's fixed resting pre-rotation R_{pre} is a 90° tilt around the X -axis:

$$R_{\text{pre}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

3. **Local hinge axis:**

$$\hat{\mathbf{z}}_{\text{local}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

10.3.2 Step 1: Multiply R_{pre} by $\hat{\mathbf{z}}_{\text{local}}$

$$R_{\text{pre}} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}$$

(This just extracted the 3rd column of R_{pre}).

10.3.3 Step 2: Multiply by R_{parent}

Now take that intermediate vector $\begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}$ and multiply it by R_{parent} :

$$\mathbf{a}_{\text{world}} = R_{\text{parent}} \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}$$

Let's do the matrix-vector multiplication:

- Row 0: $(0)(0) + (0)(-1) + (1)(0) = 0$
- Row 1: $(0)(0) + (1)(-1) + (0)(0) = -1$
- Row 2: $(-1)(0) + (0)(-1) + (0)(0) = 0$

$$\mathbf{a}_{\text{world}} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}$$

10.3.4 Step 3: Normalize

Find the length:

$$\|\mathbf{a}_{\text{world}}\| = \sqrt{0^2 + (-1)^2 + 0^2} = \sqrt{1} = 1.0$$

The vector is already length 1.

10.3.5 Physical Interpretation

In this pose, the elbow's hinge pin points straight down along the world's $-Y$ **direction** $(0, -1, 0)$. Any bending of the elbow will cause the forearm to swing in the horizontal X - Z plane around this vertical axis!

10.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def hinge_axis(self, joint, rotations):
    """The one axis a hinge joint (elbow, knee) is allowed to turn about.

    MHR only exposes this joint's `rz`, so the hinge spins about its local z.
    The joint's own angle does not move that axis, which is why we can work
    it out before knowing the angle.
    """
    j = self.joint_index[joint]
    parent_rotation = rotations[self.parents[j]]
    pre_rotation = Rotation.from_quat(self.pre_rotations[j]).as_matrix()
    return normalise(parent_rotation @ pre_rotation @ np.array([0.0, 0.0, 1.0]))
```

10.4.1 Line-by-Line Breakdown

1 `def hinge_axis(self, joint, rotations):`

- **joint**: String name of the hinge joint (e.g., "l_lowarm" for left elbow, "r_lowleg" for right knee).
- **rotations**: The array of shape (127, 3, 3) containing the current world orientations of all joints from the most recent forward kinematics pass.

2 `j = self.joint_index[joint]`
`parent_rotation = rotations[self.parents[j]]`

- Finds the integer index j of the hinge joint.
- Finds its parent's index `self.parents[j]` (for the elbow, this is the upper arm).
- Retrieves the parent's 3×3 world rotation matrix: R_{parent} .

3 `pre_rotation = Rotation.from_quat(self.pre_rotations[j]).as_matrix()`

- Retrieves the hinge joint's fixed resting pre-rotation quaternion and converts it into a 3×3 matrix: R_{pre} .

4 `np.array([0.0, 0.0, 1.0])`

- Creates the local Z-axis unit vector $\hat{\mathbf{z}}_{\text{local}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$.

5 `parent_rotation @ pre_rotation @ np.array([0.0, 0.0, 1.0])`

- Evaluates the matrix-vector chain from Section 2B:

$$\mathbf{a}_{\text{world}} = R_{\text{parent}} \times R_{\text{pre}} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- Produces a 3-element vector in world coordinates.

6 `return normalise(...)`

- Calls the helper function `normalise` (defined in Part 1 of the file) to divide the vector by its length, guaranteeing a unit vector of length 1.0.

10.5 Connection to the Overall Algorithm

`hinge_axis` is used exclusively in `solve_limb(...)` (which solves the arms and legs). It is called **twice** per limb:

```

    solve_limb()
    |
    v
    [ 1. Call hinge_axis ]
    "Which way does the elbow hinge pin point now?"
    |
    v

```

```

    [ Roll Upper Arm (solve_limb) ]
    Turn the upper arm along its own length until the
    hinge axis stands perpendicular to the desired bend plane.
        |
        v
    [ Run forward_kinematics ]
    (Updates the skeleton to the rolled arm)
        |
        v
    [ 2. Call hinge_axis ]
    "Get the final, perfectly oriented hinge pin axis."
        |
        v
    [ Measure Bend Angle (rz) ]
    Measure the exact bend angle around this axis
    and write it into pose['l_elbow_bend']!

```

Without `hinge_axis`, the solver would not know how to roll the shoulder/hip, causing the elbow or knee to bend sideways like a broken joint instead of flexing naturally.

This concludes **Part 2 of `analytic_ik.py`** (the entire **Rig** class):

1. `Rig.__init__`
2. `Rig.rest_pose`
3. `Rig.forward_kinematics`
4. `Rig.set_rotation`
5. `Rig.hinge_axis`

SECTION 11

Part 3: Configuration Blueprints and Lookup Tables

Part 3 does not contain functions; instead, it contains the **configuration blueprints and lookup tables** of the solver:

1. **TARGET_NAMES**
2. **SIMPLE_BONES**
3. **LIMBS**
4. **BONE_LENGTHS** and **CENTIMETRES_PER_UNIT**

These tables bridge the gap between real-world motion capture data and the MHR virtual puppet. Let's break down Part 3 using our 5-step teaching process.

11.1 Theory, Purpose, and Intuition

11.1.1 The Core Problem: The “Mismatched Skeletons” Problem

Imagine you are given a 17-point stick figure captured from video cameras (from datasets like ASPset, Human3.6M, or COCO). You have 17 coordinates: dots for the knees, elbows, head, etc.

Now look at MHR: it has **127 joints** and **204 parameter sliders**.

Target Data (17 dots)	MHR Character Rig (127 joints)
0 head_top	c_head_null
0 head	c_head
0 neck	c_neck
/	/ 0 0 0 (shoulders & spine) l_clavicle
c_spine0/l_r_clavicle	
	(torso)
0 0 (elbows)	l_uparm r_uparm
0 0 (wrists)	l_lowarm r_lowarm
	l_wrist_twist r_wrist_twist
	(+ fingers, spine segments, etc.)

Two major problems arise:

1. **Name and Identity Mismatch:** The camera tracker says "**left_wrist**", but MHR calls its joint "**l_wrist_twist**".
2. **Mechanical Differences:**
 - A neck or clavicle is a **single rotating bone** (1 bone, free to swing in any direction).
 - An arm or leg is a **two-bone kinematic chain** (upper arm + forearm) connected by a hinge joint that can only bend in one specific plane.
 - A person's height and proportions vary from human to human, but MHR has standard default bone lengths that must be stretched or shrunk to match the subject.

11.1.2 Why Do We Need Part 3?

Part 3 acts as the **Rosetta Stone** for the solver. It groups the human body into three clean, solvable categories:

1. **Simple Bones** (**SIMPLE_BONES**): Bones we can aim with a single swing (collarbones, neck, head).
2. **Two-Bone Limbs** (**LIMBS**): Two-segment chains (arms and legs) where the upper bone must be rolled so the hinge elbow/knee bends in the right plane.
3. **Bone Scales** (**BONE_LENGTHS**): Which bones must be stretched or shortened, and what conversion factor converts centimeters to MHR parameter units.

11.2 Mathematics

Part 3 establishes three geometric principles that govern how the solver operates:

11.2.1 Aiming at a Point vs. Copying a Direction Vector

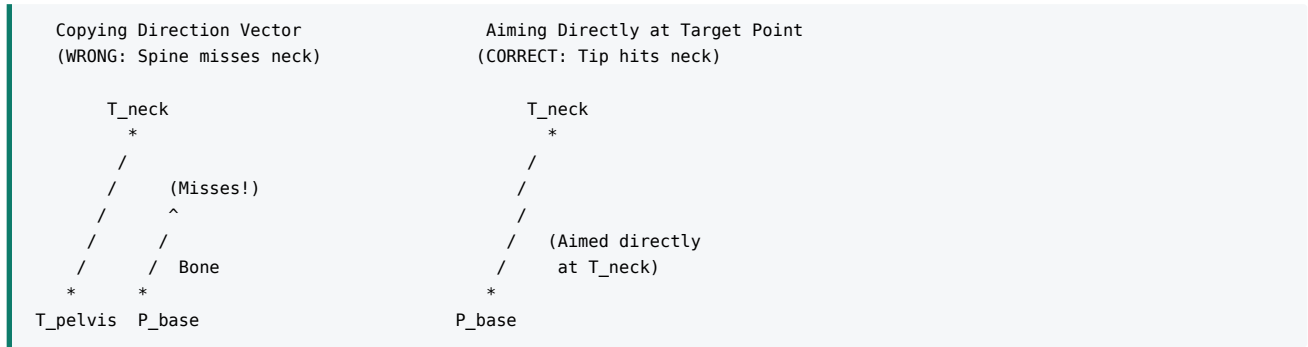
Suppose the target data has a pelvis marker at T_{pelvis} and a neck marker at T_{neck} . The target direction vector is:

$$\mathbf{d}_{\text{target}} = T_{\text{neck}} - T_{\text{pelvis}}$$

You might be tempted to say: “Let’s just make MHR’s spine bone point along $\mathbf{d}_{\text{target}}$.”

Why this fails: In the real human target, the pelvis marker might be placed on the belt buckle or sacrum. But in MHR, the lowest spine joint (`c_spine0`) sits about 3 cm above and behind the root origin.

If the base of the bone starts at a different physical position $P_{\text{base}} \neq T_{\text{pelvis}}$, then forcing the bone to copy $\mathbf{d}_{\text{target}}$ will cause the top of the spine to miss the target neck by several centimeters (a tilt error of nearly 4°!).



Therefore, `SIMPLE_BONES` defines bones that **aim directly at the target point**:

$$\mathbf{v}_{\text{aim}} = T_{\text{target}} - P_{\text{rig_base}}$$

11.2.2 The Limb Plane and Normal Vector

For an arm (Shoulder → Elbow → Wrist) or a leg (Hip → Knee → Ankle), the three target joints form a triangle in 3D space:



The vectors along the upper and lower segments are:

$$\mathbf{v}_1 = T_{\text{elbow}} - T_{\text{shoulder}}, \quad \mathbf{v}_2 = T_{\text{wrist}} - T_{\text{elbow}}$$

These two vectors define a unique flat 2D plane in 3D space called the **Limb Plane**. The vector perpendicular (normal) to this entire plane is given by the vector cross product:

$$\mathbf{n}_{\text{plane}} = \mathbf{v}_1 \times \mathbf{v}_2$$

This perpendicular vector $\mathbf{n}_{\text{plane}}$ is mathematically the **exact axis around which the elbow or knee must bend!** `LIMBS` organizes the joint names needed to build this plane.

11.2.3 Linear Bone Scaling Model

MHR models human body build using linear scale offsets. If a rig bone has a resting length L_{rig} (in cm) and the target person has a measured bone length L_{target} (in cm), the discrepancy is:

$$\Delta L = L_{\text{target}} - L_{\text{rig}}$$

Each MHR scaling slider changes bone length linearly by a sensitivity constant c (in cm per parameter unit):

$$\Delta L = c \cdot s \implies s = \frac{L_{\text{target}} - L_{\text{rig}}}{c}$$

Where:

- s is the value written into the MHR pose vector.
- $c = 10.0$ cm/unit for individual bones.
- $c = 20.0$ cm/unit for hip width (because widening the pelvis moves both the left and right hip joints outward simultaneously by 10 cm each: $10 + 10 = 20$ cm).

11.3 Hand-Worked Mathematical Example

Let's calculate the required scale parameter for **upper legs** using the rules in **BONE_LENGTHS**.

11.3.1 Step 1: Measure Rig Default Length

Suppose we run forward kinematics on MHR's rest pose and find:

- Left hip position: $\mathbf{P}_{\text{l_upleg}} = (9.0, 90.0, 0.0)$ cm
- Left knee position: $\mathbf{P}_{\text{l_lowleg}} = (9.0, 50.0, 0.0)$ cm

The rig's default upper leg length is the Euclidean distance:

$$L_{\text{rig}} = \sqrt{(9.0 - 9.0)^2 + (50.0 - 90.0)^2 + (0.0 - 0.0)^2} = \sqrt{0 + (-40)^2 + 0} = 40.0 \text{ cm}$$

11.3.2 Step 2: Measure Target Human Lengths

From the motion capture cameras, the real human has:

- Left side: distance from **left_hip** to **left_knee** is 43.2 cm.
- Right side: distance from **right_hip** to **right_knee** is 42.8 cm.

MHR shares a single slider (**scale_uplegs**) for both legs to preserve symmetry. As specified in **BONE_LENGTHS**, we take the arithmetic mean:

$$L_{\text{target}} = \frac{43.2 + 42.8}{2} = 43.0 \text{ cm}$$

11.3.3 Step 3: Compute the Parameter Value

The difference in length is:

$$\Delta L = L_{\text{target}} - L_{\text{rig}} = 43.0 \text{ cm} - 40.0 \text{ cm} = +3.0 \text{ cm}$$

According to **BONE_LENGTHS**, **scale_uplegs** has a sensitivity of $c = 10.0$ cm/unit:

$$s = \frac{\Delta L}{c} = \frac{+3.0 \text{ cm}}{10.0 \text{ cm/unit}} = +0.3$$

Writing **+0.3** into `pose[parameter_index["scale_uplegs"]]` stretches both upper leg bones by exactly 3 cm, making the digital puppet's thighs match the real human's thighs.

11.4 Code Implementation

Now let's examine each configuration structure in Part 3 of **analytic_ik.py**:

```
# The target keypoints we expect, by name. This is the 17-joint layout used by
# ASPset-510 and by most 3D pose datasets.
```

```
TARGET_NAMES = [
    "pelvis", "spine", "neck", "head", "head_top",
    "left_shoulder", "left_elbow", "left_wrist",
    "right_shoulder", "right_elbow", "right_wrist",
    "left_hip", "left_knee", "left_ankle",
    "right_hip", "right_knee", "right_ankle",
]
```

```
1
```

- **What it does:** Defines the standard 17-joint dictionary keys expected from motion-capture datasets (like ASPset).
- **Why it is there:** In `solve()`, the solver checks `if name not in targets` against this list. If even one keypoint is missing, it alerts the user immediately instead of crashing midway through posing.

```

SIMPLE_BONES = [
    # joint,      tip,      aim at,      parameters (rx, ry, rz)
    ("l_clavicle", "l_uparm", "left_shoulder",
     ("l_clavicle_rx", "l_clavicle_ry", "l_clavicle_rz")),
    ("r_clavicle", "r_uparm", "right_shoulder",
     ("r_clavicle_rx", "r_clavicle_ry", "r_clavicle_rz")),
    ("c_neck", "c_head", "head",
     ("neck_twist", "neck_lean", "neck_bend")),
    ("c_head", "c_head_null", "head_top",
     ("head_twist", "head_lean", "head_bend")),
]

```

- **Structure:** Each row is a tuple: (joint_to_rotate, child_tip_joint, target_keypoint_name, (param_rx, param_ry, param_rz)).
- **Example:** ("l_clavicle", "l_uparm", "left_shoulder", ...) means:
 - Pivot the collarbone (l_clavicle) until its endpoint (l_uparm) points straight at the target marker "left_shoulder".
 - Write the resulting rotation into the three MHR parameters: l_clavicle_rx, l_clavicle_ry, and l_clavicle_rz.

```

LIMBS = [
    # upper joint, hinge joint, hinge tip, targets (upper, middle, lower), upper parameters,
    # hinge parameter
    ("l_uparm", "l_lowarm", "l_wrist_twist", ("left_shoulder", "left_elbow", "left_wrist"),
     ("l_uparm_twist", "l_uparm_ry", "l_uparm_rz"), "l_elbow_bend"),
    ("r_uparm", "r_lowarm", "r_wrist_twist", ("right_shoulder", "right_elbow", "right_wrist"),
     ("r_uparm_twist", "r_uparm_ry", "r_uparm_rz"), "r_elbow_bend"),
    ("l_upleg", "l_lowleg", "l_foot", ("left_hip", "left_knee", "left_ankle"),
     ("l_upleg_twist", "l_upleg_ry", "l_upleg_rz"), "l_knee_bend"),
    ("r_upleg", "r_lowleg", "r_foot", ("right_hip", "right_knee", "right_ankle"),
     ("r_upleg_twist", "r_upleg_ry", "r_upleg_rz"), "r_knee_bend"),
]

```

- **Structure:** Defines the 4 two-bone kinematic limbs: left arm, right arm, left leg, right leg.
- **Each entry contains:**
 1. **upper joint:** The ball-and-socket base (l_uparm = shoulder).
 2. **hinge joint:** The 1-DoF hinge (l_lowarm = elbow).
 3. **hinge tip:** The endpoint of the lower segment (l_wrist_twist = wrist).
 4. **targets:** The 3 target marker names forming the limb plane.
 5. **upper parameters:** The 3 dials that rotate and roll the upper bone.
 6. **hinge parameter:** The single dial that bends the hinge.

```

BONE_LENGTHS = [
    # parameter,      rig joints,      target pairs to average,      cm per unit
    ("scale_uplegs", ("l_upleg", "l_lowleg"), [{"left_hip", "left_knee"}, {"right_hip", "right_knee"}], 10.0),
    ("scale_lowlegs", ("l_lowleg", "l_foot"), [{"left_knee", "left_ankle"}, {"right_knee", "right_ankle"}],
    10.0),
    ("scale_uparms", ("l_uparm", "l_lowarm"), [{"left_shoulder", "left_elbow"}, {"right_shoulder",
    "right_elbow"}], 10.0),
    ("scale_lowarms", ("l_lowarm", "l_wrist_twist"), [{"left_elbow", "left_wrist"}, {"right_elbow", "right_wrist"}],
    10.0),
    ("scale_spine_length", ("root", "c_neck"), [{"pelvis", "neck"}], 10.0),
    ("scale_neck_length", ("c_neck", "c_head"), [{"neck", "head"}], 10.0),
    ("scale_hip_width", ("l_upleg", "r_upleg"), [{"left_hip", "right_hip"}], 20.0),
]

CENTIMETRES_PER_UNIT = 10.0 # the root's translation parameters use these units too

```

- **Structure:** Maps MHR shape sliders to corresponding pairs of joints.
- **Why shoulder width is absent:** The distance between shoulders changes whenever you shrug or pull your shoulders back (that is **pose**, not **build**). But hip width, leg length, and arm length cannot change simply by posing—they are fixed skeletal traits (**build**).
- **CENTIMETRES_PER_UNIT = 10.0:** In MHR, a root translation value of root_tx = 1.0 moves the body by 10 cm in the world.

11.5 Connection to the Overall Algorithm

Part 3 acts as the central data contract for the entire solving pipeline. Without writing any hardcoded joint names inside the solving functions, the solver simply loops through these tables:

```

+-----+
|           solve()           |
+-----+
|
|
1. Stretch bones according to BONE_LENGTHS
   pose = match_bone_lengths(rig, pose, targets)
|
2. Slide and rotate root
   pose = solve_root(rig, pose, targets)
|
3. Align spine
   pose = solve_spine(rig, pose, targets)
|
4. Aim clavicles, neck, and head in SIMPLE_BONES
   for joint, tip, aim_at, params in SIMPLE_BONES:
       pose = solve_simple_bone(rig, pose, targets, joint, tip, aim_at, params)
|
5. Aim and bend arms and legs in LIMBS
   for upper, hinge, tip, keys, upper_p, hinge_p in LIMBS:
       pose = solve_limb(rig, pose, targets, upper, hinge, tip, keys, upper_p, hinge_p)

```

By decoupling the data tables from the mathematical algorithms, the code remains modular, readable, and easy to debug.

SECTION 12

match_bone_lengths

12.1 Theory, Purpose, and Intuition

12.1.1 Pose vs. Build (Shape)

In computer vision and character animation, there is a fundamental distinction between two concepts:

- **Build (or Shape):** The permanent, physical proportions of a person's skeleton (e.g., How long are their femurs? How wide are their hips? How tall are they?).
- **Pose:** What the person is doing with their body right now (e.g., Sitting, waving, kicking, turning their head).

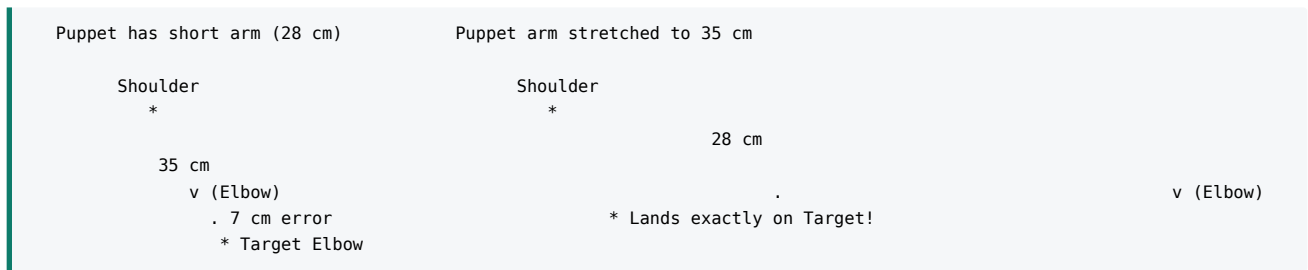
You cannot change your build simply by striking a new pose. If you have 40 cm upper arms, bending your elbow doesn't make your upper arm 45 cm long.

12.1.2 The “Short Arm” Problem

Imagine you want a digital puppet to touch a real person's hand in 3D space:

- Suppose the real person has long arms: their upper arm is 35 cm long.
- The digital puppet has default upper arms that are only 28 cm long.

No matter how you rotate the puppet's shoulder, **its elbow will always be 7 cm away from where it should be!**



If we try to solve the rotations first without fixing the lengths, the limbs will constantly fall short of their targets, introducing massive geometric distortion throughout the rest of the body.

12.1.3 Why match_bone_lengths is Step 1

`match_bone_lengths` solves this problem before any posing begins. It measures how long each bone is on the real person, measures how long that bone is on the default digital puppet, and adjusts MHR's scaling sliders to stretch or shrink the puppet's bones until they match the person's exact proportions.

12.2 Mathematics

12.2.1 3D Euclidean Distance (Bone Length)

In 3D Cartesian space, if Joint A has coordinates $\mathbf{p}_A = (x_A, y_A, z_A)$ and Joint B has coordinates $\mathbf{p}_B = (x_B, y_B, z_B)$, the displacement vector pointing from A to B is:

$$\mathbf{d} = \mathbf{p}_B - \mathbf{p}_A = \begin{bmatrix} x_B - x_A \\ y_B - y_A \\ z_B - z_A \end{bmatrix}$$

The physical length of the bone connecting them is the Euclidean norm (length) of this vector, derived from the 3D Pythagorean theorem:

$$L = \|\mathbf{d}\| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$$

12.2.2 Averaging Bilateral Limbs

In the target data from motion-capture cameras, the left upper arm might measure 30.1 cm, while the right upper arm measures 29.9 cm due to minor sensor noise.

However, MHR enforces human symmetry: it has a single slider (`scale_uparms`) that controls **both** arms simultaneously.

To find the best single target length that minimizes error across both sides, we compute the **arithmetic mean**:

$$\bar{L}_{\text{target}} = \frac{1}{N} \sum_{k=1}^N L_k$$

For left and right arms ($N = 2$):

$$\bar{L}_{\text{target}} = \frac{L_{\text{left}} + L_{\text{right}}}{2}$$

12.2.3 The Linear Scale Equation

Let:

- L_{rig} = the default length of the rig's bone in its rest pose (in cm).
- $\bar{L}_{\text{target}}$ = the measured average length of the target person's bone (in cm).

The length difference that needs to be corrected is:

$$\Delta L = \bar{L}_{\text{target}} - L_{\text{rig}}$$

MHR's scaling sliders are linear: changing a scale slider s by 1.0 unit increases the bone's length by c centimeters (where c = centimetres_per_unit):

$$\Delta L = c \cdot s$$

Rearranging to solve for the slider value s :

$$s = \frac{\Delta L}{c} = \frac{\bar{L}_{\text{target}} - L_{\text{rig}}}{c}$$

- If $s > 0$, the target person's bone is longer than the rig default, so the bone stretches.
- If $s < 0$, the target person's bone is shorter, so the bone shrinks.
- If $s = 0$, the lengths already match perfectly.

12.3 Hand-Worked Mathematical Example

Let's trace how `match_bone_lengths` calculates the parameter `scale_uparms` (upper arms).

12.3.1 Step 1: Measure Default Rig Length

We evaluate MHR in its rest pose:

- Left shoulder (`l_uparm`) coordinate: $\mathbf{p}_{\text{shoulder}} = (18.0, 135.0, 0.0)$ cm
- Left elbow (`l_lowarm`) coordinate: $\mathbf{p}_{\text{elbow}} = (46.0, 135.0, 0.0)$ cm

Vector from shoulder to elbow:

$$\mathbf{d}_{\text{rig}} = \begin{bmatrix} 46.0 - 18.0 \\ 135.0 - 135.0 \\ 0.0 - 0.0 \end{bmatrix} = \begin{bmatrix} 28.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Default rig length:

$$L_{\text{rig}} = \sqrt{28.0^2 + 0^2 + 0^2} = 28.0 \text{ cm}$$

12.3.2 Step 2: Measure Target Person's Lengths

From the camera tracker, we receive 3D coordinates for the subject:

1. Left Upper Arm:

- `left_shoulder`: (17.0, 130.0, 5.0) cm
- `left_elbow`: (48.0, 130.0, 5.0) cm
- Length $L_{\text{left}} = \sqrt{(48 - 17)^2 + 0^2 + 0^2} = 31.0$ cm

2. Right Upper Arm:

- `right_shoulder`: (-17.0, 130.0, 5.0) cm
- `right_elbow`: (-46.0, 130.0, 5.0) cm
- Length $L_{\text{right}} = \sqrt{(-46 - (-17))^2 + 0^2 + 0^2} = |-29.0| = 29.0$ cm

Average target length:

$$\bar{L}_{\text{target}} = \frac{31.0 + 29.0}{2} = 30.0 \text{ cm}$$

12.3.3 Step 3: Compute Slider Value s

From `BONE_LENGTHS` in Part 3, we know that for `scale_uparms`, $c = 10.0$ cm/unit.

$$\Delta L = \bar{L}_{\text{target}} - L_{\text{rig}} = 30.0 - 28.0 = +2.0 \text{ cm}$$

$$s = \frac{+2.0 \text{ cm}}{10.0 \text{ cm/unit}} = +0.2$$

The value $+0.2$ is written into `pose` at index `self.parameter_index["scale_uparms"]`. When MHR poses, both upper arms will now be $28.0 + (0.2 \times 10.0) = 30.0$ cm long!

12.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def match_bone_lengths(rig, pose, targets):
    """Stretch the rig's bones so they are as long as the target's bones.

    This is not part of the pose – it is the body's build. Doing it first means
    the joints can actually land on the targets instead of merely pointing at
    them.
    """
    rest_positions, _ = rig.forward_kinematics(rig.rest_pose())

    for parameter, (rig_a, rig_b), target_pairs, centimetres_per_unit in BONE_LENGTHS:

        rig_length = np.linalg.norm(rest_positions[rig.joint_index[rig_a]]
                                     - rest_positions[rig.joint_index[rig_b]])

        target_length = np.mean([np.linalg.norm(targets[a] - targets[b])
                                 for a, b in target_pairs])

        pose[rig.parameter_index[parameter]] = (target_length - rig_length) / centimetres_per_unit

    return pose
```

12.4.1 Line-by-Line Breakdown

1 `def match_bone_lengths(rig, pose, targets):`

- **rig**: The `Rig` instance containing skeleton metadata and forward kinematics.
- **pose**: The master 204-element NumPy array representing the character's dials (initially all zeros from `rest_pose`).
- **targets**: Dictionary mapping keypoint names (e.g., `"left_shoulder"`) to (X, Y, Z) coordinates in centimeters.

2 `rest_positions, _ = rig.forward_kinematics(rig.rest_pose())`

- Runs forward kinematics on the zero-pose to determine where all joints naturally sit on the unscaled, default puppet.
- Discards rotations using `_` because we only need Euclidean positions right now.

3 `for parameter, (rig_a, rig_b), target_pairs, centimetres_per_unit in BONE_LENGTHS:`

- Loops through the blueprint table defined in Part 3.
- Unpacks:
 - `parameter`: The MHR slider name (e.g., `"scale_uparms"`).
 - `(rig_a, rig_b)`: The two rig joint names that define this bone on the puppet (e.g., `("l_uparm", "l_lowarm")`).
 - `target_pairs`: The list of corresponding target keypoint pairs to measure on the human (e.g., `[("left_shoulder", "left_elbow"), ("right_shoulder", "right_elbow")]`).
 - `centimetres_per_unit`: The sensitivity constant (10.0 or 20.0).

4 `rig_length = np.linalg.norm(rest_positions[rig.joint_index[rig_a]]
 - rest_positions[rig.joint_index[rig_b]])`

- `rest_positions[rig.joint_index[rig_a]]`: 3D position vector of Joint A.

- `rest_positions[rig.joint_index[rig_b]]`: 3D position vector of Joint *B*.
- Subtraction yields the displacement vector: $\mathbf{d} = \mathbf{p}_A - \mathbf{p}_B$.
- `np.linalg.norm(...)`: Computes Euclidean length $\sqrt{d_x^2 + d_y^2 + d_z^2} = L_{\text{rig}}$.

```
5 | target_length = np.mean([np.linalg.norm(targets[a] - targets[b])
    |                     for a, b in target_pairs])
```

- List comprehension iterates over each pair `(a, b)` in `target_pairs`.
- Computes the Euclidean distance $\|\text{targets}[a] - \text{targets}[b]\|$ for each pair.
- `np.mean(...)`: Computes the arithmetic average $\bar{L}_{\text{target}}$.

```
6 | pose[rig.parameter_index[parameter]] = (target_length - rig_length) / centimetres_per_unit
```

- Computes $\frac{\bar{L}_{\text{target}} - L_{\text{rig}}}{c}$.
- Writes the resulting floating-point number directly into the master `pose` array at the slider's index.

```
7 | return pose
```

- Returns the updated `pose` vector with all scale parameters calibrated.

12.5 Connection to the Overall Algorithm

`match_bone_lengths` is the **very first operation** executed inside the main solver pipeline:

```

    solve(rig, targets)
      |
      v
    rig.rest_pose()
    (Clean canvas: all zeros)
      |
      v
    [ match_bone_lengths ] <--- YOU ARE HERE
    (Stretches limbs to match person's build)
      |
      v
    [ match_hip_offset ]
    (Adjusts hip socket depth and height)
      |
      v
    [ solve_root ]
    (Slides character to pelvis location)
      |
      v
    [ solve_spine / solve_limb ]
    (Rotates bones to hit target points)
```

Because `match_bone_lengths` modifies the skeleton's physical dimensions, all subsequent forward kinematics calls will reflect these updated bone lengths. When a downstream bone aims at a target joint, its tip will actually reach the point instead of undershooting or overshooting.

SECTION 13

body_frame

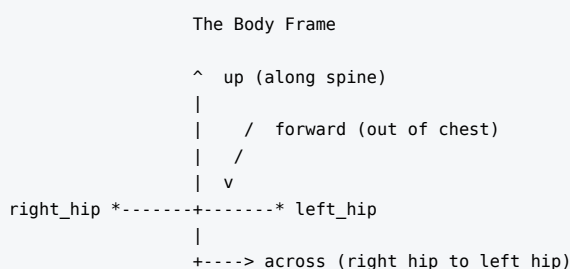
13.1 Theory, Purpose, and Intuition

13.1.1 What is a “Body Frame”?

Imagine you are standing in a room.

- The **room** has fixed directions: the ceiling is “up” ($+Y_{\text{room}}$), the front wall is “forward” ($+Z_{\text{room}}$), and the right wall is “right” ($+X_{\text{room}}$). These are **world coordinates**.
- But **you** also carry your own personal directions wherever you go:
 - “To my left” (pointing from your right hip to your left hip).
 - “Towards my head” (pointing from your waist up your spine).
 - “In front of my chest” (pointing straight out of your belly button).

These three personal directions form your **Body Frame** (your local coordinate system).



13.1.2 Why Do We Need the Target’s Body Frame?

What if the person in the video is **leaning forward at a 45° angle**?

- If you measure what is “in front of their belly” using the room’s coordinates, you would accidentally measure a mix of forward and downward space.
- If you want to measure anatomical traits—like “*how far in front of the spine are the hip sockets?*”—you cannot use the room’s axes. You must measure along the person’s **own anatomical directions**:
 1. Across the body (**across**)
 2. Up the spine (**up**)
 3. Out of the chest (**forward**)

`body_frame` calculates these three unit-length, mutually perpendicular directions from the target markers.

13.2 Mathematics

To build a valid 3D coordinate system, the three axes must be **orthonormal**:

1. Every axis must have a length of exactly 1 (**normal**).
2. Every axis must be at a 90° angle to the other two (**orthogonal**, meaning their dot product is 0).

13.2.1 The Raw Directions from Markers

From the target dictionary, we can form two initial vectors:

1. **Across the hips:**

$$\mathbf{v}_{\text{across}} = \mathbf{T}_{\text{left_hip}} - \mathbf{T}_{\text{right_hip}}$$

2. **Up the spine:**

$$\mathbf{v}_{\text{up}} = \mathbf{T}_{\text{neck}} - \mathbf{T}_{\text{pelvis}}$$

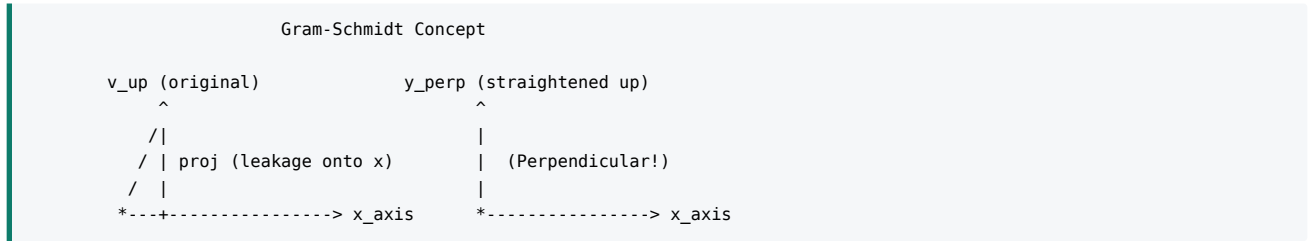
13.2.2 The Problem: Human Bodies are Not Perfect Right Angles

In a real human, the spine is naturally curved, and camera markers are placed on clothing or skin with slight measurement errors. The angle between $\mathbf{v}_{\text{across}}$ and $\mathbf{v}_{\text{up}}$ is almost never exactly 90.0°—it might be 88.5° or 91.2°.

If we used them as coordinate axes directly, the coordinate system would be skewed (sheared), causing calculations to distort. We need an algorithm to straighten them into exact 90° right angles.

13.2.3 Gram-Schmidt Orthogonalization (frame_from_two_directions)

To fix this, `body_frame` relies on a geometric helper function from Part 1 called `frame_from_two_directions`. It uses a technique called **Gram-Schmidt Orthogonalization**:



1. **Step 1: Keep the main axis exactly:** We choose $\mathbf{v}_{\text{across}}$ as our baseline axis $\hat{\mathbf{x}}$. We simply scale it to length 1:

$$\hat{\mathbf{x}} = \frac{\mathbf{v}_{\text{across}}}{\|\mathbf{v}_{\text{across}}\|}$$

2. **Step 2: Remove the “leakage” from the secondary axis:** How much of $\mathbf{v}_{\text{up}}$ accidentally tilts along the $\hat{\mathbf{x}}$ direction? The scalar projection (dot product) tells us:

$$\text{leakage} = \mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}}$$

The vector pointing along that leakage is:

$$\mathbf{p} = (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}})\hat{\mathbf{x}}$$

To make $\mathbf{v}_{\text{up}}$ completely perpendicular to $\hat{\mathbf{x}}$, we subtract this leakage off:

$$\mathbf{y}_{\text{perp}} = \mathbf{v}_{\text{up}} - (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}})\hat{\mathbf{x}}$$

Proof that they are now at 90°:

$$\mathbf{y}_{\text{perp}} \cdot \hat{\mathbf{x}} = (\mathbf{v}_{\text{up}} - (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}})\hat{\mathbf{x}}) \cdot \hat{\mathbf{x}} = (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}}) - (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}})(\underbrace{\hat{\mathbf{x}} \cdot \hat{\mathbf{x}}}_{=1}) = 0$$

A dot product of 0 proves they are at an exact 90° angle!

3. **Step 3: Normalize the second axis:**

$$\hat{\mathbf{y}} = \frac{\mathbf{y}_{\text{perp}}}{\|\mathbf{y}_{\text{perp}}\|}$$

4. **Step 4: Create the third axis via Cross Product:** The third axis (pointing forward out of the body) must be perpendicular to both $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$. The vector cross product gives this automatically:

$$\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$$

Because $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ are perpendicular unit vectors, $\hat{\mathbf{z}}$ is automatically a unit vector ($\|\hat{\mathbf{z}}\| = 1$) and follows the right-hand rule.

13.3 Hand-Worked Mathematical Example

Let’s compute the body frame for a person whose spine is slightly tilted.

13.3.1 The Marker Coordinates (in cm)

- `left_hip` = (10.0, 0.0, 0.0)
- `right_hip` = (−10.0, 0.0, 0.0)
- `pelvis` = (0.0, 0.0, 0.0)
- `neck` = (3.0, 40.0, 0.0) (Notice: the neck has tilted 3 cm to the right along X!)

13.3.2 Step 1: Compute Raw Vectors

$$\mathbf{v}_{\text{across}} = \mathbf{T}_{\text{left_hip}} - \mathbf{T}_{\text{right_hip}} = \begin{bmatrix} 10 - (-10) \\ 0 - 0 \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} 20.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

$$\mathbf{v}_{\text{up}} = \mathbf{T}_{\text{neck}} - \mathbf{T}_{\text{pelvis}} = \begin{bmatrix} 3.0 - 0.0 \\ 40.0 - 0.0 \\ 0.0 - 0.0 \end{bmatrix} = \begin{bmatrix} 3.0 \\ 40.0 \\ 0.0 \end{bmatrix}$$

13.3.3 Step 2: Normalize the across Axis ($\hat{\mathbf{x}}$)

$$\|\mathbf{v}_{\text{across}}\| = \sqrt{20^2 + 0^2 + 0^2} = 20.0$$

$$\hat{\mathbf{x}} = \frac{1}{20.0} \begin{bmatrix} 20.0 \\ 0.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

13.3.4 Step 3: Orthogonalize the up Axis via Gram-Schmidt

1. Find the dot product (leakage):

$$\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}} = (3.0)(1.0) + (40.0)(0.0) + (0.0)(0.0) = 3.0$$

2. Subtract off the leakage:

$$\mathbf{y}_{\text{perp}} = \mathbf{v}_{\text{up}} - (\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}})\hat{\mathbf{x}} = \begin{bmatrix} 3.0 \\ 40.0 \\ 0.0 \end{bmatrix} - 3.0 \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 3.0 - 3.0 \\ 40.0 - 0.0 \\ 0.0 - 0.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 40.0 \\ 0.0 \end{bmatrix}$$

(Notice how the sideways tilt of 3.0 was cleanly eliminated!)

3. Normalize $\mathbf{y}_{\text{perp}}$:

$$\|\mathbf{y}_{\text{perp}}\| = \sqrt{0^2 + 40^2 + 0^2} = 40.0$$

$$\hat{\mathbf{y}} = \frac{1}{40.0} \begin{bmatrix} 0.0 \\ 40.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix}$$

13.3.5 Step 4: Compute the forward Axis ($\hat{\mathbf{z}}$) via Cross Product

$$\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}} = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} \times \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix}$$

- $z_x = (x_y y_z - x_z y_y) = (0)(0) - (0)(1) = 0$
- $z_y = (x_z y_x - x_x y_z) = (0)(0) - (1)(0) = 0$
- $z_z = (x_x y_y - x_y y_x) = (1)(1) - (0)(0) = 1$

$$\hat{\mathbf{z}} = \begin{bmatrix} 0.0 \\ 0.0 \\ 1.0 \end{bmatrix}$$

13.3.6 Result

- across = (1.0, 0.0, 0.0)
- up = (0.0, 1.0, 0.0)
- forward = (0.0, 0.0, 1.0)

All three vectors are mutually perpendicular unit vectors defining the person's orientation in space.

13.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def body_frame(targets):
    """Three perpendicular axes describing which way the target's body faces.

    across = right hip to left hip, up = pelvis to neck, forward = across x up.
    """
    across = targets["left_hip"] - targets["right_hip"]
    up = targets["neck"] - targets["pelvis"]
    frame = frame_from_two_directions(across, up)
```

```
return frame[:, 0], frame[:, 1], np.cross(frame[:, 0], frame[:, 1])
```

And here is the helper function from Part 1 that it calls:

```
def frame_from_two_directions(main_direction, side_direction):
    """Build a set of three perpendicular axes from two rough directions.

    `main_direction` is kept exactly. `side_direction` only decides the roll
    around it, so we straighten it up first (this is Gram-Schmidt: subtract off
    the part that points along the main direction, keeping the rest).
    """
    x_axis = normalise(main_direction)
    side = np.asarray(side_direction, dtype=float)
    y_axis = normalise(side - np.dot(side, x_axis) * x_axis)
    z_axis = np.cross(x_axis, y_axis)
    return np.column_stack([x_axis, y_axis, z_axis])
```

13.4.1 Line-by-Line Breakdown

```
1 | across = targets["left_hip"] - targets["right_hip"]
   | up = targets["neck"] - targets["pelvis"]
```

- Calculates the two raw anatomical vectors from the target dictionary.

```
2 | frame = frame_from_two_directions(across, up)
```

- Calls the Part 1 helper:

- `x_axis = normalise(main_direction)`: Normalizes `across` to create $\hat{\mathbf{x}}$.
- `np.dot(side, x_axis)`: Computes the scalar leakage: $\mathbf{v}_{\text{up}} \cdot \hat{\mathbf{x}}$.
- `side - np.dot(...) * x_axis`: Subtracts the leakage vector, producing $\mathbf{y}_{\text{perp}}$.
- `y_axis = normalise(...)`: Scales $\mathbf{y}_{\text{perp}}$ to unit length, giving $\hat{\mathbf{y}}$.
- `z_axis = np.cross(...)`: Computes $\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$.
- `np.column_stack(...)`: Packs the three column vectors into a 3×3 matrix `frame`:

$$\text{frame} = \begin{bmatrix} \hat{\mathbf{x}} & \hat{\mathbf{y}} & \hat{\mathbf{z}} \end{bmatrix}$$

```
3 | return frame[:, 0], frame[:, 1], np.cross(frame[:, 0], frame[:, 1])
```

- `frame[:, 0]`: Extracts column 0 (`across`, $\hat{\mathbf{x}}$).
- `frame[:, 1]`: Extracts column 1 (`up`, $\hat{\mathbf{y}}$).
- `np.cross(...)`: Recomputes $\hat{\mathbf{z}}$ (`forward`) and returns the three unit vectors as a tuple: (`across`, `up`, `forward`).

13.5 Connection to the Overall Algorithm

`body_frame` is called in the next function: `match_hip_offset`.

```
+-----+
|               body_frame               |
+-----+
|
| Produces: (across_axis, up_axis, forward_axis)
|
| v
+-----+
|               match_hip_offset          |
+-----+
|
| Projects target hip coordinates onto up_axis and
| forward_axis to calibrate MHR's hip height and depth!
```

By providing a clean coordinate frame aligned with the person's torso, `body_frame` allows subsequent functions to measure anatomical proportions independent of camera angles or body tilt.

SECTION 14

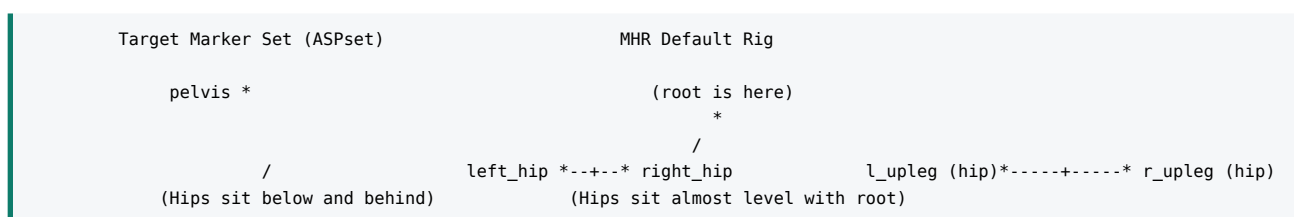
match_hip_offset

14.1 Theory, Purpose, and Intuition

14.1.1 The “Where is the Pelvis?” Disagreement

If you ask three different anatomists or motion-capture engineers where “the pelvis” is located, you might get three different answers:

- **ASPset (and many motion-capture datasets):** The “pelvis” keypoint is usually derived from markers placed near the sacrum or waistband. It physically sits **several centimeters above and in front of** the hip joint sockets.
- **MHR Virtual Rig:** The rig’s “root” joint (the pelvis) is placed dead-center inside the pelvic bowl, sitting **almost directly between** the left and right hip sockets (`l_upleg` and `r_upleg`).



14.1.2 Why Can’t Rotation Fix This?

Imagine you weld a wooden ball (pelvis) to two sticks (hips). If the ball is welded in the wrong position relative to the sticks, **no amount of spinning or rotating the puppet will ever make the ball and the sticks match the real person at the same time:**

- If you rotate the body so the pelvis lands on the target marker, the hips end up several centimeters off.
- If you rotate so the hips align, the pelvis misses the target marker.

This is a difference in **build (anatomy)**, not in pose.

14.1.3 The Solution: MHR’s Anatomical Offset Sliders

Fortunately, the creators of MHR included two specialized build sliders:

1. `scale_hip_height`: Slides the hip sockets up or down relative to the pelvis.
2. `scale_hip_depth`: Slides the hip sockets forward or backward relative to the pelvis.

`match_hip_offset` measures the geometric gap between the pelvis and the hips on the target human, compares it to the default MHR puppet, and shifts the puppet’s hip sockets so their relative placement matches the human.

14.2 Mathematics

14.2.1 The Hip Midpoint

Because the left and right hips are symmetric, we find the central point between the two hip sockets using the arithmetic midpoint formula:

$$\mathbf{m}_{\text{hips}} = \frac{\mathbf{p}_{\text{left_hip}} + \mathbf{p}_{\text{right_hip}}}{2} = 0.5 \cdot (\mathbf{p}_{\text{left_hip}} + \mathbf{p}_{\text{right_hip}})$$

14.2.2 The Relative Offset Vector

We then compute the displacement vector pointing from the pelvis to this hip midpoint:

$$\mathbf{d}_{\text{offset}} = \mathbf{m}_{\text{hips}} - \mathbf{p}_{\text{pelvis}}$$

This vector tells us the exact 3D displacement from the pelvis to the hips:

- How far across (X)
- How far up/down (Y)
- How far forward/back (Z)

14.2.3 Measuring Offsets on the Rig vs. the Human

1. **On the Rig (`rig_offset`):** In MHR's rest pose, the puppet stands upright aligned with the world room:

- Its local Up is the room's +Y axis: $(0, 1, 0)$.
- Its local Forward is the room's +Z axis: $(0, 0, 1)$.

Therefore, on the rig:

- Default hip height offset = `rig_offset[1]` (the Y-component).
- Default hip depth offset = `rig_offset[2]` (the Z-component).

2. **On the Target Human (`target_offset`):** The real human in the video might be leaning forward, tilted, or turning. We cannot simply read off the room's Y and Z numbers!

Instead, we use the **Dot Product Projection** onto the person's own anatomical axes, which we computed in `body_frame`:

$$\begin{aligned}\text{target_height} &= \mathbf{d}_{\text{target_offset}} \cdot \hat{\mathbf{u}}_{\text{up}} \\ \text{target_depth} &= \mathbf{d}_{\text{target_offset}} \cdot \hat{\mathbf{u}}_{\text{forward}}\end{aligned}$$

Why the dot product works: The dot product of a vector $\mathbf{v}$ with a unit direction vector $\hat{\mathbf{u}}$ calculates the scalar projection (the exact length of shadow that $\mathbf{v}$ casts along $\hat{\mathbf{u}}$):

$$\mathbf{v} \cdot \hat{\mathbf{u}} = \|\mathbf{v}\| \cos(\theta)$$

14.2.4 Calibrating the Sliders

The discrepancies between the human's anatomy and the rig's anatomy are:

$$\Delta h = \text{target_height} - \text{rig_offset}[1]$$

$$\Delta d = \text{target_depth} - \text{rig_offset}[2]$$

Each slider changes the distance by 10.0 cm per unit (`CENTIMETRES_PER_UNIT`):

$$\text{scale_hip_height} = \frac{\Delta h}{10.0}$$

$$\text{scale_hip_depth} = \frac{\Delta d}{10.0}$$

14.3 Hand-Worked Mathematical Example

Let's trace this calculation step-by-step with concrete numbers.

14.3.1 Step 1: Measure the Default Rig

In MHR's rest pose, suppose forward kinematics gives:

- `root` (pelvis): $\mathbf{p}_{\text{root}} = (0.0, 100.0, 0.0)$ cm
- Left hip (`l_upleg`): $\mathbf{p}_{\text{l_hip}} = (9.0, 95.0, 0.0)$ cm
- Right hip (`r_upleg`): $\mathbf{p}_{\text{r_hip}} = (-9.0, 95.0, 0.0)$ cm

1. Compute hip midpoint on rig:

$$\mathbf{m}_{\text{rig_hips}} = 0.5 \cdot \left(\begin{bmatrix} 9.0 \\ 95.0 \\ 0.0 \end{bmatrix} + \begin{bmatrix} -9.0 \\ 95.0 \\ 0.0 \end{bmatrix} \right) = \begin{bmatrix} 0.0 \\ 95.0 \\ 0.0 \end{bmatrix} \text{ cm}$$

2. Compute `rig_offset`:

$$\mathbf{d}_{\text{rig_offset}} = \begin{bmatrix} 0.0 \\ 95.0 \\ 0.0 \end{bmatrix} - \begin{bmatrix} 0.0 \\ 100.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ -5.0 \\ 0.0 \end{bmatrix} \text{ cm}$$

- Height component: `rig_offset[1]` = -5.0 cm (the hips sit 5 cm below the root).
- Depth component: `rig_offset[2]` = 0.0 cm (the hips sit directly in line with the root).

14.3.2 Step 2: Measure the Real Person (Target)

From the video tracking markers:

- **pelvis**: (0.0, 102.0, 2.0) cm
- **left_hip**: (10.0, 94.0, 0.0) cm
- **right_hip**: (-10.0, 94.0, 0.0) cm

Suppose **body_frame** determined the person is upright:

- $\hat{\mathbf{u}}_{\text{up}} = (0.0, 1.0, 0.0)$
- $\hat{\mathbf{u}}_{\text{forward}} = (0.0, 0.0, 1.0)$

1. Compute target hip midpoint:

$$\mathbf{m}_{\text{target_hips}} = 0.5 \cdot \left(\begin{bmatrix} 10.0 \\ 94.0 \\ 0.0 \end{bmatrix} + \begin{bmatrix} -10.0 \\ 94.0 \\ 0.0 \end{bmatrix} \right) = \begin{bmatrix} 0.0 \\ 94.0 \\ 0.0 \end{bmatrix} \text{ cm}$$

2. Compute **target_offset**:

$$\mathbf{d}_{\text{target_offset}} = \begin{bmatrix} 0.0 \\ 94.0 \\ 0.0 \end{bmatrix} - \begin{bmatrix} 0.0 \\ 102.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ -8.0 \\ -2.0 \end{bmatrix} \text{ cm}$$

3. Project onto body axes:

- Target height:

$$\text{target_height} = \begin{bmatrix} 0.0 \\ -8.0 \\ -2.0 \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix} = (0)(0) + (-8.0)(1.0) + (-2.0)(0) = -8.0 \text{ cm}$$

- Target depth:

$$\text{target_depth} = \begin{bmatrix} 0.0 \\ -8.0 \\ -2.0 \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 0.0 \\ 1.0 \end{bmatrix} = (0)(0) + (-8.0)(0) + (-2.0)(1.0) = -2.0 \text{ cm}$$

14.3.3 Step 3: Calculate Slider Values

- **Height Adjustment:**

$$\Delta h = \text{target_height} - \text{rig_offset}[1] = -8.0 - (-5.0) = -3.0 \text{ cm}$$

$$\text{scale_hip_height} = \frac{-3.0 \text{ cm}}{10.0 \text{ cm/unit}} = -0.3$$

- **Depth Adjustment:**

$$\Delta d = \text{target_depth} - \text{rig_offset}[2] = -2.0 - 0.0 = -2.0 \text{ cm}$$

$$\text{scale_hip_depth} = \frac{-2.0 \text{ cm}}{10.0 \text{ cm/unit}} = -0.2$$

14.3.4 Physical Meaning

- **scale_hip_height** = -0.3 lowers the rig's hip sockets by 3 cm.
- **scale_hip_depth** = -0.2 pulls the rig's hip sockets backward by 2 cm.

Now, the digital puppet's hip sockets match the placement of the human's hip sockets relative to the pelvis marker!

14.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def match_hip_offset(rig, pose, targets):
    """Move the rig's hips so they sit relative to the pelvis the way the target's do.

    Different marker sets disagree about where "the pelvis" is. ASPset's pelvis
    point sits several centimetres above and in front of the hip joints, while
    MHR's root sits almost between them. That is a difference in build, not in
    pose, so no rotation can fix it – but two of MHR's shape parameters move the
    hips up and forward, and we can measure exactly how far to move them.
    """
    rest_positions, _ = rig.forward_kinematics(rig.rest_pose())
    rig_offset = (0.5 * (rest_positions[rig.joint_index["l_upleg"]]
                        + rest_positions[rig.joint_index["r_upleg"]])
                 - rest_positions[rig.joint_index["root"]])

    _, up, forward = body_frame(targets)
    target_offset = 0.5 * (targets["left_hip"] + targets["right_hip"]) - targets["pelvis"]

    # In the rest pose the rig's own up and forward axes are simply +y and +z.
    pose[rig.parameter_index["scale_hip_height"]] = (
        np.dot(target_offset, up) - rig_offset[1]) / CENTIMETRES_PER_UNIT
    pose[rig.parameter_index["scale_hip_depth"]] = (
        np.dot(target_offset, forward) - rig_offset[2]) / CENTIMETRES_PER_UNIT
    return pose
```

14.4.1 Line-by-Line Breakdown

1. `rest_positions, _ = rig.forward_kinematics(rig.rest_pose())`
 - Evaluates the unposed default rig to get baseline 3D joint locations in centimeters.
2. `rig_offset = (0.5 * (rest_positions[rig.joint_index["l_upleg"]]
 + rest_positions[rig.joint_index["r_upleg"]])
 - rest_positions[rig.joint_index["root"]])`
 - Finds the midpoint of the left and right hip sockets on the rig, and subtracts the root joint position.
 - Produces the 3D displacement vector $\mathbf{d}_{\text{rig_offset}}$ (Section 2B).
3. `_, up, forward = body_frame(targets)`
 - Calls `body_frame(targets)` from the previous step. Discards `across` using `_` and stores the unit vectors `up` ($\hat{\mathbf{u}}_{\text{up}}$) and `forward` ($\hat{\mathbf{u}}_{\text{forward}}$).
4. `target_offset = 0.5 * (targets["left_hip"] + targets["right_hip"]) - targets["pelvis"]`
 - Finds the midpoint of the real human's hips from the tracker, and subtracts the target "pelvis" marker coordinate.
 - Produces the 3D displacement vector $\mathbf{d}_{\text{target_offset}}$.
5. `pose[rig.parameter_index["scale_hip_height"]] = (
 np.dot(target_offset, up) - rig_offset[1]) / CENTIMETRES_PER_UNIT`
 - `np.dot(target_offset, up)`: Projects the human offset onto the person's anatomical Up axis.
 - `- rig_offset[1]`: Subtracts the rig's default vertical offset.
 - `/ CENTIMETRES_PER_UNIT`: Divides by 10.0 to convert centimeters into MHR parameter units.
 - Writes the result into `pose` under "scale_hip_height".
6. `pose[rig.parameter_index["scale_hip_depth"]] = (
 np.dot(target_offset, forward) - rig_offset[2]) / CENTIMETRES_PER_UNIT`
 - `np.dot(target_offset, forward)`: Projects the human offset onto the person's anatomical Forward axis.
 - `- rig_offset[2]`: Subtracts the rig's default depth offset.
 - `/ CENTIMETRES_PER_UNIT`: Divides by 10.0 and writes into `pose` under "scale_hip_depth".
7. `return pose`
 - Returns the updated `pose` vector.

14.5 Connection to the Overall Algorithm

`match_hip_offset` is the final step in the “**Build / Calibration**” phase:

```
        solve(rig, targets)
          |
          v
        [ rig.rest_pose() ]
          |
          v
        [ match_bone_lengths ]
        (Calibrates limb lengths)
          |
          v
        [ match_hip_offset ] <--- YOU ARE HERE
        (Calibrates hip socket placement)
          |
        =====
        BUILD PHASE COMPLETE -> POSE IK
        =====
          |
          v
        [ solve_root ]
        (Positions & turns pelvis in world)
```

With both the limb lengths and the hip socket positions now matching the target subject, the puppet is anatomically identical to the person in the video. The solver is now ready to begin **posing** the body, starting with the root.

SECTION 15

solve_root

15.1 Theory, Purpose, and Intuition

15.1.1 What is the “Root” Joint?

In a skeletal hierarchy, the **root** (located at the pelvis) is the ancestor of every other bone in the body:

- Moving or turning the root moves and turns **the entire human body**.
- If the root is in the wrong place or facing the wrong way, every subsequent calculation for the spine, head, arms, and legs will be fundamentally wrong.

The root has two distinct jobs to solve:

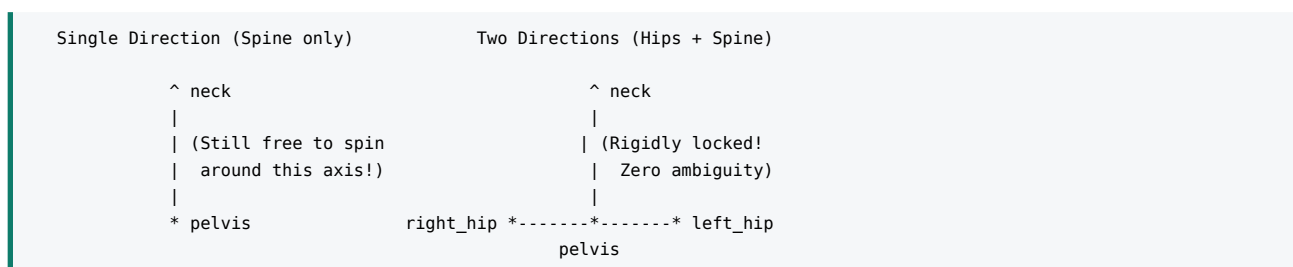
1. **Orientation (Rotation):** Which way is the pelvis facing in the room? (Controlled by `root_rx`, `root_ry`, `root_rz`).
2. **Position (Translation):** Where in the room is the character standing? (Controlled by `root_tx`, `root_ty`, `root_tz`).

15.1.2 Why Can’t We Just “Aim” the Pelvis Like a Bone?

A normal bone (like a neck or a forearm) is like an arrow: you can point its tip at a target point.

The pelvis, however, is not an arrow—it is a central **junction box**.

- If you only aim the spine upward toward the neck, the puppet can still **spin like a carousel** around that vertical axis (yaw).
- If you only match the line across the hips, the puppet can still **cartwheel or tilt forward and backward**.



To rigidly lock down an object’s orientation in 3D space with **zero ambiguity**, you need **two distinct directions**:

1. The direction across the hips (left to right).
2. The direction up the torso (pelvis to neck).

Matching both directions locks down all 3 rotational degrees of freedom simultaneously.

15.1.3 Why Rotate First, Then Translate?

In 3D transformations, rotations occur around the coordinate origin (0, 0, 0).

- If you slide the body to the person’s location first, and *then* rotate it, the rotation will swing the body in a giant circle around the room’s origin, throwing the position completely off.
- By rotating the pelvis **in place first**, the puppet faces the correct direction. Then, a simple linear shift slides the root directly onto the target pelvis marker.

15.2 Mathematics

15.2.1 The Two Directions

We measure two vectors on both the rig and the target human:

1. **Across the hips:**

$$\mathbf{v}_{\text{rig_hips}} = \mathbf{p}_{\text{l_upleg}} - \mathbf{p}_{\text{r_upleg}}, \quad \mathbf{v}_{\text{target_hips}} = \mathbf{T}_{\text{left_hip}} - \mathbf{T}_{\text{right_hip}}$$

2. **Up the spine:**

$$\mathbf{v}_{\text{rig_spine}} = \mathbf{p}_{\text{c_neck}} - \mathbf{p}_{\text{root}}, \quad \mathbf{v}_{\text{target_spine}} = \mathbf{T}_{\text{neck}} - \mathbf{T}_{\text{pelvis}}$$

15.2.2 Building Coordinate Frames (`frame_from_two_directions`)

Using the Gram-Schmidt method from `body_frame`, we construct two 3×3 orthonormal coordinate frame matrices:

1. **Source Frame** (F_{source}): The orientation the rig currently has:

$$F_{\text{source}} = \begin{bmatrix} \hat{\mathbf{x}}_{\text{src}} & \hat{\mathbf{y}}_{\text{src}} & \hat{\mathbf{z}}_{\text{src}} \end{bmatrix}$$

2. **Target Frame** (F_{target}): The orientation the human target has:

$$F_{\text{target}} = \begin{bmatrix} \hat{\mathbf{x}}_{\text{tgt}} & \hat{\mathbf{y}}_{\text{tgt}} & \hat{\mathbf{z}}_{\text{tgt}} \end{bmatrix}$$

Each column represents the unit X, Y, and Z axes of that body in space.

15.2.3 The Mathematical Derivation of `rotation_between_frames`

We need a single 3×3 rotation matrix R_{turn} that rotates the source frame so that it lands perfectly on the target frame:

$$R_{\text{turn}} \times F_{\text{source}} = F_{\text{target}}$$

How do we solve for R_{turn} ? Because F_{source} is an orthonormal matrix, its mathematical inverse is simply its **transpose**:

$$F_{\text{source}}^{-1} = F_{\text{source}}^T$$

Multiply both sides of the equation on the right by F_{source}^T :

$$R_{\text{turn}} \times \underbrace{(F_{\text{source}} \times F_{\text{source}}^T)}_{=I} = F_{\text{target}} \times F_{\text{source}}^T$$

$$R_{\text{turn}} = F_{\text{target}} \times F_{\text{source}}^T$$

This elegant equation computes the exact rotation matrix needed to turn one pair of directions onto another in a single operation!

15.2.4 Updating Root Rotation

We apply this corrective turn to the root's current world rotation:

$$R_{\text{new_world}} = R_{\text{turn}} \times R_{\text{root_current}}$$

We then pass $R_{\text{new_world}}$ to `rig.set_rotation`, which writes the Euler angles `root_rx`, `root_ry`, `root_rz` into `pose`.

15.2.5 Root Translation

Once the character is rotated, we run forward kinematics to find the root's updated world position: $\mathbf{p}_{\text{root}}$.

The distance the entire body needs to slide is:

$$\mathbf{s} = \mathbf{T}_{\text{pelvis}} - \mathbf{p}_{\text{root}} = \begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix}$$

Because MHR's root translation parameters are measured in units of 10.0 cm (`CENTIMETRES_PER_UNIT`), we convert:

$$\text{root_tx} = \frac{s_x}{10.0}, \quad \text{root_ty} = \frac{s_y}{10.0}, \quad \text{root_tz} = \frac{s_z}{10.0}$$

15.3 Hand-Worked Mathematical Example

Let's work through a practical numerical example by hand.

15.3.1 Step 1: Current Rig vs. Target Orientation

Suppose the rig currently stands aligned with the room axes:

$$F_{\text{source}} = I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(Hips point along +X, spine points along +Y, chest faces +Z).

Now suppose the real person in the video has turned 90° clockwise around the vertical axis to face the right wall (+X):

- Their chest now faces +X: $\hat{\mathbf{z}}_{\text{tgt}} = (1, 0, 0)$.
- Their spine still points straight up: $\hat{\mathbf{y}}_{\text{tgt}} = (0, 1, 0)$.
- Their hips now run along -Z: $\hat{\mathbf{x}}_{\text{tgt}} = (0, 0, -1)$.

The target frame matrix is:

$$F_{\text{target}} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

15.3.2 Step 2: Compute R_{turn}

Using our formula:

$$R_{\text{turn}} = F_{\text{target}} \times F_{\text{source}}^T$$

Since $F_{\text{source}} = I \Rightarrow F_{\text{source}}^T = I$:

$$R_{\text{turn}} = F_{\text{target}} \times I = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

Verification: Does R_{turn} carry the rig's hips $(1, 0, 0)$ onto the target's hips $(0, 0, -1)$?

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix} \quad \checkmark \text{ (Exact match!)}$$

15.3.3 Step 3: Compute Root Translation

After applying this rotation, forward kinematics reports that the rig's root sits at:

$$\mathbf{p}_{\text{root}} = (0.0, 0.0, 0.0) \text{ cm}$$

From the camera data, the target pelvis marker is located at:

$$\mathbf{T}_{\text{pelvis}} = (35.0, 92.0, -120.0) \text{ cm}$$

1. Compute shift vector:

$$\mathbf{s} = \mathbf{T}_{\text{pelvis}} - \mathbf{p}_{\text{root}} = \begin{bmatrix} 35.0 - 0.0 \\ 92.0 - 0.0 \\ -120.0 - 0.0 \end{bmatrix} = \begin{bmatrix} 35.0 \\ 92.0 \\ -120.0 \end{bmatrix} \text{ cm}$$

2. Convert to MHR units (10.0 cm/unit):

$$\begin{aligned} \text{root_tx} &= \frac{35.0}{10.0} = 3.5 \\ \text{root_ty} &= \frac{92.0}{10.0} = 9.2 \\ \text{root_tz} &= \frac{-120.0}{10.0} = -12.0 \end{aligned}$$

The character is now standing in the correct spot in the 3D room, turned to face the exact same direction as the subject!

15.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def solve_root(rig, pose, targets):
    """Place the root at the target's pelvis and turn it to face the same way.

    The root has no single bone to aim, so instead we match two directions at
    once: the line across the hips, and the line up the spine. Two directions
    pin down a rotation completely.
    """
    # Which way do the hips and the spine run on the rig as it stands?
    positions, rotations = rig.forward_kinematics(pose)

    rig_hips = (positions[rig.joint_index["l_upleg"]]
                - positions[rig.joint_index["r_upleg"]])
    rig_spine = (positions[rig.joint_index["c_neck"]]
                 - positions[rig.joint_index["root"]])

    # And in the target?
    target_hips = targets["left_hip"] - targets["right_hip"]
    target_spine = targets["neck"] - targets["pelvis"]

    # Turn the rig's pair of directions onto the target's pair.
    turn = rotation_between_frames(rig_hips, rig_spine, target_hips, target_spine)
    new_world_rotation = turn @ rotations[rig.joint_index["root"]]

    rig.set_rotation(pose, "root", ("root_rx", "root_ry", "root_rz"),
                    new_world_rotation, rotations)

    # Now slide the whole body so the root sits on the target's pelvis.
    positions, _ = rig.forward_kinematics(pose)
    shift = targets["pelvis"] - positions[rig.joint_index["root"]]

    for name, amount in zip(("root_tx", "root_ty", "root_tz"), shift):
        pose[rig.parameter_index[name]] = amount / CENTIMETRES_PER_UNIT

    return pose
```

15.4.1 Line-by-Line Breakdown

1. `positions, rotations = rig.forward_kinematics(pose)`
 - Runs forward kinematics to see where the joints currently sit after the build calibration steps (`match_bone_lengths` and `match_hip_offset`).
2. `rig_hips = positions[rig.joint_index["l_upleg"]] - positions[rig.joint_index["r_upleg"]]`
`rig_spine = positions[rig.joint_index["c_neck"]] - positions[rig.joint_index["root"]]`
 - Extracts the rig's current hip direction and spine direction in centimeters.
3. `target_hips = targets["left_hip"] - targets["right_hip"]`
`target_spine = targets["neck"] - targets["pelvis"]`
 - Extracts the target person's hip direction and spine direction from the camera tracking markers.
4. `turn = rotation_between_frames(rig_hips, rig_spine, target_hips, target_spine)`
 - Calls the Part 1 helper function `rotation_between_frames`:

```
source = frame_from_two_directions(from_main, from_side)
target = frame_from_two_directions(to_main, to_side)
return target @ source.T
```
 - Evaluates $R_{\text{turn}} = F_{\text{target}} \times F_{\text{source}}^T$ (Section 2C).
5. `new_world_rotation = turn @ rotations[rig.joint_index["root"]]`
 - Multiplies the corrective turn by the root's current world rotation to find its desired final orientation in the room.

```
6 | rig.set_rotation(pose, "root", ("root_rx", "root_ry", "root_rz"),
    | new_world_rotation, rotations)
```

- Calls `set_rotation` (which we studied in Part 2) to factor out pre-rotations and write the Euler angles into `root_rx`, `root_ry`, and `root_rz`.

```
7 | positions, _ = rig.forward_kinematics(pose)
```

- Re-evaluates forward kinematics so we know where the newly rotated root joint actually sits in world coordinates.

```
8 | shift = targets["pelvis"] - positions[rig.joint_index["root"]]
```

- Computes the remaining linear distance from the rig's root to the target pelvis marker.

```
9 | for name, amount in zip(("root_tx", "root_ty", "root_tz"), shift):
    | pose[rig.parameter_index[name]] = amount / CENTIMETRES_PER_UNIT
```

- Divides the (X, Y, Z) shifts by 10.0 and writes them into `root_tx`, `root_ty`, and `root_tz`.

```
10. python    return pose
```

- Hands back the updated pose vector.

15.5 Connection to the Overall Algorithm

`solve_root` is the foundation of the entire posing pass:

```

solve(rig, targets)
|
v
[ Build Calibration Steps ]
(Bone lengths & hip offset fixed)
|
v
[ solve_root ] <--- YOU ARE HERE
(Positions & turns pelvis in world space)
|
v
[ solve_spine ]
(Bends torso toward neck)
|
v
[ solve_simple_bone / solve_limb ]
(Collars, Head, Arms, and Legs)
```

By placing the root at the pelvis and pointing it in the right direction, every child joint (the spine and the upper legs) now starts from the correct anatomical anchor in space.

SECTION 16

solve_spine

16.1 Theory, Purpose, and Intuition

16.1.1 What Does the Spine Control?

In human anatomy, the spine connects the pelvis to the neck and head. More importantly, attached to the upper torso are the **collarbones and shoulders**.

- If you lean your back forward, your shoulders and head move forward.
- If you twist your chest to look over your shoulder, your shoulders swing around in a circle.

In MHR, the lowest spine joint that controls the torso's flexibility is called **c_spine0**. It has three rotational dials defined by **SPINE_PARAMETERS**:

- **spine0_rx_flexible** (twist / roll along the spine)
- **spine0_ry_flexible** (side-lean / tilt)
- **spine0_rz_flexible** (forward bend / pitch)

16.1.2 The “Broomstick” Problem: Why Aiming Alone is Not Enough

Imagine holding a broomstick upright on the floor with your palm on top:

1. You can tilt the broomstick forward, backward, left, or right so its tip points toward any target point in the room. This takes care of **2 degrees of freedom**.
2. But even while pointing straight at that target, **you can still freely spin the broomstick between your hands!**

The Spine Twist Problem

```

(neck)
 *
 /|
 *-----* right_shoulder
 |
 | <-- Spine is aimed at neck,
 |      but spinning it swings the
 *      shoulders round like airplane wings!
spine0

```

Because the shoulders are rigidly attached across the top of the torso, spinning the spine about its own length swings the shoulders through the air!

- If we *only* aimed the spine at the neck, the puppet's neck would land in the right place, but its chest might be twisted backwards or sideways.
- To eliminate this rotational ambiguity, we need a **second direction**: the line running across the shoulders from right shoulder to left shoulder.

By matching **both** the spine direction (**up**) and the shoulder line (**across**), the solver locks down the torso completely: the neck hits its target point, and the chest faces the exact direction the subject is facing.

16.2 Mathematics

16.2.1 Measuring Current vs. Wanted Directions

We identify four key vectors in 3D space:

1. **Current Rig Directions:** Where is the spine pointing, and where are the shoulders pointing on the puppet right now?

$$\mathbf{v}_{\text{current_up}} = \mathbf{p}_{\text{c_neck}} - \mathbf{p}_{\text{c_spine0}}$$

$$\mathbf{v}_{\text{current_across}} = \mathbf{p}_{\text{l_uparm}} - \mathbf{p}_{\text{r_uparm}}$$

2. **Wanted Target Directions:** Where should the spine point, and where should the shoulders point based on the camera data?

$$\mathbf{v}_{\text{wanted_up}} = \mathbf{T}_{\text{neck}} - \mathbf{p}_{\text{c_spine0}}$$

$$\mathbf{v}_{\text{wanted_across}} = \mathbf{T}_{\text{left_shoulder}} - \mathbf{T}_{\text{right_shoulder}}$$

Notice something critical about $\mathbf{v}_{\text{wanted_up}}$: We compute $\mathbf{T}_{\text{neck}} - \mathbf{p}_{\text{c_spine0}}$ (pointing from the rig's actual spine base to the target neck). As we learned in Part 3, aiming directly at the target marker prevents any position errors caused by differences in marker placement.

16.2.2 Building Orthonormal Frames via Gram-Schmidt

Just like in `solve_root`, we use `frame_from_two_directions` to turn these pairs of rough directions into proper 3×3 orthonormal coordinate frames:

1. **Source Frame** (F_{current}):
 - Primary axis $\hat{\mathbf{x}}_{\text{curr}}$: Normalized $\mathbf{v}_{\text{current_up}}$ (along the spine).
 - Secondary axis $\hat{\mathbf{y}}_{\text{curr}}$: Normalized $\mathbf{v}_{\text{current_across}}$ with any tilt along the spine subtracted off via Gram-Schmidt.
 - Third axis $\hat{\mathbf{z}}_{\text{curr}}$: Cross product $\hat{\mathbf{x}}_{\text{curr}} \times \hat{\mathbf{y}}_{\text{curr}}$ (out of the chest).
2. **Target Frame** (F_{wanted}):
 - Primary axis $\hat{\mathbf{x}}_{\text{want}}$: Normalized $\mathbf{v}_{\text{wanted_up}}$.
 - Secondary axis $\hat{\mathbf{y}}_{\text{want}}$: Normalized $\mathbf{v}_{\text{wanted_across}}$ orthogonalized against $\hat{\mathbf{x}}_{\text{want}}$.
 - Third axis $\hat{\mathbf{z}}_{\text{want}}$: Cross product $\hat{\mathbf{x}}_{\text{want}} \times \hat{\mathbf{y}}_{\text{want}}$.

16.2.3 The Turn Matrix (R_{turn})

We compute the 3×3 rotation matrix that carries the current frame onto the wanted frame:

$$R_{\text{turn}} = F_{\text{wanted}} \times F_{\text{current}}^T$$

16.2.4 Updating `c_spine0`

We apply this corrective turn to `c_spine0`'s current world rotation:

$$R_{\text{new_world}} = R_{\text{turn}} \times R_{\text{spine0_current}}$$

We then pass $R_{\text{new_world}}$ to `rig.set_rotation`, which peels away the parent rotation (from the root) and the pre-rotation, and stores the resulting Euler angles into `SPINE_PARAMETERS`.

16.3 Hand-Worked Mathematical Example

Let's work through a numerical example where the spine is already pointing straight up, but the person has **twisted their torso by 90°** .

16.3.1 Step 1: Current Rig State

- The spine base sits at `here` = $(0, 10, 0)$ cm.
- The neck sits at $(0, 50, 0)$ cm.

$$\mathbf{v}_{\text{current_up}} = \begin{bmatrix} 0 - 0 \\ 50 - 10 \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 40 \\ 0 \end{bmatrix} \Rightarrow \hat{\mathbf{x}}_{\text{curr}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

- The rig's shoulders run along the X-axis: left shoulder at $(15, 45, 0)$, right shoulder at $(-15, 45, 0)$:

$$\mathbf{v}_{\text{current_across}} = \begin{bmatrix} 15 - (-15) \\ 45 - 45 \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} 30 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \hat{\mathbf{y}}_{\text{curr}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

- Third axis (chest facing):

$$\hat{\mathbf{z}}_{\text{curr}} = \hat{\mathbf{x}}_{\text{curr}} \times \hat{\mathbf{y}}_{\text{curr}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}$$

The current frame is:

$$F_{\text{current}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

16.3.2 Step 2: Wanted Target State

Suppose the target person's neck is also straight up at (0, 50, 0) cm:

$$\mathbf{v}_{\text{wanted_up}} = \begin{bmatrix} 0 \\ 40 \\ 0 \end{bmatrix} \Rightarrow \hat{\mathbf{x}}_{\text{want}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

However, the person has twisted their torso so their shoulders run along the **Z-axis** (left shoulder at (0, 45, 15), right shoulder at (0, 45, -15)):

$$\mathbf{v}_{\text{wanted_across}} = \begin{bmatrix} 0 - 0 \\ 45 - 45 \\ 15 - (-15) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 30 \end{bmatrix} \Rightarrow \hat{\mathbf{y}}_{\text{want}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

■ Third axis:

$$\hat{\mathbf{z}}_{\text{want}} = \hat{\mathbf{x}}_{\text{want}} \times \hat{\mathbf{y}}_{\text{want}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

The wanted frame is:

$$F_{\text{wanted}} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

16.3.3 Step 3: Compute R_{turn}

First, transpose F_{current} (swap rows and columns):

$$F_{\text{current}}^T = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

Now multiply:

$$R_{\text{turn}} = F_{\text{wanted}} \times F_{\text{current}}^T = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

- Row 0: $[(0)(0) + (0)(1) + (1)(0), (0)(1) + (0)(0) + (1)(0), (0)(0) + (0)(0) + (1)(-1)] = [0, 0, -1]$
- Row 1: $[(1)(0) + (0)(1) + (0)(0), (1)(1) + (0)(0) + (0)(0), (1)(0) + (0)(0) + (0)(-1)] = [0, 1, 0]$
- Row 2: $[(0)(0) + (1)(1) + (0)(0), (0)(1) + (1)(0) + (0)(0), (0)(0) + (1)(0) + (0)(-1)] = [1, 0, 0]$

$$R_{\text{turn}} = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

16.3.4 Verification

What does this matrix do to the current shoulder direction $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$?

$$\begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

It rotates the shoulders from pointing along X to pointing along Z!

And what does it do to the spine direction $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$?

$$\begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

The spine is left pointing straight up. This is a pure 90° torso twist.

16.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
SPINE_PARAMETERS = ("spine0_rx_flexible", "spine0_ry_flexible", "spine0_rz_flexible")

def solve_spine(rig, pose, targets):
    """Swing the spine until the neck lands right, and twist it so the shoulders line up.

    Aiming on its own is not enough here. A bone that is merely aimed is still
    free to spin about its own length, and spinning the spine carries the
    shoulders round with it. The line across the shoulders pins that last
    freedom down, so we match two directions at once, exactly as we did at the
    root.
    """
    positions, rotations = rig.forward_kinematics(pose)
    here = positions[rig.joint_index["c_spine0"]]

    current_up = positions[rig.joint_index["c_neck"]] - here
    current_across = (positions[rig.joint_index["l_uparm"]]
                     - positions[rig.joint_index["r_uparm"]])

    wanted_up = targets["neck"] - here
    wanted_across = targets["left_shoulder"] - targets["right_shoulder"]

    turn = rotation_between_frames(current_up, current_across, wanted_up, wanted_across)
    new_world_rotation = turn @ rotations[rig.joint_index["c_spine0"]]
    rig.set_rotation(pose, "c_spine0", SPINE_PARAMETERS, new_world_rotation, rotations)
    return pose
```

16.4.1 Line-by-Line Breakdown

- 1 `SPINE_PARAMETERS = ("spine0_rx_flexible", "spine0_ry_flexible", "spine0_rz_flexible")`
 - Defines the exact tuple of parameter names controlling `c_spine0`'s 3D rotation in MHR.
- 2 `positions, rotations = rig.forward_kinematics(pose)`
`here = positions[rig.joint_index["c_spine0"]]`
 - Runs forward kinematics so the spine solver sees where joints are after `solve_root` placed the pelvis.
 - `here`: The (X, Y, Z) coordinates of the base joint `c_spine0`.
- 3 `current_up = positions[rig.joint_index["c_neck"]] - here`
`current_across = (positions[rig.joint_index["l_uparm"]]`
 `- positions[rig.joint_index["r_uparm"]])`
 - Measures where the puppet's spine and shoulders currently point.
- 4 `wanted_up = targets["neck"] - here`
`wanted_across = targets["left_shoulder"] - targets["right_shoulder"]`
 - Calculates the target spine direction (from current spine base to target neck marker) and target shoulder direction.
- 5 `turn = rotation_between_frames(current_up, current_across, wanted_up, wanted_across)`
 - Computes $R_{\text{turn}} = F_{\text{wanted}} \times F_{\text{current}}^T$ to carry the current pair of directions onto the target pair.

```
6 | new_world_rotation = turn @ rotations[rig.joint_index["c_spine0"]]
```

- Applies the corrective rotation to `c_spine0`'s current world rotation matrix.

```
7 | rig.set_rotation(pose, "c_spine0", SPINE_PARAMETERS, new_world_rotation, rotations)
```

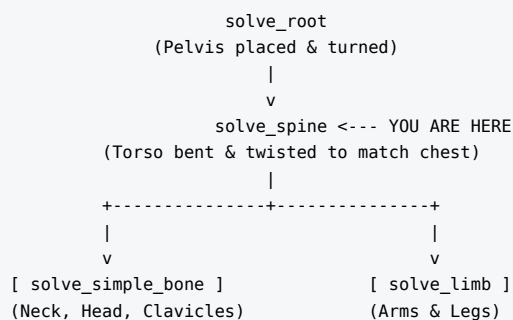
- Calls `set_rotation` to decompose `new_world_rotation` into Euler angles and update `pose`.

```
8 | return pose
```

- Returns the updated pose vector.

16.5 Connection to the Overall Algorithm

`solve_spine` is the bridge between the pelvis and the upper body:



Because `solve_spine` correctly aligns the chest, the base points for the collarbones (`l_clavicle`, `r_clavicle`) and neck (`c_neck`) are now positioned where they belong in 3D space, allowing subsequent solvers to aim those bones directly at their respective target markers.

SECTION 17

solve_simple_bone

17.1 Theory, Purpose, and Intuition

17.1.1 What is a “Simple Bone”?

In our body, some bones are parts of complex hinge mechanisms (like elbows and knees), while others function like simple pointers pivoting around a ball joint:

- **Collarbones** (`l_clavicle`, `r_clavicle`): Pivot at the base of the throat to swing the shoulder socket into place.
- **Neck** (`c_neck`): Pivots at the top of the spine to point toward the head.
- **Head** (`c_head`): Pivots on the neck to point toward the top of the skull.

For these bones, we do not need to worry about complex roll alignments or bending planes. All we want is: > *“Take this bone, and swing it directly until its tip points straight at the target marker.”*



17.1.2 What is the “Shortest Arc” Rotation?

Imagine you have a toy globe of the Earth.

- Suppose there is a sticker on London (vector **a**).
- You want to rotate the globe so that the sticker ends up where Tokyo is (vector **b**).

There are infinitely many crazy ways you could spin and tumble the globe to get London to Tokyo. But there is only **one single, most direct way**:

1. Find the single axis that stands perpendicular to both London and Tokyo.
2. Spin the globe around that axis by the exact angle between them.

This minimal, direct rotation is called the **shortest arc rotation**. It changes the bone’s pointing direction without introducing any unnecessary twists along its length.

17.2 Mathematics

To understand `solve_simple_bone`, we must examine the mathematics of the helper function it calls from Part 1: `shortest_arc_rotation`.

17.2.1 The Direction Vectors

Let:

- $\mathbf{p}_{\text{base}}$ = 3D position of the joint being rotated (`here`).
- $\mathbf{p}_{\text{tip}}$ = current 3D position of its child endpoint (`tip`).
- $\mathbf{T}_{\text{aim}}$ = 3D position of the target marker (`aim_at`).

The current pointing vector and the wanted pointing vector are:

$$\mathbf{v}_{\text{current}} = \mathbf{p}_{\text{tip}} - \mathbf{p}_{\text{base}}$$

$$\mathbf{v}_{\text{wanted}} = \mathbf{T}_{\text{aim}} - \mathbf{p}_{\text{base}}$$

We normalize both to unit vectors (length 1):

$$\hat{\mathbf{a}} = \frac{\mathbf{v}_{\text{current}}}{\|\mathbf{v}_{\text{current}}\|}, \quad \hat{\mathbf{b}} = \frac{\mathbf{v}_{\text{wanted}}}{\|\mathbf{v}_{\text{wanted}}\|}$$

17.2.2 Finding the Rotation Axis ($\hat{\mathbf{u}}$)

The axis perpendicular to both $\hat{\mathbf{a}}$ and $\hat{\mathbf{b}}$ is their **vector cross product**:

$$\mathbf{k} = \hat{\mathbf{a}} \times \hat{\mathbf{b}}$$

The length of this cross product is:

$$\|\mathbf{k}\| = \|\hat{\mathbf{a}}\| \|\hat{\mathbf{b}}\| \sin(\theta) = \sin(\theta)$$

(since $\|\hat{\mathbf{a}}\| = \|\hat{\mathbf{b}}\| = 1$).

We scale $\mathbf{k}$ to length 1 to get the unit rotation axis:

$$\hat{\mathbf{u}} = \frac{\mathbf{k}}{\|\mathbf{k}\|} = \frac{\hat{\mathbf{a}} \times \hat{\mathbf{b}}}{\|\hat{\mathbf{a}} \times \hat{\mathbf{b}}\|}$$

17.2.3 Finding the Rotation Angle (θ)

The **dot product** of the two unit vectors gives the cosine of the angle between them:

$$\hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = \|\hat{\mathbf{a}}\| \|\hat{\mathbf{b}}\| \cos(\theta) = \cos(\theta)$$

To compute the angle θ accurately without numerical instability, we use the 2-argument arctangent:

$$\theta = \text{atan2}(\|\mathbf{k}\|, \hat{\mathbf{a}} \cdot \hat{\mathbf{b}}) = \text{atan2}(\sin \theta, \cos \theta)$$

17.2.4 Two Special Edge Cases

In `shortest_arc_rotation`, there are two situations where the cross product $\mathbf{k} = \hat{\mathbf{a}} \times \hat{\mathbf{b}}$ has length zero:

1. **Vectors already point in the exact same direction ($\theta = 0^\circ$):**
 - $\hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = 1$ and $\|\mathbf{k}\| \approx 0$.
 - Nothing needs to move! The function returns the 3×3 Identity Matrix I .
2. **Vectors point in exactly opposite directions ($\theta = 180^\circ$):**
 - $\hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = -1$ and $\|\mathbf{k}\| \approx 0$.
 - The bone needs to do a complete 180° U-turn. Any axis perpendicular to $\hat{\mathbf{a}}$ will work. The code picks a helper vector $(1, 0, 0)$ (or $(0, 1, 0)$ if parallel), computes $\hat{\mathbf{u}} = \text{normalise}(\hat{\mathbf{a}} \times \text{helper})$, and rotates by π radians (180°).

17.2.5 Rodrigues' Rotation Formula (Axis-Angle to Matrix)

Once we have the unit axis $\hat{\mathbf{u}} = (u_x, u_y, u_z)$ and angle θ , we convert them into a 3×3 rotation matrix using **Rodrigues' Rotation Formula**:

$$R = I + (\sin \theta)K + (1 - \cos \theta)K^2$$

Where K is the skew-symmetric cross-product matrix:

$$K = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

SciPy's `Rotation.from_rotvec(axis * angle).as_matrix()` performs this exact calculation.

17.2.6 Applying the Turn to the Joint

We multiply the turn matrix by the joint's current world rotation:

$$R_{\text{new_world}} = R_{\text{turn}} \times R_{\text{joint_current}}$$

Then `rig.set_rotation` writes the corresponding Euler angles into `pose`.

17.3 Hand-Worked Mathematical Example

Let's trace a collarbone being swung by hand with simple numbers.

17.3.1 The Setup

- Base of the collarbone sits at: **here** = (0.0, 0.0, 0.0) cm.
- The tip currently points straight up along +Y:

$$\mathbf{v}_{\text{current}} = \begin{bmatrix} 0.0 \\ 10.0 \\ 0.0 \end{bmatrix} \Rightarrow \hat{\mathbf{a}} = \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix}$$

- The target shoulder marker is at (10.0, 0.0, 0.0) cm (along +X):

$$\mathbf{v}_{\text{wanted}} = \begin{bmatrix} 10.0 \\ 0.0 \\ 0.0 \end{bmatrix} \Rightarrow \hat{\mathbf{b}} = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

17.3.2 Step 1: Compute Cross Product (Rotation Axis)

$$\mathbf{k} = \hat{\mathbf{a}} \times \hat{\mathbf{b}} = \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix} \times \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

- $k_x = (1)(0) - (0)(0) = 0$
- $k_y = (0)(1) - (0)(0) = 0$
- $k_z = (0)(0) - (1)(1) = -1$

$$\mathbf{k} = \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix}$$

Length: $\|\mathbf{k}\| = \sqrt{0^2 + 0^2 + (-1)^2} = 1.0$. Unit axis: $\hat{\mathbf{u}} = (0.0, 0.0, -1.0)$ (pointing down the $-Z$ axis).

17.3.3 Step 2: Compute Dot Product and Angle

$$\hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = (0)(1) + (1)(0) + (0)(0) = 0.0$$

$$\theta = \text{atan2}(\|\mathbf{k}\|, \hat{\mathbf{a}} \cdot \hat{\mathbf{b}}) = \text{atan2}(1.0, 0.0) = \frac{\pi}{2} = 90^\circ$$

17.3.4 Step 3: Compute Rotation Matrix (R_{turn})

Rotating by 90° around axis $(0, 0, -1)$ is equivalent to a -90° rotation around $+Z$:

$$R_{\text{turn}} = \begin{bmatrix} \cos(-90^\circ) & -\sin(-90^\circ) & 0 \\ \sin(-90^\circ) & \cos(-90^\circ) & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

17.3.5 Verification

Does this matrix rotate the bone from $\hat{\mathbf{a}}$ onto $\hat{\mathbf{b}}$?

$$R_{\text{turn}} \hat{\mathbf{a}} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} = \hat{\mathbf{b}} \quad \checkmark \text{ (Exact match!)}$$

The bone has swung from +Y to +X along the shortest possible arc!

17.4 Code Implementation

Here is the code for `solve_simple_bone` in `analytic_ik.py`:

```
def solve_simple_bone(rig, pose, targets, joint, tip, aim_at, parameters):
    """Swing one bone until its tip points at a target keypoint.

    Both directions are measured from where the rig's joint actually is, so this
    stays correct even when the rig joint and the target keypoint do not sit in
    quite the same place.
    """
    positions, rotations = rig.forward_kinematics(pose)
    here = positions[rig.joint_index[joint]]

    current_direction = positions[rig.joint_index[tip]] - here
    wanted_direction = targets[aim_at] - here

    turn = shortest_arc_rotation(current_direction, wanted_direction)
    new_world_rotation = turn @ rotations[rig.joint_index[joint]]
    rig.set_rotation(pose, joint, parameters, new_world_rotation, rotations)
    return pose
```

And here is the helper function from Part 1 that it calls:

```
def shortest_arc_rotation(from_direction, to_direction):
    """The smallest rotation that turns one direction into another.

    Think of spinning a globe so that one city ends up where another was: there
    is a single axis to spin about (perpendicular to both) and a single angle.
    """
    a = normalise(from_direction)
    b = normalise(to_direction)

    axis = np.cross(a, b)
    axis_length = np.linalg.norm(axis)

    # The two directions already agree, so there is nothing to do.
    if axis_length < 1e-9 and np.dot(a, b) > 0:
        return np.eye(3)

    # The two directions are exactly opposite. Any perpendicular axis turns one
    # into the other; pick one that is definitely not parallel to a.
    if axis_length < 1e-9:
        helper = np.array([1.0, 0.0, 0.0])
        if abs(np.dot(a, helper)) > 0.9:
            helper = np.array([0.0, 1.0, 0.0])
        axis = normalise(np.cross(a, helper))
        return Rotation.from_rotvec(axis * np.pi).as_matrix()

    angle = np.arctan2(axis_length, np.dot(a, b))
    return Rotation.from_rotvec(normalise(axis) * angle).as_matrix()
```

17.4.1 Line-by-Line Breakdown

1. positions, rotations = rig.forward_kinematics(pose)
here = positions[rig.joint_index[joint]]

- Evaluates forward kinematics so the solver knows where the base joint **joint** is located right now after its parent was posed.

2. current_direction = positions[rig.joint_index[tip]] - here
wanted_direction = targets[aim_at] - here

- Computes the vector pointing from **here** to the current tip position, and the vector pointing from **here** to the target marker.

3. turn = shortest_arc_rotation(current_direction, wanted_direction)

- Calls the helper:
 - Normalizes both directions into unit vectors $\hat{\mathbf{a}}$ and $\hat{\mathbf{b}}$.
 - Checks if they already agree or are exactly opposite.
 - Computes rotation axis $\hat{\mathbf{u}} = \text{normalise}(\hat{\mathbf{a}} \times \hat{\mathbf{b}})$.
 - Computes angle $\theta = \arctan2(\text{axis_length}, \hat{\mathbf{a}} \cdot \hat{\mathbf{b}})$.

- Generates the 3×3 rotation matrix via Rodrigues' formula (`Rotation.from_rotvec(...)`).

```
4 | new_world_rotation = turn @ rotations[rig.joint_index[joint]]
```

- Applies the shortest-arc turn matrix to the joint's current world rotation.

```
5 | rig.set_rotation(pose, joint, parameters, new_world_rotation, rotations)
```

- Calls `set_rotation` to decompose `new_world_rotation` into the three specified Euler parameters and write them into `pose`.

```
6 | return pose
```

- Returns the updated pose vector.

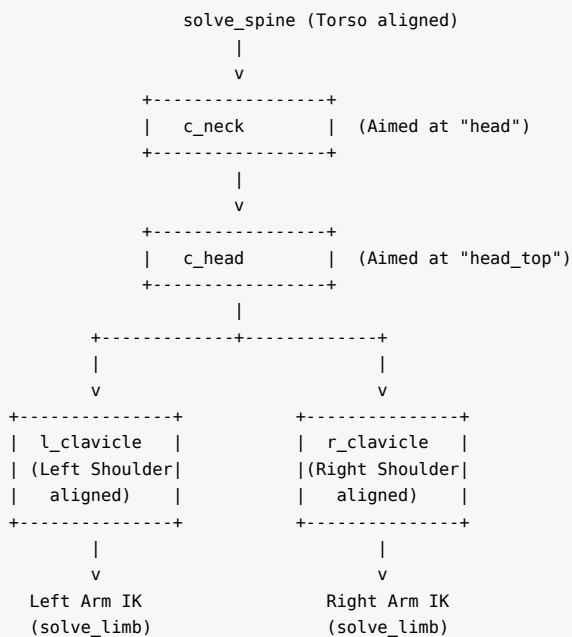
17.5 Connection to the Overall Algorithm

In the main `solve(...)` pipeline, `solve_simple_bone` is called inside a loop over the `SIMPLE_BONES` table:

```
for joint, tip, aim_at, parameters in SIMPLE_BONES:
    pose = solve_simple_bone(rig, pose, targets, joint, tip, aim_at, parameters)
```

Here is the exact order in which it runs:

1. **l_clavicle**: Swings left collarbone so `l_uparm` hits `left_shoulder`.
2. **r_clavicle**: Swings right collarbone so `r_uparm` hits `right_shoulder`.
3. **c_neck**: Swings neck bone so `c_head` hits `head`.
4. **c_head**: Swings head bone so `c_head_null` hits `head_top`.



Because each simple bone runs forward kinematics at the start, the head is solved *after* the neck has already moved, and the arm sockets are solved *after* the collarbones have already placed the shoulders in position.

SECTION 18

solve_limb

18.1 Theory, Purpose, and Intuition

18.1.1 What is a Two-Bone Limb?

In the human body, the arms and legs are two-segment mechanical chains:

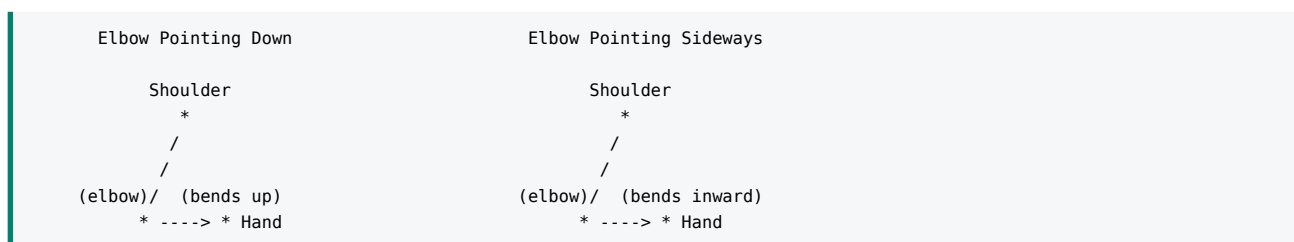
- **Arm:** Shoulder (upper) → Elbow (hinge) → Wrist (tip)
- **Leg:** Hip (upper) → Knee (hinge) → Ankle (tip)

Unlike a simple bone (like a neck or collarbone), you cannot solve an arm or leg simply by aiming bones one at a time.

18.1.2 The “Elbow Roll” Dilemma

Try this physical experiment with your own arm right now:

1. Hold your arm straight out in front of you.
2. Without moving your hand or shoulder position, **roll your shoulder**.
 - Notice how your elbow’s “point” can face toward the floor, toward the wall, or even toward the ceiling!
3. Now, try bending your elbow.
 - If your elbow pointed down, your hand moves straight up toward your chest.
 - If your elbow pointed out to the side, your hand swings horizontally across your chest.



Because the elbow is a **hinge joint** (it only bends in one plane), **which plane it bends in is determined entirely by how the upper arm is rolled!**

- If you aim the upper arm but roll it the wrong way, bending the elbow will send the wrist off into empty space, completely missing the target.

18.1.3 The Geometric Solution: The Limb Triangle

The three target points (Shoulder, Elbow, Wrist) form a 2D triangle floating in 3D space:

1. The **Limb Plane** is the flat surface containing all three points.
2. The axis that the elbow must bend around **must stand perpendicular (at a 90° right angle) to this triangle**.
3. In vector mathematics, the perpendicular vector to any two vectors is given by their **cross product**!

Therefore, the target cross product $\mathbf{w}_{\text{upper}} \times \mathbf{w}_{\text{lower}}$ gives us the exact hinge direction for free. We roll the upper arm until the rig’s hinge pin points along that axis, and then bend the elbow directly toward the wrist.

18.2 Mathematics

18.2.1 Target Vectors and the Hinge Normal

From the target dictionary:

- Upper bone vector: $\mathbf{w}_{\text{upper}} = \mathbf{T}_{\text{elbow}} - \mathbf{T}_{\text{shoulder}}$
- Lower bone vector: $\mathbf{w}_{\text{lower}} = \mathbf{T}_{\text{wrist}} - \mathbf{T}_{\text{elbow}}$

The vector perpendicular to the triangle formed by the arm is:

$$\mathbf{w}_{\text{axis}} = \mathbf{w}_{\text{upper}} \times \mathbf{w}_{\text{lower}}$$

18.2.2 The “Dead Straight” Collinear Edge Case

What if the person’s arm is completely straight? If the shoulder, elbow, and wrist lie in a single straight line, the angle between the bones is 0° (or 180°). The cross product of two parallel vectors is zero:

$$\|\mathbf{w}_{\text{upper}} \times \mathbf{w}_{\text{lower}}\| = \|\mathbf{w}_{\text{upper}}\| \|\mathbf{w}_{\text{lower}}\| \sin(0^\circ) = 0$$

If $\mathbf{w}_{\text{axis}} = \mathbf{0}$, no unique triangle exists, and no bend plane is defined! The code checks for this condition:

$$\|\mathbf{w}_{\text{axis}}\| < 10^{-6} \cdot \|\mathbf{w}_{\text{upper}}\| \|\mathbf{w}_{\text{lower}}\|$$

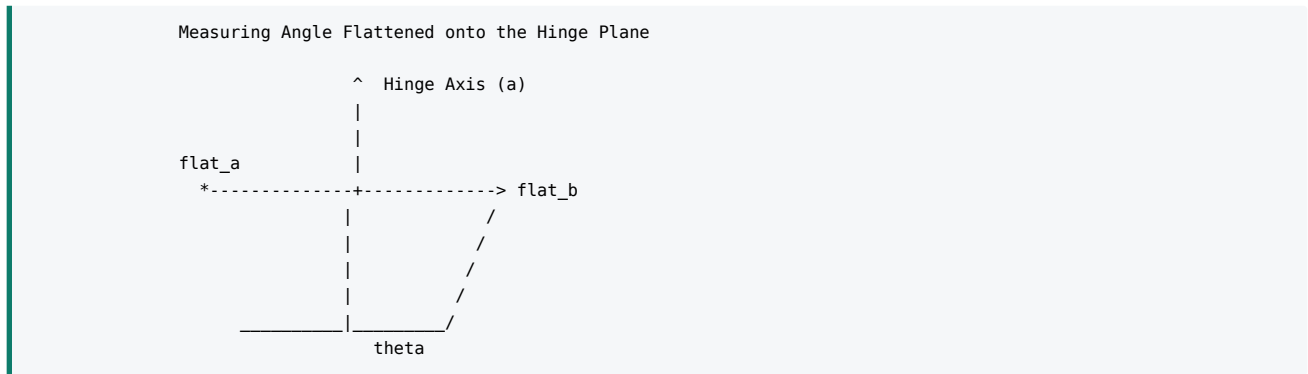
- **If dead straight:** Roll does not matter because the elbow won’t bend anyway. We simply aim the upper bone using `shortest_arc_rotation`.
- **If bent:** We use `rotation_between_frames` to match both the upper bone direction ($\mathbf{w}_{\text{upper}}$) and the hinge axis ($\mathbf{w}_{\text{axis}}$) simultaneously.

18.2.3 Measuring the Bend: `signed_angle_about_axis`

After the upper arm is aimed and rolled, we run forward kinematics. The elbow is now in the right place, and its hinge axis $\hat{\mathbf{a}}$ is aligned with the target plane.

Now we must answer: **By what angle θ should the hinge bend around axis $\hat{\mathbf{a}}$?**

To measure this angle without any 3D distortion, `signed_angle_about_axis` (from Part 1) uses the following mathematics:



1. **Flatten the vectors onto the hinge plane:** Any part of the forearm vector that accidentally tilts along the hinge axis $\hat{\mathbf{a}}$ is irrelevant to the bend. We subtract off the projection along $\hat{\mathbf{a}}$:

$$\mathbf{v}_{\text{flat}} = \mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{a}}) \hat{\mathbf{a}}$$

We do this for both the current forearm ($\mathbf{c}_{\text{lower}}$) and the target forearm ($\mathbf{w}_{\text{lower}}$), then normalize them to unit vectors $\hat{\mathbf{f}}_a$ and $\hat{\mathbf{f}}_b$.

2. **Compute Cosine and Sine:**

- The dot product gives the cosine:

$$\cos(\theta) = \hat{\mathbf{f}}_a \cdot \hat{\mathbf{f}}_b$$

- The cross product $\hat{\mathbf{f}}_a \times \hat{\mathbf{f}}_b$ produces a vector pointing along $\pm \hat{\mathbf{a}}$. Taking its dot product with $\hat{\mathbf{a}}$ gives the signed sine:

$$\sin(\theta) = (\hat{\mathbf{f}}_a \times \hat{\mathbf{f}}_b) \cdot \hat{\mathbf{a}}$$

3. **Compute the Signed Angle:**

$$\theta = \text{atan2}(\sin \theta, \cos \theta)$$

The sign follows the **right-hand rule** around $\hat{\mathbf{a}}$. This single floating-point number is written directly into `l_elbow_bend` or `r_knee_bend`.

18.3 Hand-Worked Mathematical Example

Let’s solve an arm by hand where the elbow bends by 90° .

18.3.1 Step 1: Target Coordinates

- Shoulder: $\mathbf{T}_{\text{shoulder}} = (0.0, 0.0, 0.0)$ cm
- Elbow: $\mathbf{T}_{\text{elbow}} = (20.0, 0.0, 0.0)$ cm (upper arm along +X)
- Wrist: $\mathbf{T}_{\text{wrist}} = (20.0, -15.0, 0.0)$ cm (forearm bends down along -Y)

1. Compute target bone vectors:

$$\mathbf{w}_{\text{upper}} = \begin{bmatrix} 20 - 0 \\ 0 - 0 \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} 20.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

$$\mathbf{w}_{\text{lower}} = \begin{bmatrix} 20 - 20 \\ -15 - 0 \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ -15.0 \\ 0.0 \end{bmatrix}$$

2. Compute target hinge axis ($\mathbf{w}_{\text{axis}}$):

$$\mathbf{w}_{\text{axis}} = \mathbf{w}_{\text{upper}} \times \mathbf{w}_{\text{lower}} = \begin{bmatrix} 20.0 \\ 0.0 \\ 0.0 \end{bmatrix} \times \begin{bmatrix} 0.0 \\ -15.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \\ -300.0 \end{bmatrix}$$

Normalized unit axis:

$$\hat{\mathbf{w}}_{\text{axis}} = \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix}$$

(The hinge pin must point down the -Z axis!)

18.3.2 Step 2: Roll and Aim the Upper Arm

Suppose currently on the rig, the upper arm points along +X (20, 0, 0), but its hinge axis points along +Z (0, 0, 1) instead of -Z.

`rotation_between_frames` computes the turn that leaves the bone pointing along +X while flipping the hinge axis from +Z to -Z. This is a pure 180° roll around the upper arm's length!

We apply this rotation to the shoulder using `rig.set_rotation`.

18.3.3 Step 3: Measure the Hinge Bend

Now we re-run forward kinematics. The shoulder is rolled, and the forearm is currently unbent (straight out along +X):

- Current forearm: $\mathbf{c}_{\text{lower}} = \begin{bmatrix} 15.0 \\ 0.0 \\ 0.0 \end{bmatrix} \Rightarrow \hat{\mathbf{f}}_a = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix}$
- Target forearm: $\mathbf{w}_{\text{lower}} = \begin{bmatrix} 0.0 \\ -15.0 \\ 0.0 \end{bmatrix} \Rightarrow \hat{\mathbf{f}}_b = \begin{bmatrix} 0.0 \\ -1.0 \\ 0.0 \end{bmatrix}$
- Hinge axis: $\hat{\mathbf{a}} = \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix}$

Let's compute `signed_angle_about_axis`:

1. **Cosine:**

$$\cos(\theta) = \hat{\mathbf{f}}_a \cdot \hat{\mathbf{f}}_b = (1)(0) + (0)(-1) + (0)(0) = 0.0$$

2. **Cross product:**

$$\hat{\mathbf{f}}_a \times \hat{\mathbf{f}}_b = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} \times \begin{bmatrix} 0.0 \\ -1.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix}$$

3. Sine:

$$\sin(\theta) = (\hat{\mathbf{f}}_a \times \hat{\mathbf{f}}_b) \cdot \hat{\mathbf{a}} = \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 0.0 \\ -1.0 \end{bmatrix} = +1.0$$

4. Angle:

$$\theta = \text{atan2}(1.0, 0.0) = \frac{\pi}{2} \approx +1.5708 \text{ radians } (+90^\circ)$$

The elbow bend parameter `l_elbow_bend` is set to $+1.5708$ radians (90°). The forearm swings down along $-Y$ and lands on the target wrist marker.

18.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def solve_limb(rig, pose, targets, upper, hinge, tip, target_keys, upper_parameters, hinge_parameter):
    """Solve a two-bone limb: shoulder-elbow-wrist, or hip-knee-ankle.

    Aiming the upper bone is not enough on its own. A hinge can only bend in one
    plane, so before bending it we have to roll the upper bone until that plane
    contains the target. The axis the elbow bends about must stand perpendicular
    to both bones – that is exactly the cross product of the two target bones,
    and it gives us the roll for free.
    """
    upper_key, middle_key, lower_key = target_keys

    wanted_upper = targets[middle_key] - targets[upper_key]
    wanted_lower = targets[lower_key] - targets[middle_key]

    # --- the upper bone: aim it, and roll it so the hinge lines up -----
    positions, rotations = rig.forward_kinematics(pose)
    current_upper = positions[rig.joint_index[hinge]] - positions[rig.joint_index[upper]]
    current_axis = rig.hinge_axis(hinge, rotations)

    wanted_axis = np.cross(wanted_upper, wanted_lower)

    if np.linalg.norm(wanted_axis) < 1e-6 * np.linalg.norm(wanted_upper) * np.linalg.norm(wanted_lower):
        # The limb is dead straight, so no plane is defined and no roll is
        # needed either. Just aim the upper bone.
        turn = shortest_arc_rotation(current_upper, wanted_upper)
    else:
        turn = rotation_between_frames(current_upper, current_axis, wanted_upper, wanted_axis)

    rig.set_rotation(pose, upper, upper_parameters, turn @ rotations[rig.joint_index[upper]], rotations)

    # --- the hinge: measure the bend directly -----
    positions, rotations = rig.forward_kinematics(pose)
    current_lower = positions[rig.joint_index[tip]] - positions[rig.joint_index[hinge]]
    axis = rig.hinge_axis(hinge, rotations)

    pose[rig.parameter_index[hinge_parameter]] = signed_angle_about_axis(
        current_lower, wanted_lower, axis)
    return pose
```

And here is the helper function from Part 1 that measures the angle:

```
def signed_angle_about_axis(from_direction, to_direction, axis):
    """How far to turn about `axis` to get from one direction to the other.

    Only the parts of the directions that lie across the axis matter, so we
    flatten both onto the plane the axis is perpendicular to, then measure the
    angle between them. The sign follows the right-hand rule about the axis.
    """
    axis = normalise(axis)

    a = np.asarray(from_direction, dtype=float)
    b = np.asarray(to_direction, dtype=float)

    flat_a = a - np.dot(a, axis) * axis
    flat_b = b - np.dot(b, axis) * axis
```

```

if np.linalg.norm(flat_a) < 1e-9 or np.linalg.norm(flat_b) < 1e-9:
    return 0.0 # a direction lies along the axis: no turn is defined

flat_a = normalise(flat_a)
flat_b = normalise(flat_b)

cosine = np.dot(flat_a, flat_b)
sine = np.dot(np.cross(flat_a, flat_b), axis)

return float(np.arctan2(sine, cosine))

```

18.4.1 Line-by-Line Breakdown

1.


```

upper_key, middle_key, lower_key = target_keys
wanted_upper = targets[middle_key] - targets[upper_key]
wanted_lower = targets[lower_key] - targets[middle_key]

```

 - Unpacks the three marker names (e.g., "left_shoulder", "left_elbow", "left_wrist").
 - Computes the 3D target vectors for the upper and lower segments.
2.


```

positions, rotations = rig.forward_kinematics(pose)
current_upper = positions[rig.joint_index[hinge]] - positions[rig.joint_index[upper]]
current_axis = rig.hinge_axis(hinge, rotations)

```

 - Runs forward kinematics to see where the limb currently sits.
 - **current_upper**: Vector from shoulder to elbow.
 - **current_axis**: Calls **rig.hinge_axis** to determine the world direction of the elbow's hinge pin.
3.


```

wanted_axis = np.cross(wanted_upper, wanted_lower)

```

 - Computes the normal vector to the target limb plane via cross product.
4.


```

if np.linalg.norm(wanted_axis) < 1e-6 * np.linalg.norm(wanted_upper) * np.linalg.norm(wanted_lower):
    turn = shortest_arc_rotation(current_upper, wanted_upper)
else:
    turn = rotation_between_frames(current_upper, current_axis, wanted_upper, wanted_axis)

```

 - Checks if the arm is dead straight (collinear).
 - If straight, aims the upper bone with **shortest_arc_rotation**.
 - If bent, computes the 3D rotation that matches both bone direction and hinge roll with **rotation_between_frames**.
5.


```

rig.set_rotation(pose, upper, upper_parameters, turn @ rotations[rig.joint_index[upper]], rotations)

```

 - Writes the upper bone's orientation into **pose** (e.g., **l_uparm_twist**, **l_uparm_ry**, **l_uparm_rz**).
6.


```

positions, rotations = rig.forward_kinematics(pose)
current_lower = positions[rig.joint_index[tip]] - positions[rig.joint_index[hinge]]
axis = rig.hinge_axis(hinge, rotations)

```

 - Re-evaluates forward kinematics so the forearm reflects the newly rotated shoulder.
 - **current_lower**: Vector from elbow to wrist.
 - **axis**: Re-queries **rig.hinge_axis** to get the final hinge pin orientation.
7.


```

pose[rig.parameter_index[hinge_parameter]] = signed_angle_about_axis(
    current_lower, wanted_lower, axis)

```

 - Computes the signed bend angle around **axis** using **signed_angle_about_axis** and writes it directly into **pose[hinge_parameter]** (e.g., **l_elbow_bend**).
8.


```

return pose

```

 - Returns the updated pose vector with the entire limb solved.

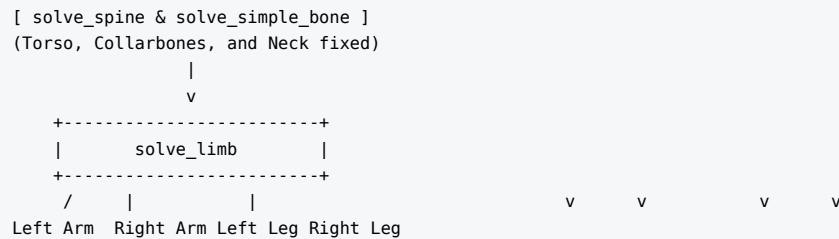
18.5 Connection to the Overall Algorithm

solve_limb is the final solver step in the pipeline. It is executed in a loop over the **LIMBS** table defined in Part 3:

```
for upper, hinge, tip, target_keys, upper_parameters, hinge_parameter in LIMBS:
    pose = solve_limb(rig, pose, targets, upper, hinge, tip,
                     target_keys, upper_parameters, hinge_parameter)
```

The loop runs in this exact anatomical order:

1. **Left Arm:** Solves shoulder orientation and elbow bend (`l_uparm, l_lowarm`).
2. **Right Arm:** Solves shoulder orientation and elbow bend (`r_uparm, r_lowarm`).
3. **Left Leg:** Solves hip orientation and knee bend (`l_upleg, l_lowleg`).
4. **Right Leg:** Solves hip orientation and knee bend (`r_upleg, r_lowleg`).



Once all four limbs are solved, all 204 parameters in `pose` have been determined.

SECTION 19

solve

19.1 Theory, Purpose, and Intuition

19.1.1 The Master Conductor

Think of the functions we have studied so far as specialist musicians in an orchestra:

- `match_bone_lengths` calibrates bone lengths.
- `match_hip_offset` aligns the hip sockets.
- `solve_root` positions and turns the pelvis.
- `solve_spine` bends and twists the torso.
- `solve_simple_bone` aims the neck, head, and collarbones.
- `solve_limb` aims and bends the arms and legs.

`solve` is the **conductor**. It does not perform low-level math itself; instead, it coordinates every specialist in the exact right order.

19.1.2 Why “Analytic” (Non-Iterative)?

Most 3D pose-fitting systems use **iterative optimization** (like Gradient Descent or Levenberg-Marquardt):

- They guess a random pose, check how far off the joints are, take a tiny nudge step, and repeat this process 50 to 100 times.
- This is slow, requires heavy computation, can get trapped in strange contorted poses (local minima), and can cause joints to jitter between video frames.

This solver is **analytic**:

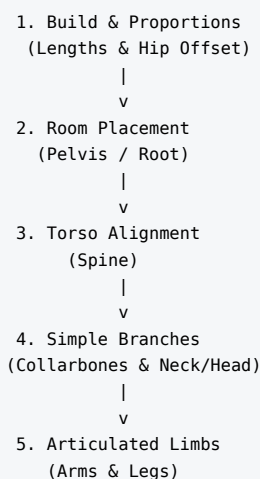
- There is no guessing, no gradient descent, and no looping back and forth.
- Every rotation is computed using direct, closed-form 3D geometry: *“here is where the bone points now, here is where it must point, rotate it.”*
- The entire pose is constructed in a single pass down the skeleton tree in just a few milliseconds.

19.1.3 Why the Sequence is Mathematically Strict

Why can’t we solve the arm first, and then solve the shoulder or pelvis? Because of the **parent-child hierarchy**:

- If you bend an elbow to reach a wrist target, and *afterwards* you rotate the pelvis or torso, that pelvis rotation will swing the entire torso and arm across the room, destroying your elbow calculation!
- By working strictly **from root to tip**, every bone is solved against a parent that has already been permanently placed in space.

THE SOLVER WATERFALL



19.2 Mathematics

19.2.1 Input Verification as Set Difference

Let the set of required 17 joint names be $\mathcal{S}_{\text{expected}}$:

$$\mathcal{S}_{\text{expected}} = \{\text{"pelvis"}, \text{"neck"}, \text{"left_shoulder"}, \dots, \text{"right_ankle"}\}$$

Let $\mathcal{K}$ be the set of keys provided in the user's `targets` dictionary. The set of missing joints $\mathcal{M}$ is the mathematical set difference:

$$\mathcal{M} = \mathcal{S}_{\text{expected}} \setminus \mathcal{K} = \{x \in \mathcal{S}_{\text{expected}} \mid x \notin \mathcal{K}\}$$

If $|\mathcal{M}| > 0$, the algorithm halts immediately.

19.2.2 State-Space Pipeline

Mathematically, the character's pose is a state vector $\mathbf{p} \in \mathbb{R}^{204}$. `solve` treats the posing process as a sequence of discrete function transformations:

1. **Initialization:**

$$\mathbf{p}_0 = \mathbf{0} \quad (\text{rest pose})$$

2. **Build calibration:**

$$\mathbf{p}_1 = T_{\text{lengths}}(\mathbf{p}_0)$$

$$\mathbf{p}_2 = T_{\text{hip_offset}}(\mathbf{p}_1)$$

3. **Root placement:**

$$\mathbf{p}_3 = T_{\text{root}}(\mathbf{p}_2)$$

4. **Spine alignment:**

$$\mathbf{p}_4 = T_{\text{spine}}(\mathbf{p}_3)$$

5. **Simple bones (Collarbones, Neck, Head):**

$$\mathbf{p}_5 = T_{\text{simple},4}(T_{\text{simple},3}(T_{\text{simple},2}(T_{\text{simple},1}(\mathbf{p}_4))))$$

6. **Limbs (Left/Right Arms, Left/Right Legs):**

$$\mathbf{p}_{\text{final}} = T_{\text{limb},4}(T_{\text{limb},3}(T_{\text{limb},2}(T_{\text{limb},1}(\mathbf{p}_5))))$$

At each step, a specialized operator overwrites specific indices of $\mathbf{p}$, leaving previously established parent parameters untouched.

19.3 Hand-Worked Mathematical Example

Let's trace how the 204-number vector $\mathbf{p}$ accumulates its values as it flows through the `solve` function:

Step	Function Executed	Parameters Modified in $\mathbf{p}$	Concrete Example Values	Physical Meaning
0	<code>rest_pose()</code>	All 204 indices	<code>[0.0, 0.0, ..., 0.0]</code>	Blank canvas (T-pose at origin).
1	<code>match_bone_lengths</code>	<code>scale_uparms</code> , <code>scale_lowlegs</code> , ...	<code>scale_uparms = +0.2</code>	Upper arm stretched by 2.0 cm.
2	<code>match_hip_offset</code>	<code>scale_hip_height</code> , <code>scale_hip_depth</code>	<code>scale_hip_height = -0.3</code>	Hip sockets lowered by 3.0 cm.
3	<code>solve_root</code>	<code>root_rx</code> , <code>ry</code> , <code>rz</code> , <code>root_tx</code> , <code>ty</code> , <code>tz</code>	<code>root_ty = 9.2</code> , <code>root_ry = 1.57</code>	Body shifted to pelvis height, turned 90°.
4	<code>solve_spine</code>	<code>spine0_rx</code> , <code>ry</code> , <code>rz_flexible</code>	<code>spine0_rx_flexible = 0.05</code>	Torso twisted so chest matches shoulders.
5	<code>solve_simple_bone</code> (Loop)	<code>l_clavicle_r*</code> , <code>neck_*</code> , <code>head_*</code>	<code>neck_bend = 0.12</code>	Neck aimed at head; collarbones aligned.
6	<code>solve_limb</code> (Loop)	<code>l_uparm_*</code> , <code>l_elbow_bend</code> , ...	<code>l_elbow_bend = 1.57</code>	Left elbow bent 90°, knee bent, etc.

By the time the function reaches `return pose`, every slider corresponding to a major anatomical joint has been filled with its exact geometric value.

19.4 Code Implementation

Here is the code from `analytic_ik.py`:

```
def solve(rig, targets):
    """Work out MHR pose parameters that match a set of 3D target keypoints.

    `targets` is a dictionary of name -> (3,) position in centimetres, +y up.
    Returns the (204,) pose parameter vector.
    """
    missing = [name for name in TARGET_NAMES if name not in targets]
    if missing:
        raise ValueError(f"missing target keypoints: {missing}")

    pose = rig.rest_pose()

    # 1. the body's build, then where it stands and which way it faces
    pose = match_bone_lengths(rig, pose, targets)
    pose = match_hip_offset(rig, pose, targets)
    pose = solve_root(rig, pose, targets)

    # 2. up the spine, then the neck and head
    pose = solve_spine(rig, pose, targets)
    for joint, tip, aim_at, parameters in SIMPLE_BONES:
        pose = solve_simple_bone(rig, pose, targets, joint, tip, aim_at, parameters)

    # 3. the four limbs
    for upper, hinge, tip, target_keys, upper_parameters, hinge_parameter in LIMBS:
        pose = solve_limb(rig, pose, targets, upper, hinge, tip,
                          target_keys, upper_parameters, hinge_parameter)

    return pose
```

19.4.1 Line-by-Line Breakdown

```
1 missing = [name for name in TARGET_NAMES if name not in targets]
   if missing:
       raise ValueError(f"missing target keypoints: {missing}")
```

- **What it does:** Uses a list comprehension to scan the input dictionary `targets` against `TARGET_NAMES`.
- **Why it is there:** If the camera system dropped tracking for even a single joint (e.g., `"left_knee"`), downstream functions would crash with a `KeyError`. Raising a clear `ValueError` alerts the user immediately.

```
2 pose = rig.rest_pose()
```

- Creates a fresh NumPy array of shape `(204,)` filled with zeros.

```
3 pose = match_bone_lengths(rig, pose, targets)
   pose = match_hip_offset(rig, pose, targets)
   pose = solve_root(rig, pose, targets)
```

- **Step 1:** Solves the build (bone lengths and hip offsets), then slides and turns the root to place the puppet's pelvis onto the human's pelvis.

```
4 pose = solve_spine(rig, pose, targets)
   for joint, tip, aim_at, parameters in SIMPLE_BONES:
       pose = solve_simple_bone(rig, pose, targets, joint, tip, aim_at, parameters)
```

- **Step 2:** Solves the torso orientation (`c_spine0`), and then loops through `SIMPLE_BONES` to aim the collarbones (`l_clavicle`, `r_clavicle`), neck (`c_neck`), and head (`c_head`).

```
5 for upper, hinge, tip, target_keys, upper_parameters, hinge_parameter in LIMBS:
    pose = solve_limb(rig, pose, targets, upper, hinge, tip,
                      target_keys, upper_parameters, hinge_parameter)
```

- **Step 3:** Loops through `LIMBS` to aim and roll the upper limbs and bend the hinges for all 4 limbs (left arm, right arm, left leg, right leg).

```
6 return pose
```

- Returns the completed 204-dimensional pose vector.

19.5 Connection to the Overall Algorithm

`solve` is the central API function called by user scripts. In `example.py`, after loading a motion-capture frame from the dataset, the user runs:

```
target_points = ik.targets_from_aspset(frames[index])
pose = ik.solve(rig, target_points)
```

From this single call, the user receives the complete `pose` vector, which can then be passed to:

1. `ik.posed_keypoints(rig, pose)`: To extract the solved 3D skeleton coordinates.
2. `ik.mesh_from_pose(rig, pose)`: To deform the 3D skin mesh into the matching pose for visualization or video rendering.